

Thursday 5:38 PM

diff between them

```
//int? x = null;
///  
//int x = 3;
```

The difference is **whether the `int` variable is allowed to contain `null`** .

1. `int? x = null;`

```
</> C#  
  
int? x = null;
```

`int?` means **nullable int**.

It is shorthand for:

```
</> C#  
  
Nullable<int> x = null;
```

So these two are exactly equivalent:

```
</> C#  
  
int? x = null;  
Nullable<int> y = null;
```

The variable can contain either:

```
</> Plain text  
  
an integer value → 5  
null → no value
```

Example:

```
</> C#  
  
int? age = null;
```

```
age = 25;    // valid  
age = null; // valid
```

2. `Nullable<int> y = null;`

```
</> C#  
  
Nullable<int> y = null;
```

This is the **long form** of `int?` .

So:

```
</> C#  
  
int? x
```

and

```
</> C#  
  
Nullable<int> x
```

mean the same thing.

Usually developers use `int?` because it's shorter and easier to read.

3. `int x = 3;`

```
</> C#  
  
int x = 3;
```

This is a **normal non-nullable integer**.

It must always contain an integer value.

```
</> C#
```

10/3/26, 7:48 PMC# Recap

```
int x = 3;    // ✓
x = 10;      // ✓
x = 0;       // ✓
x = null;    // ✗
```

Think of it like this:

Declaration	Can hold number?	Can hold null ?
int x	✓	✗
int? x	✓	✓
Nullable<int> x	✓	✓

Important: `int?` does **not** mean the object/value is useless when it's `null` . It means the variable represents an integer **that may or may not currently have a value**.

This is especially common with database columns where a value is optional, e.g.:

</> C#

```
int? age
DateTime? birthDate
decimal? discount
```

Because the database may contain `NULL` .

implicit casting and explicit casting

In **C#**, **casting** means converting a value from one type to another.

The main difference is **whether C# can safely perform the conversion automatically**.

1. Implicit casting

No code is required from you. C# automatically converts the type.

Usually this happens when converting from a **smaller range type** → **larger range type**, because there is no risk of losing information.

</> C#

```
int x = 10;
long y = x;
```

10/3/26, 7:48 PMC# Recap

Here:

</> Plain text

```
int → long
```

`long` can represent every possible `int` , so C# does it automatically.

Another example:

</> C#

```
int x = 10;
double y = x;
```

No cast needed.

Common examples

</> Plain text

```
byte → short → int → long → float → double
```

For example:

</> C#

```
int x = 100;
double y = x; // implicit
```

2. Explicit casting

Here, you must tell C# to perform the conversion using `(type)` .

Usually required when there is a possibility of **data loss**.

</> C#

```
double x = 10.5;
int y = (int)x;
```

The `(int)` is the explicit cast.

Result:

```
</> Plain text

x = 10.5
y = 10
```

The decimal part is lost.

Another example:

```
</> C#

long x = 100;
int y = (int)x;
```

You explicitly tell C#:

"Convert this long to an int ."

Why doesn't C# do this automatically?

Consider:

```
</> C#

long x = 5_000_000_000;
int y = (int)x;
```

int cannot hold 5,000,000,000 , so converting can cause **overflow/data loss**.

That's why C# requires:

```
</> C#

int y = (int)x;
```

You're explicitly accepting responsibility for the conversion.

Easy interview rule

- Implicit = safe conversion → C# does it automatically.
- Explicit = potentially unsafe/lossy conversion → you use (type) .

Important connection to your previous int? question

Nullable types add another situation:

```
</> C#

int? x = 10;
int y = x;      // ❌
```

Why? Because x might be null.

You can explicitly access the value:

```
</> C#

int y = (int)x;
```

or safely handle the null:

```
</> C#

int y = x ?? 0;
```

So don't confuse **casting** with **nullable value handling**—they are related but different concepts.

1 selection

it throws excpion or what?

Yes—but **not necessarily an exception** in this exact code.

```
</> C#

long x = 5_000_000_000;
int y = (int)x;
```

By default, an explicit numeric cast from long to int uses **unchecked conversion**.

Since 5,000,000,000 is larger than int.MaxValue (2,147,483,647), the value **overflows** and y gets an incorrect wrapped value.

10/3/26, 7:48 PMC# Recap</> C#Console.WriteLine(y);

You won't get an exception by default.

When does it throw?

If you use checked :

</> C#long x = 5_000_000_000;int y = checked((int)x);

Now C# detects the overflow and throws:

</> Plain textSystem.OverflowException

So:

Code	Result
(int)x	✗ Overflow, usually no exception
checked((int)x)	✗ OverflowException

This is an important distinction for C# interviews: **explicit casting does not automatically mean an exception will be thrown.**

1 selection

what it is printed here

For:

</> C#long x = 5_000_000_000;int y = (int)x;Console.WriteLine(y);

10/3/26, 7:48 PMC# RecapIt prints:</> Plain text705032704

Why? int can only hold values from:

</> Plain text-2,147,483,648to2,147,483,647

The explicit cast is **unchecked**, so the bits are truncated/wrapped around rather than throwing an exception.

So:

</> Plain text5,000,000,000L↓ (int)705,032,704

If you use:

</> C#int y = checked((int)x);

then you get an **OverflowException** instead of a value.

1 selection

why this number why not 2,147,483,647

Because (int) doesn't clamp the value to int.MaxValue .

It converts the number using the **lower 32 bits** of the long value.

Step by step

Your value is:

</> Plain text

```
long x = 5,000,000,000
```

An `int` has exactly **32 bits**.

`5,000,000,000` in binary needs more than 32 bits, so when you explicitly cast to `int`, only the lower 32 bits are kept.

Mathematically, the result is:

</> Plain text

```
5,000,000,000 - 2^32
```

Since:

</> Plain text

```
2^32 = 4,294,967,296
```

we get:

</> Plain text

```
5,000,000,000
-4,294,967,296
-----
705,032,704
```

So:

</> C#

```
long x = 5_000_000_000;
int y = (int)x;

Console.WriteLine(y);
```

prints:

</> Plain text

```
705032704
```

Why not **2,147,483,647** ?

Because `2,147,483,647` is simply the **maximum value** an `int` can represent.

C# doesn't say:

"The number is too big, so I'll use the maximum."

Instead, an unchecked numeric conversion effectively **wraps around modulo 2^{32}** .

If you actually want to prevent this overflow, use:

</> C#

```
int y = checked((int)x);
```

Then you'll get:

</> Plain text

```
OverflowException
```

Interview tip:

`(int)5_000_000_000` → **overflow/wraparound**

`checked((int)5_000_000_000)` → **OverflowException**

can `int` be implictly casted to `float`

Yes. ☒ `int` can be **implicitly converted to `float`** in C#.

</> C#

```
int x = 10;
float y = x;
```

No explicit cast is needed.

Why?

Because `float` has a much larger **range** than `int` :

</> Plain text

```
int → approximately ±2.1 billion
```

```
float → approximately  $\pm 3.4 \times 10^{38}$ 
```

So C# allows:

```
</> Plain text
```

```
int → float ☒ implicit
```

However, there's an important detail: `float` does not have enough precision to represent every possible `int` exactly.

For example, large integers can lose precision when converted to `float` :

```
</> C#
```

```
int x = 16_777_217;
float y = x;
```

```
Console.WriteLine(y);
```

The `float` may represent this as 16,777,216 because a `float` has only about **7 decimal digits of precision**.

So remember:

Implicit conversion doesn't necessarily mean "no information can ever be lost." It means C# considers the conversion allowed without requiring an explicit cast.

 1 selection

but how both has 32 bits

Exactly — **both are 32 bits**, but the bits are used differently.

The key is:

Number of bits $\neq$ range + precision in the same way.

`int` — 32 bits

An `int` uses all 32 bits to represent an **integer exactly**.

```
</> Plain text
```

```
int
[ sign + 31 other bits ]
```

Range:

```
</> Plain text
```

```
-2,147,483,648 → 2,147,483,647
```

So it can represent about **4.3 billion different integer values**.

`float` — also 32 bits

A `float` uses its 32 bits differently:

```
</> Plain text
```

```
[ sign | exponent | fraction ]
   1       8       23
```

So it sacrifices **precision** to get a huge **range**.

Conceptually:

```
</> Plain text
```

```
float = sign × significand × 2exponent
```

The **8-bit exponent** allows the number to become extremely large:

```
</> Plain text
```

```
≈ 3.4 × 1038
```

But because only **23 bits + an implicit bit** are used for the significant digits, it cannot represent every integer exactly.

Think of it this way

Imagine you have **32 boxes**.

For `int` :

</> Plain text

32 boxes → all dedicated to representing the exact integer

For `float` :

</> Plain text

1 box → sign
8 boxes → exponent
23 boxes → precision

So:

	int	float
Size	32 bits	32 bits
Huge range	✗	✓
Every integer exactly	✓ within its range	✗
Decimal numbers	✗	✓
Approx. max	2.1 × 10 ⁹	3.4 × 10 ³⁸

The important concept

`int` gives you **precision**.

`float` gives you **range** at the cost of **precision**.

That's why this is possible:

</> C#

int x = 2_000_000_000;
float y = x; // implicit

Even though `float` can represent numbers much larger than `int` , it **cannot represent every number between them exactly**.

so double and decimal are not good in pericsion like float?

Actually, **double** and **decimal** are more precise than **float** , but in different ways.

1. float

- 32 bits
- ~7 significant decimal digits
 - Very large range
 - Lower precision

</> C#

float x = 1.123456789f;
// roughly 1.123457

2. double

- 64 bits
- ~15–16 significant decimal digits
 - Much more precision than `float`
 - Very large range

</> C#

double x = 1.123456789123456789;
// roughly 1.1234567891234567

3. decimal

- 128 bits
- ~28–29 significant decimal digits
 - Designed for **high-precision decimal arithmetic**
 - Smaller range than `double`
 - Common for **money/financial calculations**

</> C#

decimal price = 10.25m;

Quick comparison

Type	Size	Precision	Common use
float	32-bit	~7 digits	Graphics, games
double	64-bit	~15–16 digits	Scientific/general calculations
decimal	128-bit	~28–29 digits	Money/financial
int	32-bit	Exact integers	Counting

So the general idea is:

</> Plain text

float → least precision
double → much more precision
decimal → highest decimal precision

One important interview point: `decimal` isn't simply "more precise `double` ." It uses a different representation and is particularly useful when exact decimal arithmetic matters, such as prices and financial values.

what do u mean by significant decimal digits

Significant decimal digits means the number of digits that a numeric type can reliably keep **with meaningful precision**.

Think of it as the number of **important digits**, starting from the first non-zero digit.

Example

Suppose:

</> Plain text

123.456789

This has **9 significant digits**:

</> Plain text

123456789
^^^^^^
9 significant digits

If a type can store about **7 significant digits**, like `float` , it can only reliably preserve something like:

</> Plain text

123.4567

The remaining digits are approximated.

Another example

Consider:

</> Plain text

123456789

There are **9 significant digits**.

A `float` has about **7 significant decimal digits**, so it cannot represent this exact integer.

You might get something like:

</> Plain text

123456792

The exact result depends on the binary representation.

What about zeros?

Leading zeros usually **don't count**:

</> Plain text

0.00012345

The significant digits are:

</> Plain text

12345
^^^^

5 significant digits

But zeros between or after meaningful digits can count:

```
</> Plain text

120.05
^^^^^
5 significant digits
```

Why does this matter for float ?

float has approximately 7 significant digits.

So:

```
</> Plain text

1234567    ← 7 digits → can be represented with reasonable precision
12345678   ← 8 digits → precision starts being lost
```

And this is why:

```
</> C#

float x = 16_777_217;
```

can become:

```
</> Plain text

16_777_216
```

Even though float has a huge range, it doesn't have enough precision to distinguish every integer at that magnitude.

Simple way to remember

Range = How big/small can the number be?

Precision = How many meaningful digits can I accurately represent?

That's the key difference between float , double , and decimal .

explain all things related to structs

Absolutely. Since you're preparing for C#/.NET interviews, let's cover structs from the interview perspective, from the basics to the tricky parts.

1. What is a struct ?

A struct is a value type in C# that allows you to create your own type containing data and behavior.

```
</> C#

struct Point
{
    public int X;
    public int Y;
}
```

You can use it:

```
</> C#

Point p = new Point();

p.X = 10;
p.Y = 20;
```

Think of a struct as a small custom value type.

2. Struct vs Class

This is the most important distinction.

```
</> C#

struct Point
{
    public int X;
    public int Y;
}
```

versus:

10/3/26, 7:48 PMC# Recap

</> C#
class Person
{
 public string Name;
}

The fundamental difference:

	struct	class
Type	Value type	Reference type
Assignment	Copies the value	Copies the reference
Can be null ?	✗ normally	✓
Inheritance	✗ cannot inherit from class/struct	✓
Can implement interfaces?	✓	✓
Default value	Zero/default values	null reference
Usually used for	Small values	Objects/entities

3. Value semantics

This is probably the **most important concept**.

Consider:

</> C#

struct Point
{
 public int X;
}

Then:

</> C#

Point p1 = new Point();
p1.X = 10;

10/3/26, 7:48 PMC# Recap

Point p2 = p1;

p2.X = 20;

What is:

</> C#

p1.X

?

10.

Why?

Because:

</> C#

Point p2 = p1;

creates a **copy** of the value.

Conceptually:



Changing p2 doesn't change p1 .

4. Compare with a class

Now:

```
</> C#

class Point
{
    public int X;
}
```

And:

```
</> C#

Point p1 = new Point();
p1.X = 10;

Point p2 = p1;

p2.X = 20;
```

Now:

```
</> C#

Console.WriteLine(p1.X);
```

prints:

```
</> Plain text

20
```

Because both variables refer to the **same object**.

Conceptually:

```
</> Plain text

p1 ───┐
      │
      ↓
  ┌───┐
  │ X=20 │
  └───┘
      ↑
p2 ───┐
```

So:

Struct assignment copies the value.
Class assignment copies the reference.

5. Where is a struct stored?

This is where many interview explanations become misleading.

You may hear:

"Structs are stored on the stack and classes are stored on the heap."

That's **not always true**.

A struct is a **value type**, but its actual storage depends on where it exists.

For example:

```
</> C#

Point p = new Point();
```

A local struct variable can commonly live in stack-like local storage.

But:

```
</> C#

class Person
{
    public Point Location;
}
```

Here `Location` is part of the `Person` object, so the struct is stored **inside the object on the heap**.

Therefore:

Value type ≠ always stack.
Reference type ≠ simply "heap variable".

The important concept is **value semantics**, not stack vs heap.

6. Structs cannot inherit from classes

You cannot do:

```
</> C#

struct Point : SomeClass
{
}
```

✗ Not allowed.

Structs implicitly derive from:

```
</> Plain text

System.ValueType
```

which itself derives from:

```
</> Plain text

System.Object
```

Conceptually:

```
</> Plain text

System.Object
  ↓
System.ValueType
  ↓
struct
```

But you can implement interfaces:

```
</> C#

struct Point : IComparable<Point>
{
    // implementation
}
```

✓ Allowed.

7. Can a struct contain methods?

Absolutely.

A struct isn't just a collection of fields.

```
</> C#

struct Point
{
    public int X;
    public int Y;

    public void Print()
    {
        Console.WriteLine($"{X}, {Y}");
    }
}
```

Then:

```
</> C#

Point p = new Point();

p.X = 10;
p.Y = 20;

p.Print();
```

8. Struct constructors

Modern C# allows structs to have constructors.

```
</> C#

struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

```
}
}
```

Then:

```
</> C#

Point p = new Point(10, 20);
```

9. Default value of a struct

Every struct has a **default value**.

For:

```
</> C#

struct Point
{
    public int X;
    public int Y;
}
```

you can do:

```
</> C#

Point p = default;
```

Then:

```
</> Plain text

X = 0
Y = 0
```

because the default value of `int` is `0`.

You can also write:

```
</> C#
```

```
Point p = new Point();
```

which produces the default struct value.

10. Can a struct be null?

Normally:

```
</> C#

Point p = null;
```

✗ Not allowed.

Because structs are value types.

But you can make a **nullable struct**:

```
</> C#

Point? p = null;
```

This is shorthand for:

```
</> C#

Nullable<Point> p = null;
```

This is exactly the same concept you were asking about with:

```
</> C#

int?
```

Remember:

```
</> Plain text

int?      → Nullable<int>
Point?    → Nullable<Point>
DateTime? → Nullable<DateTime>
```

11. Boxing

This is an important interview topic.

Because structs are value types, they can be **boxed** into an object.

```
</> C#

int x = 10;

object obj = x;
```

Here the `int` value is placed inside an object.

Conceptually:

```
</> Plain text

int x
↓
boxing
↓
object
```

The same applies to your own struct:

```
</> C#

Point p = new Point();

object obj = p;
```

That's **boxing**.

12. Unboxing

Going back from `object` to the value type is called **unboxing**.

```
</> C#

object obj = 10;
```

```
int x = (int)obj;
```

This requires an explicit cast.

Similarly:

```
</> C#

Point p = (Point)obj;
```

if `obj` actually contains a boxed `Point`.

13. Structs and Equals

Structs inherit behavior from `ValueType`.

For example:

```
</> C#

Point p1 = new Point { X = 10, Y = 20 };
Point p2 = new Point { X = 10, Y = 20 };
```

When comparing values, structs are designed around **value equality**.

Conceptually:

```
</> Plain text

p1 = (10,20)
p2 = (10,20)

p1 == p2
```

Whether `==` itself is available depends on whether the struct defines an appropriate `==` operator. Otherwise, use/override equality appropriately.

For your own structs, you should understand:

- `Equals()`
- `GetHashCode()`
- `IEquatable<T>`

especially if the struct will be used as a dictionary key.

14. Why shouldn't structs be huge?

Because structs have **value semantics**.

Suppose:

```
</> C#

struct HugeStruct
{
    // lots of fields...
}
```

And you do:

```
</> C#

HugeStruct a = ...;
HugeStruct b = a;
```

The entire value may need to be copied.

With a class:

```
</> C#

HugeClass b = a;
```

you're generally just copying a reference.

Therefore, structs are generally best suited to **small, lightweight values**.

Examples:

```
</> Plain text

Point
DateTime
TimeSpan
Guid
decimal
```

These are all structs.

15. Structs are useful for representing a value

This is the key design idea.

A Point :

```
</> Plain text

X = 10
Y = 20
```

is naturally a **value**.

A Person :

```
</> Plain text

Name
Address
Phone
...
```

is usually treated as an **object/entity** with identity.

So:

```
</> Plain text

Point → value
Money → value
Date → value
Temperature → value

Person → object
Order → object
Customer → object
Product → object
```

This isn't an absolute rule, but it's a useful design guideline.

16. Readonly structs

C# supports:

```
</> C#

readonly struct Point
{
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

A `readonly struct` is intended to be immutable.

Meaning:

Once created, its state shouldn't change.

This can improve safety and sometimes performance because the compiler/runtime can avoid certain defensive copies.

17. ref struct

This is a more advanced topic.

You can have:

```
</> C#

ref struct MyStruct
{
}
```

`ref struct` has special restrictions because it is designed for scenarios involving **stack-only/ref-like data**.

A famous example is:

```
</> C#

Span<T>
```

`Span<T>` is a `ref struct`.

Because of its restrictions, a `ref struct` cannot generally:

- be boxed
- be stored as a field of a normal class
- cross certain `async / await` boundaries
- be used where a heap object might be required

You don't need to memorize all restrictions initially. Just remember:

`ref struct` is an advanced stack-only/ref-like struct used for high-performance scenarios such as `Span<T>`.

18. Struct fields and reference types

A struct can contain reference-type fields.

For example:

```
</> C#

struct PersonInfo
{
    public string Name;
}
```

The struct itself is a value type, but `Name` is a reference to a string.

So don't think:

"Everything inside a struct must be stored directly."

A struct can contain references.

19. Struct passed to a method

By default, structs are passed **by value**.

```
</> C#

void Change(Point p)
{
    p.X = 100;
}
```

If:

```
</> C#

Point p = new Point();
p.X = 10;

Change(p);
```

the original `p.X` remains:

```
</> Plain text

10
```

because the method received a **copy**.

20. Using `ref`

You can explicitly pass the struct by reference:

```
</> C#

void Change(ref Point p)
{
    p.X = 100;
}
```

Then:

```
</> C#

Change(ref p);
```

Now the original struct is modified.

So:

```
</> Plain text

normal parameter
struct → copy

ref parameter
struct → reference to original
```

21. Mutable vs immutable structs

A mutable struct allows:

```
</> C#

p.X = 10;
p.X = 20;
```

An immutable struct doesn't allow its state to change after creation.

For example, conceptually:

```
</> C#

readonly struct Point
{
    public int X { get; }
    public int Y { get; }
}
```

For most custom structs, **immutability is generally preferred** because mutable structs can create confusing behavior due to copying.

22. The biggest interview traps

Trap 1

"Structs are always stored on the stack."

✗ Incorrect.

Structs are **value types**, but their physical storage depends on context.

Trap 2

"Structs cannot have methods."

✗ Incorrect.

They can have:

- fields
- properties
- methods
- constructors
- operators
- events
- interfaces

Trap 3

"Structs cannot implement interfaces."

✗ Incorrect.

They absolutely can.

```
</> C#
struct MyStruct : IDisposable
{
    public void Dispose()
    {
    }
}
```

Trap 4

"A struct can never be null."

Not exactly.

```
</> C#
Point p = null;
```



But:

```
</> C#
Point? p = null;
```

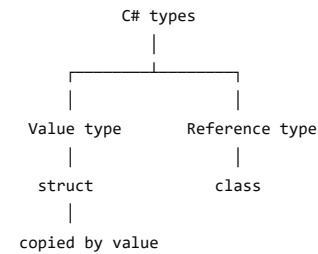


The mental model I'd recommend

Don't memorize "struct = stack, class = heap."

Instead remember:

</> Plain text



So when you see:

```
</> C#
Point p2 = p1;
```

immediately think:

A new copy of the struct's value is created.

Whereas:

```
</> C#
Person p2 = p1;
```

means:

Both variables refer to the same object.

That **value semantics vs reference semantics** distinction is the most important thing to understand about structs.

how struct is value type and has ctor and new()

Yes — this seems contradictory at first, but the key is:

new does NOT mean "reference type" and a constructor does NOT mean "class".

A `struct` is still a **value type**, even when you use `new`.

1. Without `new`

Consider:

```
</> C#

struct Point
{
    public int X;
    public int Y;
}

Point p;
```

`p` is a **value-type variable**.

You can think of it as having space for:

```
</> Plain text

p
┌───┐
│ X │
│ Y │
└───┘
```

It isn't a reference pointing to a `Point` object.

2. What does `new Point()` mean?

```
</> C#

Point p = new Point();
```

Here `new Point()` means:

Create/initialize a `Point` value using its constructor.

It does **not** mean:

"Put a `Point` object on the heap and make `p` a reference."

That's the important distinction.

Compare:

```
</> C#

class Person
{
}

Person p = new Person();
```

Here:

```
</> Plain text

p ─────────> Person object
```

`p` is a reference to an object.

But:

```
</> C#

struct Point
{
    public int X;
    public int Y;
}

Point p = new Point();
```

conceptually:

</> Plain text

p

X = 0

Y = 0

p itself contains the value.

3. Why does a struct need a constructor?

Because a struct can have initialization logic.

For example:

</> C#

struct Point

{

public int X;

public int Y;

public Point(int x, int y)

{

X = x;

Y = y;

}

}

Then:

</> C#

Point p = new Point(10, 20);

The constructor initializes the value:

</> Plain text

p

X = 10

Y = 20

It's still a value type.

4. This is the important part

new has two different-looking effects depending on the type.

For a class:

</> C#

Person p = new Person();

new creates a class object and gives you a reference to it.

For a struct:

</> C#

Point p = new Point();

new creates/initializes a **value**.

So:

</> Plain text

class + new

↓

object + reference

struct + new

↓

value

5. Then why call it a constructor?

A constructor is simply a mechanism for **initializing an instance/value**.

It isn't exclusive to classes.

Both can have constructors:

```
</> C#

class Person
{
    public Person(string name) { }
}
```

and:

```
</> C#

struct Point
{
    public Point(int x, int y) { }
}
```

The difference is the **type semantics**, not whether a constructor exists.

6. One more important thing

You might hear:

" `new` always allocates on the heap."

That's also **not correct**.

`new` is an operator used to create/initialize an instance. Whether an actual heap allocation occurs depends on the type and context.

For example:

```
</> C#

int x = new int();
```

`int` is a struct/value type.

And:

```
</> C#

DateTime date = new DateTime();
```

`DateTime` is also a struct.

Neither suddenly becomes a reference type just because `new` was used.

The rule to remember

Value type vs reference type is determined by the type definition, not by whether you use `new`.

`struct` → value type

`class` → reference type

if i want to create struct instance does i have to use new especially if there is no ctor

No. You don't always have to use `new` with a struct. This is one of the differences between structs and classes.

Suppose:

```
</> C#

struct Point
{
    public int X;
    public int Y;
}
```

1. You can declare it without `new`

```
</> C#

Point p;
```

This creates a **struct variable**, but it is **not definitely assigned yet**.

You cannot do:

```
</> C#

Console.WriteLine(p.X); // ❌
```

because `p` hasn't been initialized.

But you can assign all its fields:

`</> C#`

```
p.X = 10;
p.Y = 20;
```

Now it's definitely assigned:

`</> C#`

```
Console.WriteLine(p.X); // 10
```

2. You can use new

`</> C#`

```
Point p = new Point();
```

This initializes the struct with its **default values**:

`</> Plain text`

```
X = 0
Y = 0
```

Then you can modify it:

`</> C#`

```
p.X = 10;
p.Y = 20;
```

3. If there is a parameterized constructor

Suppose:

`</> C#`

```
struct Point
{
    public int X;
    public int Y;
```

```
public Point(int x, int y)
{
    X = x;
    Y = y;
}
```

Then to use that constructor:

`</> C#`

```
Point p = new Point(10, 20);
```

You need `new` to invoke the constructor.

Simple rule

`</> Plain text`

```
Point p;
```

➔ Declare a struct variable, but you must initialize it before reading it.

`</> Plain text`

```
Point p = new Point();
```

➔ Create the default-initialized value.

`</> Plain text`

```
Point p = new Point(10, 20);
```

➔ Create the value using your constructor.

One important modern C# detail

With structs, you may see:

`</> C#`

```
Point p = default;
```

This is another way to get the default value:

</> Plain text

```
X = 0
Y = 0
```

So **new** is not required just to have a struct variable. It is required when you want to invoke a constructor.

static and sealed class can be inherited or what and why?

No — a **sealed** class cannot be inherited, and a **static** class cannot be inherited either. But the reasons are different.

1. sealed class

</> C#

```
sealed class Animal
{
}
```

You **cannot** do:

</> C#

```
class Dog : Animal
{
}
```

✗ Compile-time error.

The purpose of **sealed** is explicitly to say:

"This class can be used, but nobody can derive another class from it."

Why would you do that?

For example, if you have a class whose behavior you don't want subclasses to modify or extend through inheritance.

</> C#

```
sealed class PaymentProcessor
{
    public void Process()
    {
    }
}
```

You can still create instances:

</> C#

```
var processor = new PaymentProcessor();
```

You just cannot inherit from it.

2. static class

</> C#

```
static class MathHelper
{
    public static int Add(int x, int y)
    {
        return x + y;
    }
}
```

You cannot inherit from it:

</> C#

```
class MyHelper : MathHelper
{
}
```

✗

But there's an additional restriction:

You **cannot** create an instance of a static class:

</> C#

You use it directly through the class:

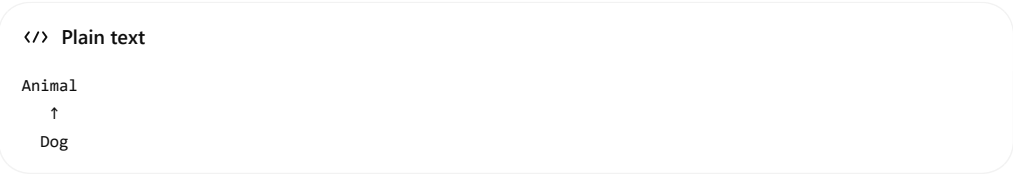
```
</> C#  
  
int result = MathHelper.Add(2, 3);
```

Why can't a static class be inherited?

Because a static class is designed to contain **only static members** and has **no instances**.

Inheritance is fundamentally about creating a derived type whose objects are also objects of the base type.

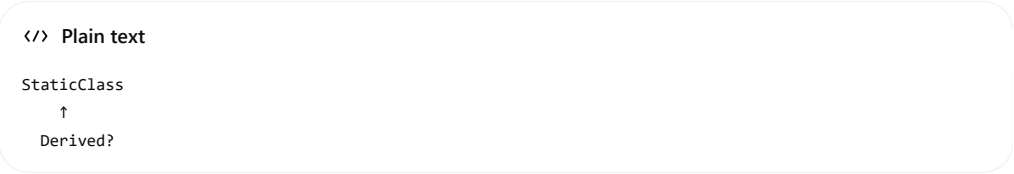
For example:



You can have:

```
</> C#  
  
Animal a = new Dog();
```

But a static class has no instances:



There's no object of the base static class to participate in that normal inheritance relationship.

Important comparison

10/3/26, 7:48 PM

	class	sealed class	static class
Can instantiate?	✓	✓	❌
Can inherit from it?	✓	❌	❌
Can have instance members?	✓	✓	❌
Can have static members?	✓	✓	✓
Purpose	Normal object	Prevent inheritance	Utility/shared functionality

One subtle point

A `sealed class` can inherit from another class:

```
</> C#  
  
class Animal  
{  
}  
  
sealed class Dog : Animal  
{  
}
```

✓ Dog inherits from Animal .

But nobody can inherit from Dog :

```
</> C#  
  
class Puppy : Dog // ❌  
{  
}
```

So `sealed` means "cannot be a base class," not "cannot inherit from another class."

class itertaors

If you mean **iterators in C#**, they're very important for .NET interviews. Let's build the concept from the beginning.

1. What is an iterator?

An **iterator** is a mechanism that allows an object to produce a sequence of values **one at a time**.

The most common keywords are:

```
</> C#

yield return
yield break
```

For example:

```
</> C#

IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

You can then:

```
</> C#

foreach (int number in GetNumbers())
{
    Console.WriteLine(number);
}
```

Output:

```
</> Plain text

1
2
3
```

2. Why not just return a List?

You could do:

```
</> C#
```

```
List<int> GetNumbers()
{
    return new List<int> { 1, 2, 3 };
}
```

But this creates and returns the **whole collection**.

With an iterator:

```
</> C#

IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

the values are produced **one at a time** as they're requested.

This is called **deferred/lazy execution**.

3. The important part: yield return

Consider:

```
</> C#

IEnumerable<int> GetNumbers()
{
    Console.WriteLine("Before 1");

    yield return 1;

    Console.WriteLine("Before 2");

    yield return 2;

    Console.WriteLine("Before 3");

    yield return 3;
}
```

If you do:

```
</> C#
var numbers = GetNumbers();
```

the method doesn't simply execute everything immediately like a normal method returning a list.

The iterator executes as the sequence is enumerated.

For example:

```
</> C#
foreach (var number in numbers)
{
    Console.WriteLine(number);
}
```

Conceptually:

```
</> Plain text
request first value
↓
execute until yield return 1
↓
return 1
↓
request next value
↓
continue from where it stopped
↓
return 2
↓
...
```

That's the important idea.

4. yield return doesn't end the method

This is a common beginner confusion.

```
</> C#
IEnumerable<int> GetNumbers()
{
    yield return 1;
```

```
yield return 2;
yield return 3;
}
```

You might think:

```
</> Plain text
yield return 1
↓
method ends
```

But that's not what happens.

Instead:

```
</> Plain text
yield return 1
↓
pause iterator
↓
caller receives 1
↓
caller asks for next
↓
resume after yield return 1
↓
yield return 2
```

The iterator **remembers its state**.

5. yield break

yield break means:

Stop producing values.

Example:

```
</> C#
IEnumerable<int> GetNumbers()
{
    yield return 1;
```

```
yield return 2;

yield break;

yield return 3;
}
```

The result is:

</> Plain text

```
1
2
```

3 will never be produced.

6. IEnumerable<T> and iterators

Usually an iterator method returns:

</> C#

```
IEnumerable<T>
```

For example:

</> C#

```
IEnumerable<int> GetNumbers()
```

`IEnumerable<T>` represents something that can be **enumerated**.

That's why this works:

</> C#

```
foreach (var number in GetNumbers())
{
}
```

7. What does `foreach` actually use?

Under the hood, `foreach` uses an **enumerator**.

Conceptually:

</> Plain text

```
IEnumerable
↓
GetEnumerator()
↓
IEnumerator
↓
MoveNext()
↓
Current
```

The important members are:

</> C#

```
MoveNext()
Current
```

For example, conceptually:

</> Plain text

```
MoveNext() → true
Current → 1
```

```
MoveNext() → true
Current → 2
```

```
MoveNext() → true
Current → 3
```

```
MoveNext() → false
```

Then the loop ends.

8. Iterator vs Enumerator

These terms are related but different.

Iterator

The mechanism/method that **produces the sequence**.

```
</> C#

IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
}
```

Enumerator

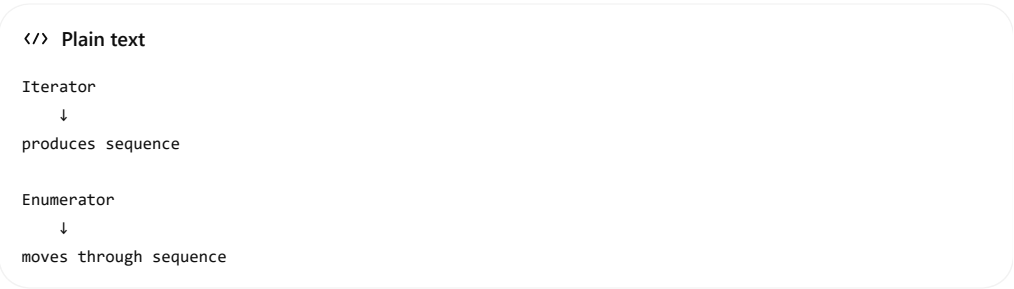
The object that **tracks the current position** while enumerating.

It exposes things like:

```
</> Plain text

MoveNext()
Current
```

Think:



9. Iterators can be infinite

This is one of the benefits of lazy execution.

```
</> C#

IEnumerable<int> GetNumbers()
{
    int i = 0;

    while (true)
    {
```

```
        yield return i++;
    }
}
```

This can theoretically produce:

```
</> Plain text

0
1
2
3
4
5
...
```

forever.

But:

```
</> C#

foreach (var n in GetNumbers())
{
    Console.WriteLine(n);
}
```

would also run forever.

You could take only some values with LINQ:

```
</> C#

var numbers = GetNumbers().Take(5);
```

Result:

```
</> Plain text

0
1
2
3
4
```

Only those requested values need to be generated.

10. Important: deferred execution

This is a major interview concept.

```
</> C#

IEnumerable<int> GetNumbers()
{
    Console.WriteLine("Executed");
    yield return 1;
}
```

Then:

```
</> C#

var numbers = GetNumbers();
```

Does "Executed" necessarily print here?

No.

The sequence hasn't necessarily been enumerated yet.

When you do:

```
</> C#

foreach (var n in numbers)
{
}
```

the iterator starts executing.

This is called **deferred execution**.

11. Why are iterators useful?

They are useful when:

Large collections

You don't need to create the entire result in memory.

Expensive calculations

You calculate each value only when requested.

Filtering

You can generate only values that satisfy some condition.

Streaming

You can process data progressively.

Infinite sequences

You can represent sequences that don't have a natural end.

12. Iterator vs returning List<T>

Consider:

```
</> C#

List<int> GetNumbers()
{
    var result = new List<int>();

    for (int i = 0; i < 1_000_000; i++)
    {
        result.Add(i);
    }

    return result;
}
```

This builds the whole list first.

Iterator:

```
</> C#

IEnumerable<int> GetNumbers()
{
    for (int i = 0; i < 1_000_000; i++)
    {
        yield return i;
    }
}
```

```
}
}
```

The second version can produce numbers **as they're requested**.

For example:

```
</> C#
GetNumbers().Take(10);
```

doesn't need to generate all one million values.

13. Very important interview distinction

Don't say:

"IEnumerable always loads everything into memory."

That's incorrect.

For example:

```
</> C#
IEnumerable<int> numbers = GetNumbers();
```

can represent a lazy iterator.

But if you do:

```
</> C#
var list = numbers.ToList();
```

then the sequence is enumerated and the results are stored in a list.

Similarly:

```
</> C#
numbers.ToArray();
```

materializes the sequence into an array.

So:

```
</> Plain text
IEnumerable
  ↓
potentially lazy

ToList()
  ↓
materialize everything
```

14. yield return and return

A method using `yield return` behaves differently from a normal method.

Normal:

```
</> C#
List<int> GetNumbers()
{
    return new List<int> { 1, 2, 3 };
}
```

The method executes and returns the complete object.

Iterator:

```
</> C#
IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

The compiler transforms the iterator method into a state-machine-like implementation that remembers where execution should resume.

You don't normally write that machinery yourself.

15. Interview question: Can an iterator method use return ?

You cannot use a normal value-returning `return` in an iterator method.

For example:

```
</> C#
IEnumerable<int> GetNumbers()
{
    yield return 1;

    return 5; // ✗
}
```

But you can use:

```
</> C#
yield break;
```

to stop iteration.

16. The mental model

Think about an iterator like a **vending machine**.

Instead of giving you the entire collection:

```
</> Plain text
[1, 2, 3, 4, 5, 6, 7, 8]
```

it gives you:

```
</> Plain text
Give me next
↓
1

Give me next
↓
```

```
2

Give me next
↓
3
```

The iterator remembers where it stopped.

That's exactly what makes:

```
</> C#
yield return
```

so useful.

The interview essentials to memorize

```
</> Plain text

Iterator
↓
Produces values one at a time

yield return
↓
Returns one value and pauses

yield break
↓
Stops iteration

IEnumerable<T>
↓
Represents an enumerable sequence

IEnumerator<T>
↓
Tracks the current position
```

If you're learning this for **C# interviews**, the next closely related topic is **IEnumerable vs IEnumerator vs ICollection vs IList vs IQueryable**, because interviewers commonly ask them together.

yes go

Yes. Let's connect them in a way that's useful for interviews.

1. The hierarchy

The easiest way to understand them is:

```
</> Plain text

IEnumerable<T>
  ↑
ICollection<T>
  ↑
IList<T>
```

And separately:

```
</> Plain text

IEnumerable<T>
  ↑
IQueryable<T>
```

IList<T> inherits from ICollection<T> , and ICollection<T> / IList<T> ultimately provide enumeration through IEnumerable<T> . [🌐 Microsoft Learn](#) +1

2. IEnumerable<T> — "I can enumerate this"

This is the most basic interface.

```
</> C#

IEnumerable<int> numbers = ...;
```

Its main purpose is:

Give me the elements one by one.

It provides GetEnumerator() . [🔗 GitHub](#)

So:

```
</> C#
```

```
foreach (var number in numbers)
{
    Console.WriteLine(number);
}
```

works.

Important

IEnumerable<T> does **not** promise that you can:

- add
- remove
- modify
- access by index

It's basically:

```
</> Plain text

"I can give you my elements."
```

3. IEnumerator<T> — "I am the thing moving through them"

Remember the iterator discussion?

IEnumerable<T> gives you an enumerator:

```
</> Plain text

IEnumerable<T>
  ↓
GetEnumerator()
  ↓
IEnumerator<T>
```

The enumerator tracks where you currently are.

Conceptually:

```
</> Plain text

Collection:
[10, 20, 30]

Enumerator:
```

```
↓  
10
```

Then:

</> Plain text

```
MoveNext()  
↓  
20
```

```
MoveNext()  
↓  
30
```

```
MoveNext()  
↓  
false
```

So:

IEnumerable<T>

"I can be enumerated."

IEnumerator<T>

"I am currently doing the enumeration."

That's why you normally use:

</> C#

```
foreach
```

instead of manually using `IEnumerator` .

4. ICollection<T> — "I am a collection"

Now we add collection-management functionality.

</> C#

```
ICollection<int> numbers = ...;
```

You get things such as:

</> Plain text

```
Count  
Add()  
Remove()  
Clear()  
Contains()  
CopyTo()
```

So the progression is:

</> Plain text

```
IEnumerable<T>  
↓  
Can enumerate  
  
ICollection<T>  
↓  
Can enumerate  
+  
Can manage the collection
```

Microsoft defines `IList<T>` as a descendant of `ICollection<T>` , with operations such as `Add` , `Remove` , `Insert` , and indexed access. [🌐 Microsoft Learn](#)

5. IList<T> — "I also have indexes"

Now we add **index-based access**.

</> C#

```
IList<int> numbers = new List<int>();
```

You can do:

</> C#

```
numbers[0]  
numbers[1]  
numbers[2]
```

And you get:

</> Plain text

```
Add()
Remove()
Clear()
Contains()
Count
+
IndexOf()
Insert()
RemoveAt()
[index]
```

So:

</> Plain text

```
IEnumerable<T>
|
| enumerate
↓
ICollection<T>
|
| add/remove/count
↓
IList<T>
|
| index/insert/remove-at
↓
```

6. Real example with List<T>

List<T> implements these interfaces.

So:

</> C#

```
List<int> list = new List<int>();
```

You can assign it to:

</> C#

```
IEnumerable<int> a = list;
ICollection<int> b = list;
IList<int> c = list;
```

But notice what happens to the available operations.

As List<T>

</> C#

```
list.Add(10);
list.Remove(10);
list[0];
```

As IEnumerable<T>

</> C#

```
a
```

You primarily treat it as:

</> Plain text

```
"something I can enumerate"
```

You don't get:

</> C#

```
a.Add(10); // ✗
a[0];      // ✗
```

The **underlying object** hasn't changed. Only the interface through which you're viewing it has changed.

This is very important in .NET.

7. Why would I use IEnumerable<T> instead of List<T>?

Suppose you have:

</> C#

```
List<Customer> customers = ...
```

and your method only needs to **read/enumerate** customers.

Instead of:

</> C#

```
void ProcessCustomers(List<Customer> customers)
```

you could use:

</> C#

```
void ProcessCustomers(IEnumerable<Customer> customers)
```

Now the method doesn't care whether the caller gives it:

</> Plain text

```
List<Customer>
Array
HashSet<Customer>
LINQ query
Iterator
etc.
```

It only requires:

"Give me something I can enumerate."

This makes the method less coupled to a specific collection type.

8. IQueryable<T> — this one is different

This is especially important for you because you're working with **EF Core**.

You might have:

</> C#

```
IQueryable<Customer> customers = _dbContext.Customers;
```

`IQueryable<T>` represents a query that can be understood by a **query provider**.

It has things like:

</> Plain text

```
Expression
Provider
ElementType
```

and it inherits from `IEnumerable`, allowing the results to eventually be enumerated. [Microsoft Learn](#)

The big difference is **where the query executes**.

9. IEnumerable vs IQueryable

This is a very common interview question.

Suppose:

</> C#

```
_dbContext.Customers
```

is an `IQueryable<Customer>`.

Then:

</> C#

```
var result = _dbContext.Customers
    .Where(x => x.Age > 18)
    .Select(x => x.Name);
```

EF Core can translate that expression into a database query.

Conceptually:

</> Plain text

```

C# LINQ
↓
Expression tree
↓
EF Core
↓
SQL
↓
Database

```

But after you materialize the data:

```

</> C#

var customers = await _dbContext.Customers.ToListAsync();

```

you now have a `List<Customer>` .

If you then do:

```

</> C#

customers.Where(x => x.Age > 18)

```

the filtering happens **in memory**.

Conceptually:

```

</> Plain text

Database
↓
all customers
↓
List<Customer>
↓
LINQ
↓
filter in memory

```

That's the key difference.

Microsoft's documentation notes that enumerating an `IQueryable` causes its expression tree to be executed, with the specific execution determined by the query provider; a provider such as an ORM can translate the expression for its underlying data source.  Microsoft Learn

10. Very important: `IEnumerable` doesn't always mean "in memory"

Be careful with this interview statement:

" `IEnumerable` means in-memory."

That's an oversimplification.

`IEnumerable<T>` describes **enumeration**.

It doesn't inherently say where the data originated.

The more useful distinction with EF Core is:

</> Plain text

`IQueryable`
→ query can be translated by the provider

`IEnumerable`
→ you're working with an enumerable sequence

Once you call:

```

</> C#

ToList()
ToArray()
ToListAsync()
ToArrayAsync()

```

you materialize the results.

11. Example you should know for interviews

Query before materialization

```

</> C#

var query = _dbContext.Customers
    .Where(x => x.Age > 18);

```

Still:

</> Plain text

```
IQueryable<Customer>
```

The query hasn't necessarily been executed yet.

Then:

</> C#

```
var result = await query.ToListAsync();
```

Now the database query executes and the results are loaded.

After materialization

</> C#

```
var result = await _dbContext.Customers
    .ToListAsync();

var adults = result
    .Where(x => x.Age > 18);
```

Now:

</> Plain text

```
Database
  ↓
List<Customer>
  ↓
Where()
  ↓
in-memory filtering
```

12. The full picture

Memorize this:



IEnumerable<T>

Can enumerate

</> Plain text

```
foreach
GetEnumerator
```

ICollection<T>

Can manage a collection

</> Plain text

```
Count
Add
Remove
Clear
Contains
```

IList<T>

Can work with indexes

</> Plain text

```
[0]
Insert
RemoveAt
IndexOf
```

IQueryable<T>

Represents a provider-backed query

</> Plain text

```

Expression
Provider
  ↓
Query provider
  ↓
Database / other data source

```

Interview answer

If the interviewer asks:

"What's the difference between IEnumerable, ICollection, IList and IQueryable?"

A good concise answer is:

" IEnumerable<T> provides enumeration. ICollection<T> adds collection-management operations such as Add, Remove and Count. IList<T> additionally provides index-based access and insertion/removal by index. IQueryable<T> is designed for provider-backed queries; it exposes an expression tree and query provider, so a provider such as EF Core can translate the query for the underlying data source."

That's a strong interview answer.

but if a class has a list and i wanna accesss iterate this list using
class index like that car[2]

Yes — a class can contain a List<T> and you can make the class itself support index access like:

```

</> C#

car[2]

```

But you need to define an **indexer** in the class.

Example

```

</> C#

class Car
{
    private List<string> parts = new List<string>
    {
        "Engine",
        "Wheel",

```

```

        "Door",
        "Seat"
    };

    public string this[int index]
    {
        get { return parts[index]; }
        set { parts[index] = value; }
    }
}

```

Now:

```

</> C#

Car car = new Car();

Console.WriteLine(car[2]);

```

Output:

```

</> Plain text

Door

```

And you can also assign:

```

</> C#

car[2] = "Window";

```

What is happening?

When you write:

```

</> C#

car[2]

```

C# translates it conceptually to:

```

</> C#

car.this[2]

```

which calls your indexer's `get` :

```
</> C#
get { return parts[index]; }
```

So the **indexer** allows an object of your class to behave like an indexed collection.

Very important distinction

If you simply have:

```
</> C#
class Car
{
    public List<string> Parts { get; set; }
}
```

You **cannot** do:

```
</> C#
car[2] ❌
```

You would have to do:

```
</> C#
car.Parts[2] ✅
```

Unless you add an **indexer**:

```
</> C#
public string this[int index] => Parts[index];
```

Then:

```
</> C#
car[2] ✅
```

This is called a **C# indexer**.

Yesterday 7:41 PM

what IEnumerable inherit from

`IEnumerable<T>` inherits from `IEnumerable` .

```
</> C#
IEnumerable<T>
    ↓
IEnumerable
```

More precisely:

```
</> C#
public interface IEnumerable<out T> : IEnumerable
```

What each one provides

`IEnumerable` provides:

```
</> C#
IEnumerator GetEnumerator();
```

`IEnumerable<T>` adds the strongly typed version:

```
</> C#
IEnumerator<T> GetEnumerator();
```

So:

```
</> C#
IEnumerable<string> names;
```

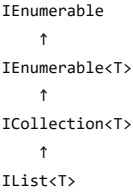
means you can iterate through it with:

```
</> C#
```

```
foreach (string name in names)
{
    // ...
}
```

Related hierarchy

</> Plain text



For example:

</> C#

```
List<int>
```

implements `IList<int>` , which inherits from `ICollection<int>` , which inherits from `IEnumerable<int>` .

Interview answer:

`IEnumerable<T>` inherits from the non-generic `IEnumerable` interface and provides strongly typed enumeration through `IEnumerator<T>` .

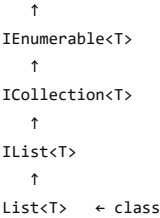
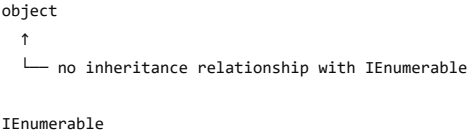
what `IEnumerable` inherit till `object`

If you mean the inheritance chain from `IEnumerable` up to `object` , there is an important point:

`IEnumerable` does NOT inherit from `object` directly

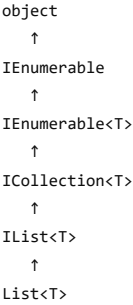
Interfaces don't inherit from `object` .

</> Plain text



`List<T>` is a **class**, so it ultimately inherits from `object` :

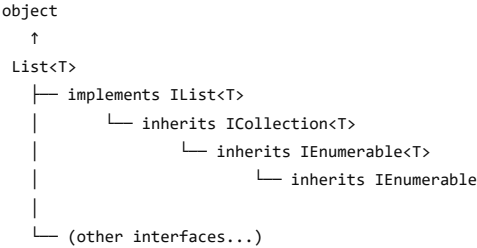
</> Plain text



But that diagram can be misleading because the arrows between `object` and the interfaces aren't inheritance arrows.

The technically correct picture is:

</> Plain text



Why?

`object` is the base class of **all classes**.

`IEnumerable` is an **interface**, not a class. It defines a contract for enumeration and doesn't have `object` as its base interface.

So if an interviewer asks:

"What does IEnumerable inherit from?"

Answer:

IEnumerable doesn't inherit from another interface; it's the base non-generic enumeration interface. **IEnumerable<T>** inherits from **IEnumerable**.

Yesterday 10:26 PM

class stack<T1 ,t2> where t2: new() plz explain generic constrians after where

Sure. In:

```
</> C#

class Stack<T1, T2> where T2 : new()
```

the part after `where` is called a **generic type constraint**.

It tells C#:

"T2 is not allowed to be just any type. It must satisfy this condition."

where T2 : new()

This specifically means:

T2 must have a public parameterless constructor.

For example:

```
</> C#

class Person
{
    public Person() { }
}
```

Then this is valid:

```
</> C#
```

```
Stack<int, Person> s;
```

because `Person` has a public parameterless constructor.

But:

```
</> C#

class Person
{
    public Person(string name) { }
}
```

Then:

```
</> C#

Stack<int, Person> s; // ❌
```

because `Person` doesn't have a parameterless constructor.

Why would we need `new()` ?

Inside the generic class, the constraint allows you to do:

```
</> C#

T2 obj = new T2();
```

Without the constraint:

```
</> C#

class Stack<T1, T2>
{
    T2 obj = new T2(); // ❌
}
```

C# doesn't know whether `T2` has a parameterless constructor.

With:

```
</> C#  
  
class Stack<T1, T2> where T2 : new()  
{  
    T2 obj = new T2(); //   
}
```

the compiler knows that `T2` is guaranteed to have one.

The important generic constraints

You'll commonly see these:

Constraint	Meaning
<code>where T : class</code>	T must be a reference type
<code>where T : struct</code>	T must be a value type
<code>where T : new()</code>	T must have a public parameterless constructor
<code>where T : SomeClass</code>	T must inherit from <code>SomeClass</code>
<code>where T : ISomething</code>	T must implement <code>ISomething</code>
<code>where T : BaseClass, Interface</code>	T must derive from the class and implement the interface

Example

```
</> C#  
  
class Repository<T> where T : Entity
```

means `T` must be `Entity` or a class derived from `Entity`.

```
</> C#  
  
class Repository<T> where T : IEntity
```

means `T` must implement `IEntity`.

And you can combine constraints:

```
</> C#  
  
class Repository<T>  
    where T : Entity, IEntity, new()
```

This means **all three conditions** must be satisfied:

- `T` derives from `Entity`
- `T` implements `IEntity`
- `T` has a public parameterless constructor

One interview detail

If you combine a class constraint, interface constraints, and `new()`, the order is generally:

```
</> C#  
  
where T : BaseClass, IInterface, new()
```

So `new()` comes **last**.

 1 selection

i don't get it

Think of `class` and `struct` here as **restrictions on what type can replace `T`**.

1. where T : class

It means:

`T` must be a **reference type**.

For example:

```
</> C#  
  
class Person { }  
class Car { }
```

Both are reference types, so:

```
</> C#  
  
class MyClass<T> where T : class
```

allows:

</> C#

```
MyClass<Person> x; // ✓
MyClass<Car> x;    // ✓
```

But:

</> C#

```
MyClass<int> x;    // ✗
MyClass<double> x; // ✗
```

because `int` and `double` are **value types**.

2. where T : struct

It means:

`T` must be a **value type**.

For example:

</> C#

```
MyClass<int> x;    // ✓
MyClass<double> x; // ✓
MyClass<bool> x;   // ✓
```

But:

</> C#

```
MyClass<Person> x; // ✗
MyClass<Car> x;    // ✗
```

because `Person` and `Car` are classes → reference types.

Think of it like a filter

</> C#

```
where T : class
```

means:

</> Plain text

T can be:

```
Person ✓
Car    ✓
string ✓
```

```
int    ✗
double ✗
bool   ✗
```

While:

</> C#

```
where T : struct
```

means:

</> Plain text

T can be:

```
int    ✓
double ✓
bool   ✓
DateTime ✓
```

```
Person ✗
Car    ✗
string ✗
```

Why would we use this?

Suppose you write:

</> C#

```
class MyClass<T> where T : class
{
    // T is guaranteed to be a reference type
}
```

The compiler can now make certain assumptions about `T` .

And:

```
</> C#  
  
class MyClass<T> where T : struct  
{  
    // T is guaranteed to be a value type  
}
```

The compiler knows `T` is a value type.

The key thing to remember:

`class` and `struct` after `where` are not declaring a class or struct. They are saying **what kind of type is allowed to be used as `T`** .

so if `t : class` then string and array is ok

Exactly. 

If you have:

```
</> C#  
  
class MyClass<T> where T : class
```

then `T` must be a **reference type**.

So these are valid:

```
</> C#  
  
MyClass<string> a;    //   
MyClass<int[]> b;    //   
MyClass<Person> c;   //   
MyClass<Car> d;      // 
```

Because:

- `string` → reference type
- `int[]` → reference type (arrays are reference types)
- `Person` → class → reference type
- `Car` → class → reference type

But:

```
</> C#  
  
MyClass<int> a;      //   
MyClass<double> b;   //   
MyClass<bool> c;     // 
```

because these are value types.

One useful exception to remember

A nullable value type like:

```
</> C#  
  
int?
```

is still a value type, so:

```
</> C#  
  
MyClass<int?> x; // 
```

where `T : class` does **not** mean "T can be null." It specifically means **T must be a reference type**.

 1 selection

why this is invallid with new() i think by default any class has paramterless or empty ctor

This is a **very important C# rule**.

You're right that a class normally gets a **parameterless constructor automatically** — but only if you **don't define any constructor yourself**.

Case 1: No constructor written

```
</> C#  
  
class Person  
{  
}
```

C# automatically provides a public parameterless constructor conceptually like:

```
</> C#

public Person()
{
}
```

So:

```
</> C#

where T : new()
```

✔ Person satisfies the constraint.

Case 2: You define a constructor

Now you have:

```
</> C#

class Person
{
    public Person(string name) { }
}
```

Because **you defined a constructor**, C# does **not** generate the parameterless constructor anymore.

So Person has only:

```
</> C#

Person(string name)
```

It does **not** have:

```
</> C#

Person()
```

Therefore:

```
</> C#
```

```
class MyClass<T> where T : new()
```

and:

```
</> C#

MyClass<Person> x;
```

✖ Invalid.

Why?

Because new() requires:

T must have a **public parameterless constructor**.

And Person doesn't have one.

If you want both

You can explicitly add it:

```
</> C#

class Person
{
    public Person()
    {
    }

    public Person(string name)
    {
    }
}
```

Now:

```
</> C#

MyClass<Person> x;
```

✔ Valid.

The rule to memorize

If you define ANY constructor, C# stops automatically providing the parameterless constructor.

So:

```
</> Plain text

class Person { }
    ↓
automatic Person() ✅

class Person
{
    Person(string name) { }
}
    ↓
automatic Person() ❌
```

This is exactly why the `new()` generic constraint is useful: it guarantees the generic code can safely do `new T()`.

explain boxing and unboxing

Sure. This is an important C# interview topic, especially because it connects **value types**, **reference types**, **structs**, and **object**.

1. First: value type vs reference type

```
</> C#

int x = 10;
```

`int` is a **value type**.

```
</> C#

object obj = x;
```

`object` is a **reference type**.

So how can an `int` be put inside an `object`?

That's where **boxing** happens.

Boxing

Boxing = converting a value type into `object` (or an interface it implements).

```
</> C#

int x = 10;

object obj = x; // Boxing
```

Conceptually:

```
</> Plain text

int x = 10
    ↓
    ↓ boxing
    ↓
object obj
    ↓
┌ 10 ─ boxed int
```

A copy of the value is placed inside an object on the managed heap, and `obj` refers to that boxed object.

Another example

```
</> C#

double price = 50.5;

object obj = price; // Boxing
```

Unboxing

Unboxing = extracting the value type from a boxed object.

```
</> C#

object obj = 10; // Boxing

int x = (int)obj; // Unboxing
```

So:

</> Plain text

```
int
↓
BOXING
↓
object
↓
UNBOXING
↓
int
```

Important: Unboxing requires the correct type

This works:

</> C#

```
object obj = 10;

int x = (int)obj; // ✔️
```

But:

</> C#

```
object obj = 10;

long x = (long)obj; // ❌ InvalidCastException
```

Why?

Because the object contains a **boxed int** , not a boxed long .

Even though:

</> C#

```
int → long
```

normally has an implicit numeric conversion, **unboxing is not numeric conversion**.

You would need:

</> C#

```
object obj = 10;

int temp = (int)obj;
long x = temp;
```

Boxing with structs

This is particularly important because **structs are value types**.

</> C#

```
struct Person
{
    public int Age;
}

Person p = new Person();

object obj = p; // Boxing
```

The `Person` value is boxed into an object.

Then:

</> C#

```
Person p2 = (Person)obj; // Unboxing
```

Boxing with interfaces

A value type can also be boxed when assigned to an interface it implements.

For example:

</> C#

```
int x = 10;
```

int implements IComparable , so this involves boxing.

Why does boxing matter?

Because it has a **performance and allocation cost**.

Compare:

```
</> C#  
  
int x = 10;  
object obj = x;
```

The boxing operation creates a boxed object.

If you're doing this millions of times, unnecessary boxing can create extra allocations and garbage-collection pressure.

This is one reason **generics** are useful.

For example:

```
</> C#  
  
ArrayList list = new ArrayList();  
list.Add(10); // Boxing
```

But:

```
</> C#  
  
List<int> list = new List<int>();  
list.Add(10); // No boxing required
```

List<int> knows that it stores int values.

Very important interview distinction

Don't think:

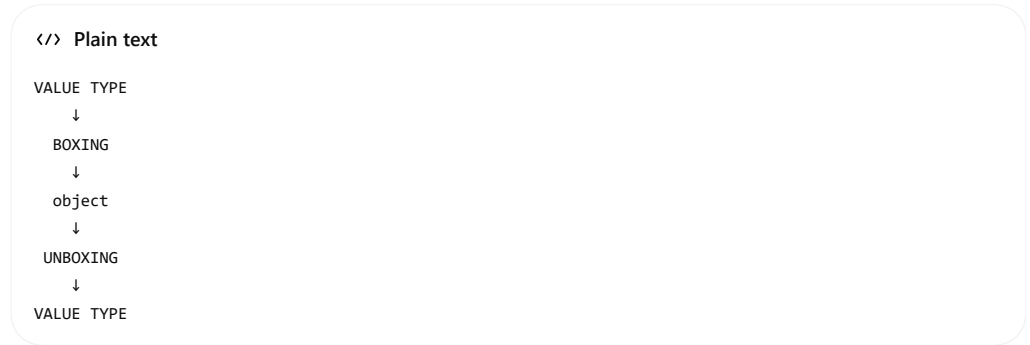
More accurately:

Boxing converts a value type to object or to an interface it implements, creating a boxed representation of that value.

And:

Unboxing extracts the value type from that boxed representation.

Easy way to remember



Example:

```
</> C#  
  
int x = 10;  
  
object obj = x; // BOXING  
  
int y = (int)obj; // UNBOXING
```

One more subtle point: **casting a value type to object is boxing, but casting object back to the value type is unboxing only when the object actually contains a box of that exact value type.**

1 selection

how to convert to interface when interface can be instancaited

The key is: **you don't create an instance of the interface**. You create an instance of the **class/struct that implements the interface**, then store it in an interface variable.

For example:

```
</> C#  
  
interface IAnimal  
{  
    void Speak();  
}  
  
class Dog : IAnimal  
{  
    public void Speak()  
    {  
        Console.WriteLine("Woof");  
    }  
}
```

You cannot do:

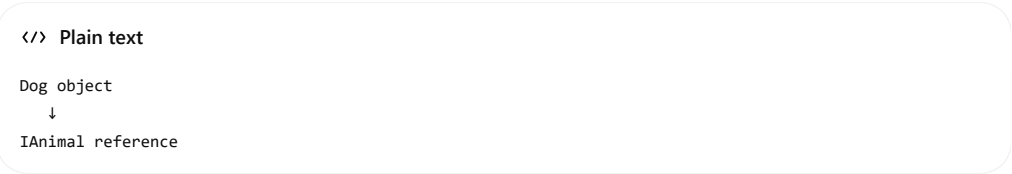
```
</> C#  
  
IAnimal animal = new IAnimal(); // ❌
```

because `IAnimal` has no implementation.

But you can do:

```
</> C#  
  
Dog dog = new Dog();  
  
IAnimal animal = dog; // ✅
```

Here:



The **object** is still a `Dog`. The variable `animal` is simply using the object through the `IAnimal` interface.

Now connect this to boxing

With a **class**:

```
</> C#  
  
Dog dog = new Dog();  
IAnimal animal = dog;
```

There is **no boxing**, because `Dog` is already a reference type.

But with a **struct**:

```
</> C#  
  
struct MyStruct : IMyInterface  
{  
    public void DoSomething() { }  
}
```

Then:

```
</> C#  
  
MyStruct x = new MyStruct();  
  
IMyInterface y = x;
```

This requires **boxing**.

Why?

Because `x` is a **value type**, while `IMyInterface` is a reference-type interface variable. C# creates a boxed object containing the struct:



So when we say:

"Boxing happens when a value type is converted to an interface it implements"

we mean something like:

```
</> C#  
  
MyStruct x = new MyStruct();  
  
IMyInterface y = x; // Boxing
```

The interface itself is never instantiated. The struct/class is instantiated; the interface variable just provides an interface-typed reference to that object.

what interafces does int and other value types impkemets by default

Yes. Value types like `int` , `double` , `bool` , `decimal` , etc. implement several interfaces automatically through the .NET type system.

For `int` , the important ones are:

```
</> Plain text  
  
int  
├─ System.Int32  
│   ├── IComparable  
│   ├── IComparable<int>  
│   ├── IEquatable<int>  
│   ├── IConvertible  
│   └─ ISpanFormattable
```

Depending on the .NET version, there are also newer numeric interfaces such as `INumber<int>` , `IBinaryInteger<int>` , etc.

The important ones for interviews

1. IComparable

Allows comparing values:

```
</> C#  
  
int x = 10;
```

```
IComparable c = x;
```

This is possible because `int` implements `IComparable` .

2. IComparable<int>

Generic, strongly typed comparison:

```
</> C#  
  
int x = 10;  
  
int result = x.CompareTo(20);
```

3. IEquatable<int>

Provides strongly typed equality comparison:

```
</> C#  
  
int x = 10;  
  
bool result = x.Equals(10);
```

4. IConvertible

Allows conversion between common types:

```
</> C#  
  
int x = 10;  
  
double d = x.ToDouble(null);
```

And this is why boxing can happen

Because `int` implements interfaces, you can write:

```
</> C#
```

```
int x = 10;

IComparable c = x;
```

`x` is a **value type**, while `IComparable` is an interface reference.

Therefore C# **boxes the `int`** .

</> Plain text

```
int x = 10
↓
BOXING
↓
IComparable c
```

What about `double` , `bool` , `decimal` ?

They similarly implement common interfaces such as:

</> Plain text

```
IComparable
IComparable<T>
IEquatable<T>
IConvertible
```

with additional type-specific/numeric interfaces depending on the type and .NET version.

For your interview, remember the key idea:

Value types can implement interfaces. When a value-type value is assigned to an interface variable, boxing occurs.

```
class Stack<T1 ,t2> where t2: new()
```

Yes. Let's break this exact declaration down:

</> C#

```
class Stack<T1, T2> where T2 : new()
```

1. `T1` and `T2`

These are **generic type parameters**.

So you could potentially have:

</> C#

```
Stack<int, Person>
```

meaning:

</> Plain text

```
T1 = int
T2 = Person
```

2. where `T2`

This says:

"I'm putting a restriction specifically on `T2` ."

Notice there is no restriction on `T1` .

So this is about `T2` , not `T1` .

3. : `new()`

This is a **constructor constraint**.

It says:

`T2` must have a **public parameterless constructor**.

Therefore:

</> C#

```
class Person
{
    public Person()
    {
    }
}
```

is valid:

```
</> C#

Stack<int, Person> s; // 
```

But:

```
</> C#

class Person
{
    public Person(string name)
    {
    }
}
```

is invalid:

```
</> C#

Stack<int, Person> s; // 
```

because there is no:

```
</> C#

Person()
```

Why is the constraint useful?

Because inside `Stack` , you can safely do:

```
</> C#

T2 item = new T2();
```

The compiler knows that **every possible** `T2` has a public parameterless constructor.

Without:

```
</> C#

where T2 : new
```

`GetType().BaseType`

`GetType().BaseType` is used to find the **immediate base class** of an object's actual runtime type.

Example

```
</> C#

class Animal
{
}

class Dog : Animal
{
}

Dog dog = new Dog();

Console.WriteLine(dog.GetType());
Console.WriteLine(dog.GetType().BaseType);
```

Output:

```
</> Plain text

Dog
Animal
```

Break it down

```
</> C#

dog.GetType()
```

returns a `Type` object representing the **actual runtime type** of `dog` .

So:

```
</> C#

dog.GetType()
```

→ `Dog`

Then:

```
</> C#  
  
dog.GetType().BaseType
```

means:

"Give me the class that `Dog` directly inherits from."

→ `Animal`

Multiple levels

```
</> C#  
  
class Animal { }  
class Mammal : Animal { }  
class Dog : Mammal { }  
  
Dog dog = new Dog();  
  
dog.GetType()           // Dog  
dog.GetType().BaseType  // Mammal  
dog.GetType().BaseType.BaseType // Animal
```

And:

```
</> C#  
  
dog.GetType().BaseType.BaseType.BaseType
```

eventually reaches:

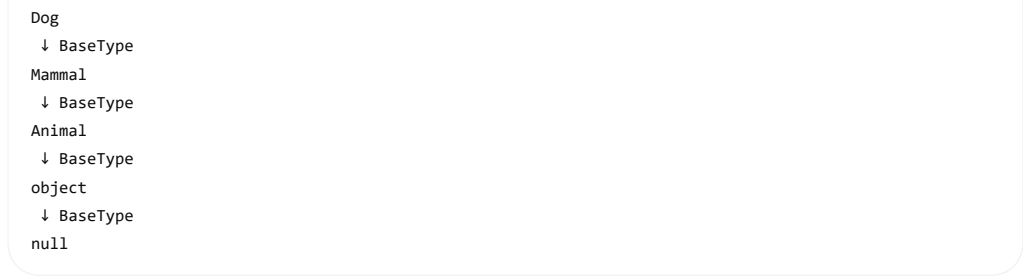
```
</> Plain text  
  
System.Object
```

because normal classes ultimately inherit from `object` .

Important distinction

`BaseType` gives you the **direct parent**, not the whole inheritance hierarchy.

```
</> Plain text
```



So when you see:

```
</> C#  
  
obj.GetType().BaseType
```

think:

"What class is the actual runtime type directly derived from?"

`gethashcode`

`GetHashCode()` is a method from `object` that returns an **integer hash code representing an object/value**.

The most important thing is to understand **why it exists**.

1. Simple example

```
</> C#  
  
string name = "Khaled";  
  
int hash = name.GetHashCode();  
  
Console.WriteLine(hash);
```

You get an integer, for example:

```
</> Plain text  
  
123456789
```

The exact number can vary between runs/types and should **not** generally be treated as a permanent ID.

2. Why do we need it?

It's mainly used by **hash-based collections** such as:

```
</> C#
```

```
Dictionary<TKey, TValue>
HashSet<T>
```

For example:

```
</> C#
```

```
HashSet<string> names = new();

names.Add("Khaled");
```

When the `HashSet` needs to determine where "Khaled" belongs, it uses its hash code as part of the lookup process.

Conceptually:

```
</> Plain text
```

```
"Khaled"
↓
GetHashCode()
↓
hash number
↓
helps find the appropriate bucket
```

This makes searching for values much faster than checking every element one by one.

3. GetHashCode() and Equals() are connected

This is **very important for interviews**.

There is a fundamental rule:

If two objects are equal according to `Equals()`, they must return the same hash code.

For example:

```
</> C#
```

```
string a = "hello";
string b = "hello";

a.Equals(b) // true
a.GetHashCode() == b.GetHashCode() // true
```

But the reverse is **not guaranteed**:

```
</> Plain text
```

```
Same hash code
↓
does NOT necessarily mean
↓
objects are equal
```

Two different objects can have the same hash code. That's called a **hash collision**.

So:

```
</> Plain text
```

```
Equals == true
⇒
same hash code

same hash code
⇏
Equals == true
```

4. Why do we override it?

Suppose you create your own class:

```
</> C#
```

```
class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

If you decide that two `Person` objects are equal based on `Id` , you should make `GetHashCode()` consistent with that equality definition.

For example, conceptually:

</> Plain text

Person 1: Id = 10
Person 2: Id = 10

Equals() → true
GetHashCode() → same value

This matters especially if you put `Person` into:

</> C#

HashSet<Person>

or use it as:

</> C#

Dictionary<Person, string>

5. Connection to structs

This is one reason I mentioned `GetHashCode()` when we discussed structs.

Structs use **value equality** by default, and if you create a custom struct that represents a value, it's common to implement:

</> C#

IEquatable<T>

and provide a consistent `GetHashCode()` implementation.

For example, conceptually:

</> Plain text

Point(10, 20)
==
Point(10, 20)

If they are considered equal, they must produce the same hash code.

Interview answer

If they ask:

What is `GetHashCode()` ?

A good answer is:

`GetHashCode()` returns an integer hash code used primarily by hash-based collections such as `Dictionary` and `HashSet` to efficiently locate objects. If two objects are equal according to `Equals()` , they must have the same hash code, but the same hash code does not guarantee equality.

explain anything related to interface

Absolutely. Since you're preparing for **C#/.NET interviews**, let's cover interfaces from the ground up and connect them to the things you've just learned: **classes, structs, boxing, generics, `IEnumerable` , `GetHashCode()` , etc.**

1. What is an interface?

An interface is a **contract** that specifies what a type must provide.

</> C#

interface IAnimal
{
 void Speak();
}

It says:

Any type that implements `IAnimal` must provide `Speak()` .

A class implements it:

</> C#

```
class Dog : IAnimal
{
    public void Speak()
    {
        Console.WriteLine("Woof");
    }
}
```

2. You cannot instantiate an interface

This is invalid:

```
</> C#

IAnimal animal = new IAnimal(); // ❌
```

Because an interface doesn't provide the concrete implementation needed to create an object.

Instead:

```
</> C#

Dog dog = new Dog();
IAnimal animal = dog;
```

The object is a `Dog` ; the variable is an `IAnimal` reference.

Think:

```
</> Plain text

      Dog object
     /      \
    /        \
   /          \
  Dog reference IAnimal reference
```

The same object can be viewed through different interfaces.

3. Interface vs class

A class can contain:

- fields

- properties
- methods
- constructors
- events
- implementation

An interface primarily defines a contract.

For example:

```
</> C#

interface IPayment
{
    void Pay(decimal amount);
}
```

Different classes can implement it differently:

```
</> C#

class CashPayment : IPayment
{
    public void Pay(decimal amount)
    {
        // cash implementation
    }
}

class CardPayment : IPayment
{
    public void Pay(decimal amount)
    {
        // card implementation
    }
}
```

Both satisfy:

```
</> Plain text

IPayment
  ↑
  └─ CashPayment
  └─ CardPayment
```

4. Why interfaces are useful

The major benefit is that your code can depend on the **contract instead of the concrete implementation**.

For example:

```
</> C#

void ProcessPayment(IPayment payment)
{
    payment.Pay(100);
}
```

You can pass:

```
</> C#

ProcessPayment(new CashPayment());
```

or:

```
</> C#

ProcessPayment(new CardPayment());
```

The method doesn't care which implementation it receives.

This is a major part of **polymorphism**.

5. Interface polymorphism

Consider:

```
</> C#

IAnimal animal = new Dog();
```

The variable type is:

```
</> Plain text

IAnimal
```

```
IAnimal
```

but the actual object is:

```
</> Plain text

Dog
```

When you call:

```
</> C#

animal.Speak();
```

the `Dog` implementation executes.

That's **runtime polymorphism**.

6. A class can implement multiple interfaces

C# doesn't allow multiple class inheritance:

```
</> C#

class C : A, B // ❌
```

But a class can implement multiple interfaces:

```
</> C#

class Dog : IAnimal, IDisposable, IComparable
{
}
```

So interfaces are one way C# provides a form of **multiple contracts**.

```
</> Plain text

IAnimal
 \
  \
   Dog
  /
```

/ IDisposable

7. Interface inheritance

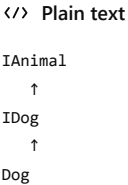
An interface can inherit from another interface.

```
</> C#  
  
interface IAnimal  
{  
    void Speak();  
}  
  
interface IDog : IAnimal  
{  
    void Fetch();  
}
```

Now a class implementing IDog must implement both:

```
</> C#  
  
class Dog : IDog  
{  
    public void Speak()  
    {  
    }  
  
    public void Fetch()  
    {  
    }  
}
```

The relationship is:



8. An interface can inherit multiple interfaces

Unlike classes, an interface can inherit multiple interfaces:

```
</> C#  
  
interface IWorker  
{  
    void Work();  
}  
  
interface IManager  
{  
    void Manage();  
}  
  
interface IEmployee : IWorker, IManager  
{  
}
```

A class implementing IEmployee must satisfy all required contracts.

9. Interface members

Modern C# interfaces can contain more than just abstract methods.

You can have:

Methods

```
</> C#  
  
interface IAnimal  
{  
    void Speak();  
}
```

Properties

```
</> C#  
  
interface IPerson  
{  
    string Name { get; set; }  
}
```

Events

```
</> C#

interface INotifier
{
    event EventHandler SomethingHappened;
}
```

Indexers

Yes, interfaces can define indexers:

```
</> C#

interface IContainer
{
    string this[int index] { get; }
}
```

Default interface implementations

Modern C# also allows an interface to provide a default implementation:

```
</> C#

interface ILogger
{
    void Log(string message)
    {
        Console.WriteLine(message);
    }
}
```

This is a more advanced feature.

10. Interfaces don't have instance fields

You don't normally define instance fields like:

```
</> C#

interface IPerson
{
```

```
int age; // ❌
}
```

Because the interface describes a contract rather than storing the object's state.

You can define a property:

```
</> C#

interface IPerson
{
    int Age { get; set; }
}
```

The implementing class owns the actual storage.

11. Interface implementation

The normal implementation is:

```
</> C#

interface IAnimal
{
    void Speak();
}

class Dog : IAnimal
{
    public void Speak()
    {
        Console.WriteLine("Woof");
    }
}
```

The class provides the implementation required by the interface.

12. Explicit interface implementation

This is an important interview topic.

You can implement an interface member explicitly:

```
</> C#
interface IA
{
    void Print();
}

interface IB
{
    void Print();
}

class Test : IA, IB
{
    void IA.Print()
    {
        Console.WriteLine("A");
    }

    void IB.Print()
    {
        Console.WriteLine("B");
    }
}
```

Now:

```
</> C#
Test t = new Test();
```

You can't simply do:

```
</> C#
t.Print(); // ❌
```

Instead:

```
</> C#
IA a = t;
a.Print();
```

prints:

```
</> Plain text
A
```

And:

```
</> C#
IB b = t;
b.Print();
```

prints:

```
</> Plain text
B
```

This allows the same method signature to have different implementations for different interfaces.

13. Interface reference vs object type

This is extremely important.

```
</> C#
Dog dog = new Dog();

IAnimal animal = dog;
```

`animal` is an **interface reference**.

But the actual object is still:

```
</> Plain text
Dog
```

So:

```
</> C#
animal.GetType()
```

returns:

```
</> Plain text
```

```
Dog
```

Not:

```
</> Plain text
```

```
IAAnimal
```

Remember our previous discussion:

```
</> C#
```

```
animal.GetType().BaseType
```

would give `Dog` 's base class.

14. Interface and casting

You can cast between compatible interfaces/types.

```
</> C#
```

```
Dog dog = new Dog();
```

```
IAAnimal animal = dog;
```

That's an implicit reference conversion because `Dog` implements `IAAnimal` .

You can also explicitly cast:

```
</> C#
```

```
Dog dog2 = (Dog)animal;
```

because the actual object is a `Dog` .

15. Interface and `is`

You can check whether an object implements an interface:

```
</> C#
```

```
if (obj is IAnimal)
{
    // obj implements IAnimal
}
```

Modern C# commonly uses pattern matching:

```
</> C#
```

```
if (obj is IAnimal animal)
{
    animal.Speak();
}
```

16. Interface and `as`

You can also use:

```
</> C#
```

```
IAAnimal animal = obj as IAnimal;
```

If `obj` doesn't implement `IAAnimal` :

```
</> Plain text
```

```
animal = null
```

instead of throwing an exception.

17. Interfaces and structs — important connection to boxing

A struct can implement an interface.

```
</> C#
```

```
interface IAnimal
{
    void Speak();
}
```

```
struct Dog : IAnimal
{
    public void Speak()
    {
    }
}
```

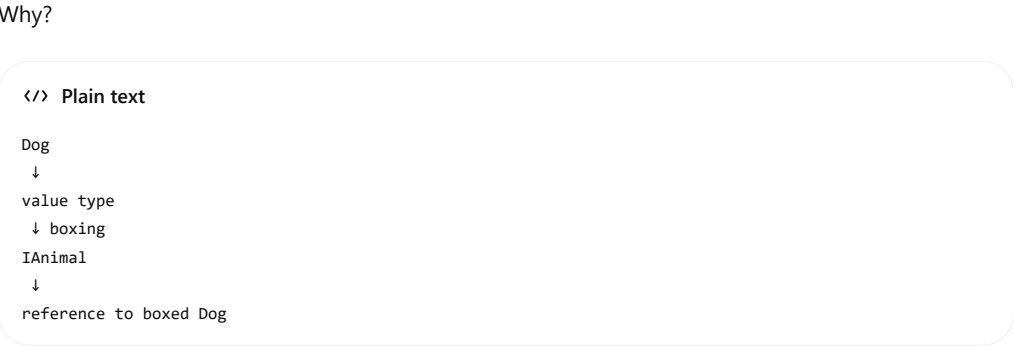
Then:

```
</> C#

Dog dog = new Dog();

IAnimal animal = dog;
```

This causes **boxing**.



This connects directly to your previous question.

With a class:

```
</> C#

Dog dog = new Dog();
IAnimal animal = dog;
```

there's no boxing because `Dog` is already a reference type.

18. Interfaces and `object`

Remember:

```
</> C#

object
```

is the base class of all normal .NET types.

Interfaces are different.

An interface doesn't inherit from `object`.

But an object that implements an interface can be referenced through `object` or the interface:

```
</> C#

Dog dog = new Dog();

object obj = dog;
IAnimal animal = dog;
```

These are different **reference views of the same object**.

19. Interfaces and dependency injection

This is extremely important in **ASP.NET Core**.

You might have:

```
</> C#

interface IEmailService
{
    void SendEmail();
}
```

Implementation:

```
</> C#

class EmailService : IEmailService
{
    public void SendEmail()
    {
    }
}
```

Then dependency injection can be configured conceptually as:

</> Plain text

IEmailService → EmailService

Your controller/service depends on:

</> C#

IEmailService

rather than:

</> C#

EmailService

This gives you **loose coupling**.

It also makes it easier to replace implementations and test code.

This is one of the main reasons you'll see interfaces everywhere in .NET projects:

</> Plain text

IAppDbContext
IUserService
IEmailService
IRepository
ILogger

20. Interfaces and abstraction

Interface:

</> C#

```
interface IPaymentService
{
    void Pay(decimal amount);
}
```

Implementation:

</> C#

```
class StripePaymentService : IPaymentService
{
    public void Pay(decimal amount)
    {
        // Stripe logic
    }
}
```

The rest of your application only needs to know:

</> Plain text

```
IPaymentService
↓
Pay()
```

It doesn't need to know how Stripe works.

That's **abstraction**.

21. Interface vs abstract class

This is a very common interview question.

Interface

</> C#

```
interface IAnimal
{
    void Speak();
}
```

Primarily defines a contract.

A class can implement **multiple interfaces**.

Abstract class

</> C#

```
abstract class Animal
{
    public string Name { get; set; }

    public abstract void Speak();
}
```

Can contain:

- fields
- constructors
- implemented methods
- abstract methods
- properties
- state

A class can inherit from only **one class**.

</> Plain text

```
class Dog : Animal // ✓
class Dog : Animal, IDog // ✓

class Dog : Animal, Car // ✗
```

22. IEnumerable is an interface

This connects directly to what we discussed earlier.

When you write:

</> C#

```
IEnumerable<int> numbers;
```

you're saying:

I don't care about the concrete collection. I only need something that provides enumeration.

For example:

</> C#

```
List<int> numbers = new();
```

```
IEnumerable<int> enumerable = numbers;
```

List<int> implements IEnumerable<int> .

That's why you can pass a List<int> to a method expecting IEnumerable<int> .

23. Interface naming convention

In C#, interfaces conventionally start with:

</> Plain text

```
I
```

Examples:

</> Plain text

```
IEnumerable
IEnumerable<T>
IList<T>
ICollection<T>
IDisposable
IComparable
IEquatable<T>
```

It's a **naming convention**, not a language requirement.

You could technically write:

</> C#

```
interface Animal
{
}
```

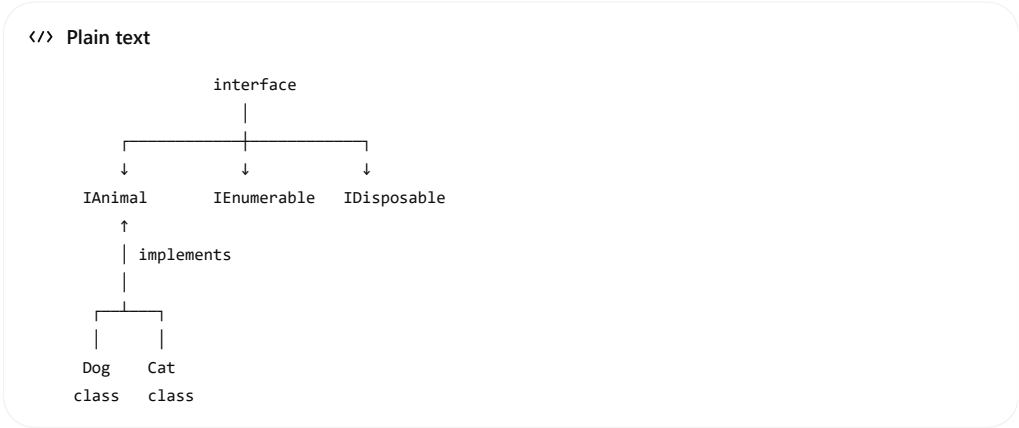
but the conventional name would be:

</> C#

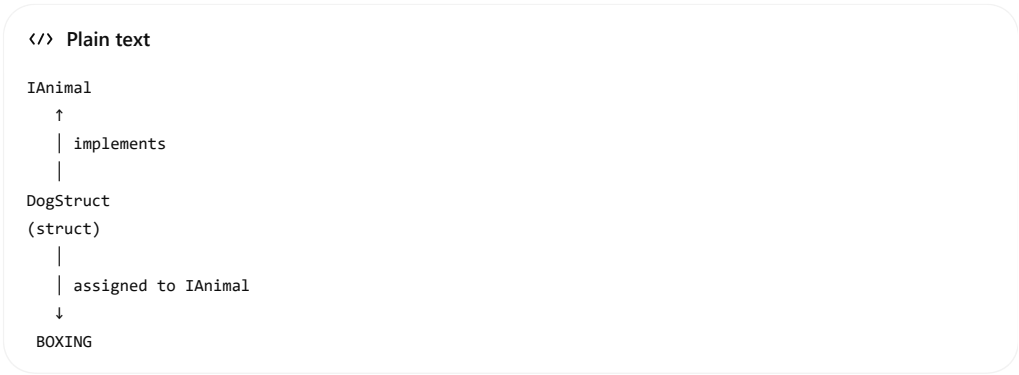
```
interface IAnimal
{
}
```

24. The big picture

You can connect almost everything we've discussed:



And a struct can also implement an interface:



The interview definition I'd memorize

An interface is a contract that defines members a type must provide. Classes and structs can implement interfaces, a type can implement multiple interfaces, and an interface reference allows polymorphism and loose coupling.

These are different **reference views of the same object**. explain this in details especially behind the scenes in the memory stack and heap level

Yes. This is one of the most important concepts to understand properly. The phrase "**different reference views of the same object**" is really about **references, the object, and the variable's compile-time type**.

Let's go step by step.

1. Start with a simple class

</> C#

```
class Dog
{
    public string Name { get; set; }

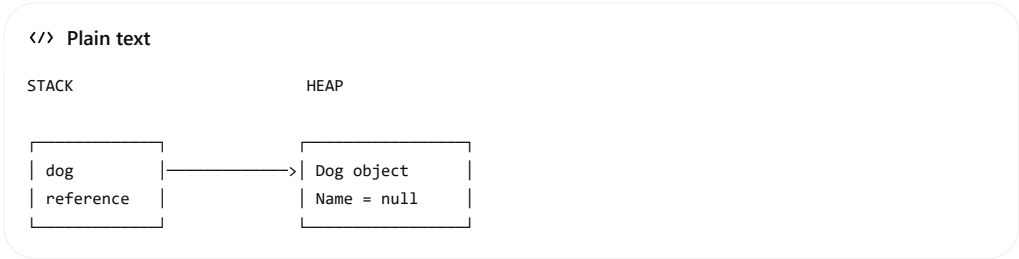
    public void Bark()
    {
        Console.WriteLine("Woof");
    }
}
```

Now:

</> C#

```
Dog dog = new Dog();
```

Conceptually, memory looks like:



Important:

- dog** is not the object itself.
- dog is a variable containing a **reference to the object**.

The actual Dog object is allocated on the managed heap.

2. Now introduce an interface

```
</> C#

interface IAnimal
{
    void Bark();
}
```

And:

```
</> C#

class Dog : IAnimal
{
    public void Bark()
    {
        Console.WriteLine("Woof");
    }
}
```

Now:

```
</> C#

Dog dog = new Dog();

IAnimal animal = dog;
```

What happened?

You did **not** create another Dog.

You already had:

```
</> Plain text

Dog object
```

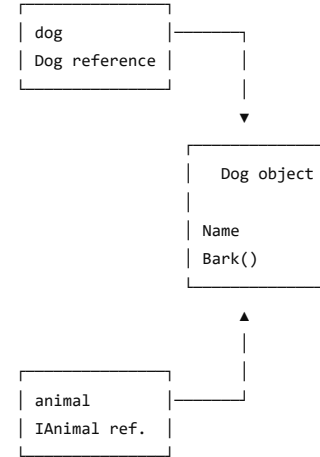
You simply created another reference to the **same object**.

Conceptually:

</> Plain text

STACK

HEAP



There is **one object**.

There are **two references** pointing to it.

3. Why call them "different views"?

Because the variables have different **compile-time types**.

</> C#

```
Dog dog = new Dog();
IAnimal animal = dog;
```

For the compiler:

</> Plain text

```
dog
↓
Dog
```

So through `dog`, you can access Dog's members.

</> C#

```
dog.Bark();
```

But through:

</> Plain text

```
animal
↓
IAnimal
```

you can only access members exposed by `IAnimal` .

For example:

</> C#

```
interface IAnimal
{
    void Bark();
}

class Dog : IAnimal
{
    public void Bark() { }

    public void Run() { }
}
```

You can do:

</> C#

```
dog.Run();      // ✔️
animal.Bark();  // ✔️
```

But:

</> C#

```
animal.Run();   // ❌
```

Why?

Because `animal` is an `IAnimal` reference.

The compiler only gives you the `IAnimal` view of the object.

4. But the object didn't change

This is extremely important.

After:

</> C#

```
Dog dog = new Dog();
IAnimal animal = dog;
```

the object is still:

</> Plain text

Dog

It didn't become an `IAnimal` .

You can prove this:

</> C#

```
Console.WriteLine(dog.GetType());
Console.WriteLine(animal.GetType());
```

Both report:

</> Plain text

Dog

because `GetType()` checks the **actual runtime object**, not the variable's compile-time type.

So:

</> Plain text

Variable type	Runtime object type
<code>IAnimal</code>	<code>Dog</code>

```
Dog dog      —————> Dog
IAnimal animal —————> Dog
```

5. What does "reference" actually mean?

A reference is essentially a way for the runtime to locate an object.

You can think of it conceptually as:

```
</> Plain text

reference
  ↓
"where is this object?"
```

For example:

```
</> C#

Dog dog = new Dog();
```

Conceptually:

```
</> Plain text

dog = 0x12345678
```

where that value represents some location/handle associated with the object.

But don't take this as saying a C# reference is literally just a raw memory address.

The CLR/GC manages object references, and the actual implementation details are more complicated.

For learning purposes, however:

```
</> Plain text

reference → identifies/points to an object
```

is the right mental model.

6. What happens when you do this?

```
</> C#

IAnimal animal = dog;
```

There is no:

```
</> Plain text

new Dog()
```

So there is no second object.

You're essentially copying the reference:

```
</> Plain text

dog
  ↓
reference A ————┐
                  ↓
                  Dog object
                  ↑
reference B ————┐
  ↓
animal
```

Both references identify the same object.

7. You can prove they're the same object

Suppose:

```
</> C#

class Dog
{
    public string Name { get; set; }
}
```

Then:

10/3/26, 7:48 PMC# Recap

```
</> C#  
  
Dog dog = new Dog();  
  
IAnimal animal = dog;
```

If `IAnimal` exposes a property or method that changes the object, both references observe that same underlying object.

For example:

```
</> C#  
  
dog.Name = "Max";
```

Now accessing the object through another reference sees:

```
</> Plain text  
  
Name = "Max"
```

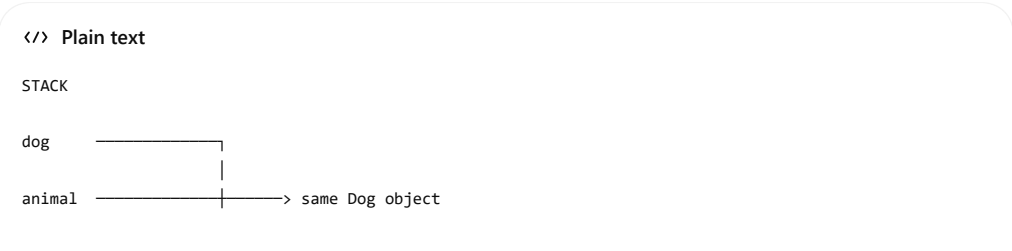
because there is only **one Dog object**.

8. What about object ?

Now we can have:

```
</> C#  
  
Dog dog = new Dog();  
  
IAnimal animal = dog;  
object obj = dog;
```

Now there are three variables:



10/3/26, 7:48 PMC# Recap

```
obj  _____
```

Three references.

One object.

This is what I meant by:

different reference views of the same object.

They all identify the same object, but the compiler sees them as different types:

```
</> Plain text  
  
dog    → Dog  
animal → IAnimal  
obj    → object
```

9. Why does obj behave differently?

Suppose:

```
</> C#  
  
class Dog  
{  
    public void Bark() { }  
}
```

Then:

```
</> C#  
  
Dog dog = new Dog();  
object obj = dog;
```

This works:

```
</> C#  
  
dog.Bark(); // ✓
```

But:

```
</> C#  
obj.Bark(); // ❌
```

Why?

Because `obj` is declared as:

```
</> C#  
object
```

and `object` doesn't define `Bark()` .

You can cast it:

```
</> C#  
((Dog)obj).Bark();
```

Now you're telling the compiler:

"I know this object is actually a Dog."

10. Interface casting works similarly

Suppose:

```
</> C#  
Dog dog = new Dog();  
  
IAnimal animal = dog;
```

Now:

```
</> C#  
animal.Bark();
```

works because `Bark()` is part of `IAnimal` .

But if Dog has:

```
</> C#  
  
public void Run()  
{  
}
```

then:

```
</> C#  
animal.Run(); // ❌
```

You can cast back:

```
</> C#  
  
Dog dog2 = (Dog)animal;  
dog2.Run();
```

You're changing the **view/reference type**, not creating another object.

11. What is actually stored on the stack?

Be careful here because people often say:

"Objects are on the heap and references are on the stack."

That's useful for beginners but **not universally true**.

For this local-variable example:

```
</> C#  
Dog dog = new Dog();
```

you can conceptually picture:

```
</> Plain text
```

```

STACK
dog reference
  |
  ▼
HEAP
Dog object

```

But references can themselves be fields inside heap objects.

Example:

```

</> C#

class Person
{
    public Dog Dog { get; set; }
}

```

Then:

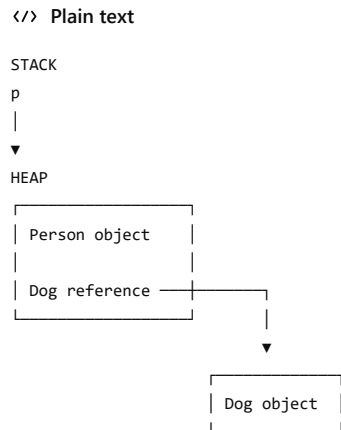
```

</> C#

Person p = new Person();
p.Dog = new Dog();

```

Conceptually:



The `Dog` reference is now **inside a heap object**.

So don't memorize:

reference = stack

Instead remember:

A reference identifies an object; where the reference is stored depends on the context.

12. Now let's look at the interface at runtime

This part is a little deeper.

When you write:

```

</> C#

IAnimal animal = dog;

```

the CLR knows:

```

</> Plain text

Actual object:
Dog

Supported interfaces:
IAnimal

```

The runtime has type metadata describing the `Dog` type and the interfaces it implements.

Conceptually:

```

</> Plain text

Dog type information

Dog
├─ base class: object
├─ interfaces:
│   └─ IAnimal
├─ methods:
│   └─ Bark()
└─ fields/properties...

```

So when you call:

10/3/26, 7:48 PMC# Recap

</> C#

animal.Bark();

the runtime can dispatch the call to the implementation provided by `Dog` .

This is part of **runtime polymorphism**.

13. This is why this works

</> C#

IAnimal animal = new Dog();

animal.Bark();

The compiler knows:

</> Plain text

IAnimal has Bark()

The runtime knows:

</> Plain text

actual object = Dog
Dog implements IAnimal
Dog provides implementation of Bark()

So the `Dog` implementation executes.

14. Compare with inheritance

The same concept exists with a base class:

</> C#

class Animal
{
 public virtual void Speak() { }
}

10/3/26, 7:48 PMC# Recap

class Dog : Animal
{
 public override void Speak() { }
}

Then:

</> C#

Animal animal = new Dog();

Again:



The variable is an `Animal` reference, but the actual object is a `Dog` .

So:

</> C#

animal.Speak();

calls `Dog`'s override.

Interfaces provide a similar polymorphic capability without requiring class inheritance from a particular base class.

15. One object can have many views

This is the big picture:

</> C#

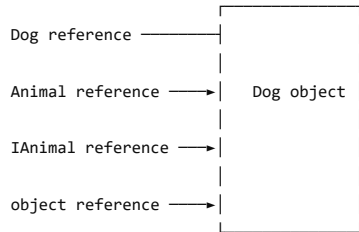
Dog dog = new Dog();

Animal animal = dog;

```
IAnimal iAnimal = dog;
object obj = dog;
```

Conceptually:

</> Plain text



4 references, 1 object.

Each reference gives you a different compile-time view:

</> Plain text

```
Dog      → Dog members
Animal   → Animal members
IAnimal  → IAnimal members
object   → object members
```

But they all refer to the same runtime object.

16. Very important: interface conversion vs boxing

This connects directly to your previous question.

Class:

</> C#

```
Dog dog = new Dog();
IAnimal animal = dog;
```

`Dog` is a reference type.

</> Plain text

```
Dog reference
  ↓
Dog object
```

No boxing.

Struct:

</> C#

```
DogStruct dog = new DogStruct();
IAnimal animal = dog;
```

`DogStruct` is a value type.

Converting it to an interface can require boxing:

</> Plain text

```
DogStruct value
  ↓
BOXING
  ↓
boxed DogStruct object
  ↓
IAnimal reference
```

So the memory situation is fundamentally different.

17. The best mental model

Don't think:

"I have a Dog, then I converted it into an IAnimal."

Think:

"I have one Dog object. I created an `IAnimal` -typed reference that points to that same object."

That's the concept behind:

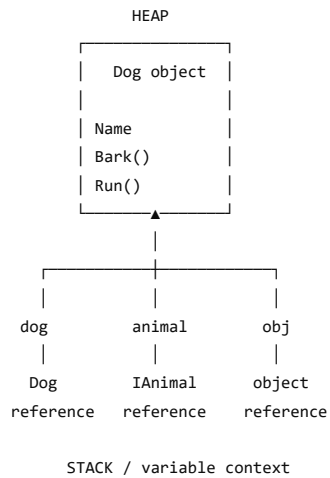
different reference views of the same object.

And this is the foundation for:

- polymorphism
- interfaces
- dependency injection
- abstraction
- loose coupling
- virtual/override dispatch
- IEnumerable
- IDisposable
- many ASP.NET Core patterns

One final diagram to memorize

</> Plain text



The **types of the references are different**, but the **runtime object is the same**.

diff between interface method and abstratc metod

The core difference is:

An abstract method belongs to an abstract class. An interface method belongs to an interface.

Both can require a derived/implementing type to provide an implementation, but they are used differently.

1. Abstract method

</> C#

```
abstract class Animal
{
    public abstract void Speak();
}
```

A derived class must implement it:

</> C#

```
class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Woof");
    }
}
```

Important: an abstract method uses `override` :

</> C#

```
public override void Speak()
```

2. Interface method

</> C#

```
interface IAnimal
{
    void Speak();
}
```

A class implements it:

</> C#

```
class Dog : IAnimal
{
    public void Speak()
    {
        Console.WriteLine("Woof");
    }
}
```

Notice:

</> Plain text

Abstract method → override
Interface method → implementation

3. The biggest difference: class inheritance

A class can inherit from **only one class**:

</> C#

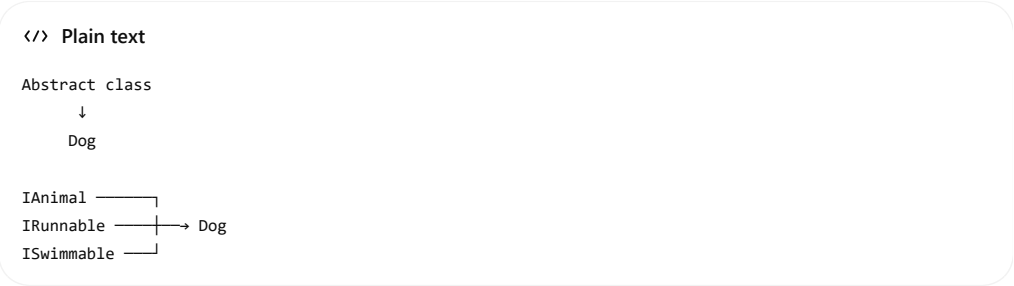
class Dog : Animal
{
}

But it can implement **multiple interfaces**:

</> C#

class Dog : IAnimal, IRunnable, ISwimmable
{
}

So:



This is one of the biggest reasons interfaces are useful.

4. Abstract class can contain state

An abstract class can have fields:

</> C#

abstract class Animal
{
 protected string name;

 public Animal(string name)
 {
 this.name = name;
 }

 public abstract void Speak();
}

The derived class gets that state:

</> C#

class Dog : Animal
{
 public Dog(string name) : base(name)
 {
 }

 public override void Speak()
 {
 Console.WriteLine(name);
 }
}

An interface traditionally doesn't provide instance state:

</> C#

interface IAnimal
{
 void Speak();
}

Think:

</> Plain text

Abstract class → contract + shared state/implementation

Interface → primarily contract/capability

Modern interfaces can have default implementations and static members, so "interface = only abstract methods" is no longer completely accurate.

5. Abstract class can have constructors

```
</> C#
abstract class Animal
{
    protected Animal(string name)
    {
        // initialization
    }

    public abstract void Speak();
}
```

Then:

```
</> C#
class Dog : Animal
{
    public Dog(string name) : base(name)
    {
    }
}
```

An interface doesn't have an instance constructor that implementing classes call.

6. Abstract class can provide common implementation

For example:

```
</> C#
abstract class Animal
{
    public void Sleep()
    {
        Console.WriteLine("Sleeping...");
    }
}
```

```
public abstract void Speak();
}
```

Now every animal automatically gets `Sleep()` .

The derived class only needs to implement `Speak()` .

With an interface:

```
</> C#
interface IAnimal
{
    void Speak();
}
```

the implementing class normally provides the implementation.

Modern C# does support **default interface implementations**, but they have different semantics from shared state/implementation in an abstract base class.

7. Access modifiers

Abstract methods can have access modifiers:

```
</> C#
public abstract void Speak();
protected abstract void Move();
```

Interface members are generally public as part of the interface contract, though modern C# has more advanced accessibility/default-member rules.

For a basic interview answer, remember:

Interface members represent the public contract; abstract class members can have normal class accessibility.

8. A very important polymorphism difference

Suppose:

```
</> C#  
  
abstract class Animal  
{  
    public abstract void Speak();  
}  
  
interface IAnimal  
{  
    void Speak();  
}
```

Both can give you:

```
</> C#  
  
Animal animal = new Dog();  
IAnimal iAnimal = new Dog();
```

Both allow polymorphism.

But their purpose is different.

Abstract class

You're saying:

"Dog is an Animal, and I want to share common Animal behavior/state."

Interface

You're saying:

"Dog can perform this capability/contract."

For example:

```
</> Plain text  
  
Animal  
├─ Dog  
├─ Cat  
└─ Horse
```

versus:

```
</> Plain text
```

```
IFlyable  
├─ Bird  
├─ Airplane  
└─ Drone
```

A Bird might be an Animal and IFlyable .

An airplane isn't an Animal , but it can still implement IFlyable .

That's a major strength of interfaces.

9. Interview comparison

	Abstract class	Interface
Can have abstract methods?	✓	✓
Can have implemented methods?	✓	✓ modern C#
Can have instance fields/state?	✓	✗ traditionally; interfaces don't provide instance fields
Can have constructor?	✓	✗ instance constructor
Multiple inheritance?	✗ one base class	✓ multiple interfaces
Method implementation uses	override	implementation
Main purpose	Common base + shared behavior/state	Contract/capability
Can be instantiated directly?	✗	✗

The easiest way to remember

Abstract class:

"What you ARE + shared implementation."

```
</> C#
```

```
abstract class Animal
```

Interface:

"What you CAN DO."

```
</> C#  
  
interface IFlyable  
interface ISwimmable  
interface IPayable
```

So in a real .NET project, you might have:

```
</> Plain text  
  
abstract class Employee  
    ↓  
    Developer  
  
IEmailSender  
IPayable  
INotifiable  
    ↓  
    Developer
```

The `Employee` abstract class represents the **common identity/base**, while the interfaces represent **capabilities/contracts**.

so interface methods with no implemetatipn is abstract by default

Yes — **conceptually, an interface method with no implementation is abstract**, but there is an important C# terminology distinction:

```
</> C#  
  
interface IAnimal  
{  
    void Speak();  
}
```

`Speak()` has **no implementation**, so the implementing class must provide one:

```
</> C#
```

```
class Dog : IAnimal  
{  
    public void Speak()  
    {  
        Console.WriteLine("Woof");  
    }  
}
```

But don't say they're exactly the same

Interface method	Abstract method
Declared in an <code>interface</code>	Declared in an <code>abstract class</code>
No implementation by default	No implementation
Implementing class provides implementation	Derived class uses <code>override</code>
No <code>abstract</code> keyword needed	Must use <code>abstract</code>
Implemented normally as <code>public</code>	Must be overridden

For example:

```
</> C#  
  
abstract class Animal  
{  
    public abstract void Speak();  
}
```

versus:

```
</> C#  
  
interface IAnimal  
{  
    void Speak();  
}
```

So for an interview, you can say:

An interface method without a body represents a contract that the implementing type must fulfill; conceptually it behaves like an abstract member, but it is not declared using the

abstract keyword.

One modern C# nuance: **interfaces can now have default implementations**, so not every interface method must be abstract/no-body anymore.

1 selection

any method or just private and protected methods

If you're asking about **default implementations in interfaces**: they can be **public or private**, but there are rules.

1. Public/default interface method

```
</> C#

interface IAnimal
{
    void Speak()
    {
        Console.WriteLine("Animal sound");
    }
}
```

This is a **default implementation**. A class implementing `IAnimal` does **not have to** implement `Speak()` .

The method is callable through the interface:

```
</> C#

IAnimal animal = new Dog();
animal.Speak();
```

2. Private interface method

```
</> C#

interface IAnimal
{
    void Speak()
    {
        Prepare();
    }

    private void Prepare()
```

```
{
    Console.WriteLine("Preparing...");
}
```

A private interface method **must have an implementation** and is only usable **inside the interface**. It cannot be called through `IAnimal` .

3. Protected?

Interfaces don't have protected members in the normal class sense.

So the important rule is:

Interface default implementations can be public (or explicitly interface-member forms) and private helper methods can also have implementations. Interfaces don't use protected instance members like classes do.

For interview purposes, the main thing to remember is: **modern C# interfaces can contain implemented methods, not only abstract/no-body methods.**

explain enum and flagged enums

Sure. **Enums** and especially **[Flags] enums** are important C# interview topics.

1. What is an enum?

An **enum** is a **value type used to represent a fixed set of named constants**.

Instead of using meaningless numbers:

```
</> C#

int status = 2;
```

you can use:

```
</> C#

enum Status
{
    Pending,
    Approved,
    Rejected
}

Status status = Status.Approved;
```

By default, the values are:

</> Plain text

```
Pending = 0
Approved = 1
Rejected = 2
```

So internally, `Status.Approved` is still an integer value, but the enum gives that value a meaningful name.

2. You can specify the values yourself

</> C#

```
enum Status
{
    Pending = 10,
    Approved = 20,
    Rejected = 30
}
```

This is useful when the numeric values have a specific meaning or must match values from another system/database.

You can also specify the underlying type:

</> C#

```
enum Status : byte
{
    Pending,
    Approved,
    Rejected
}
```

By default, the underlying type is `int`.

3. What is a Flags enum?

A normal enum generally represents **one choice**:

</> C#

```
Status status = Status.Approved;
```

But sometimes you want to represent **multiple options at the same time**.

For example, a user can have:

- Read permission
- Write permission
- Delete permission

A `[Flags]` enum is designed for this.

</> C#

```
[Flags]
enum Permission
{
    None = 0,
    Read = 1,
    Write = 2,
    Delete = 4
}
```

Notice the values:

</> Plain text

```
Read = 0001
Write = 0010
Delete = 0100
```

Each value uses a **different bit**.

4. Combining flags

You combine them using the **bitwise OR** `|` operator:

</> C#

```
Permission permission = Permission.Read | Permission.Write;
```

Conceptually:

```
</> Plain text

Read   = 0001
Write  = 0010
-----
        0011
```

So permission represents:

```
</> Plain text

Read + Write
```

without Delete.

You can combine all three:

```
</> C#

Permission permission =
    Permission.Read |
    Permission.Write |
    Permission.Delete;
```

Result:

```
</> Plain text

0001
0010
0100
----
0111
```

5. How do I check whether a flag exists?

Use bitwise AND & :

```
</> C#

if ((permission & Permission.Read) != 0)
{
```

```
// Has Read permission
}
```

Why?

Suppose:

```
</> Plain text

permission = Read | Write

permission = 0011
Read       = 0001
```

AND:

```
</> Plain text

0011
0001
----
0001
```

The result is not zero, so Read exists.

6. Modern C# way: HasFlag

You can also write:

```
</> C#

if (permission.HasFlag(Permission.Read))
{
    // Has Read permission
}
```

This is easier to read.

7. Removing a flag

Suppose:

</> C#

```
Permission permission = Permission.Read | Permission.Write;
```

You want to remove `Write` .

Use:

</> C#

```
permission &= ~Permission.Write;
```

Conceptually:

</> Plain text

```
permission = 0011
Write      = 0010

~Write     = 1101
```

Then:

</> Plain text

```
0011
1101
----
0001
```

Now only `Read` remains.

8. Why must Flags values be powers of 2?

Because each flag needs its **own** bit.

Correct:

</> C#

```
[Flags]
enum Permission
{
    None    = 0,
    Read    = 1, // 0001
```

```
Write  = 2, // 0010
Delete = 4, // 0100
Update = 8  // 1000
}
```

These are powers of 2:

</> Plain text

```
1
2
4
8
16
32
...
```

Each has exactly one bit set.

Don't do this:

</> C#

```
Read = 1,
Write = 2,
Delete = 3
```

Because `3` is already:

</> Plain text

```
0011
```

which represents:

</> Plain text

```
Read | Write
```

So `Delete = 3` would not be an independent flag.

9. What does [Flags] actually do?

This is an important interview point.

[Flags] does NOT make the enum capable of combining values.

The bitwise operations work because of the numeric values.

[Flags] mainly tells .NET that the enum is intended to be treated as a set of bit flags and affects things such as its string representation.

For example:

```
</> C#  
  
[Flags]  
enum Permission  
{  
    None = 0,  
    Read = 1,  
    Write = 2,  
    Delete = 4  
}  
  
var p = Permission.Read | Permission.Write;  
  
Console.WriteLine(p);
```

Output:

```
</> Plain text  
  
Read, Write
```

Without [Flags], the enum still has the numeric value 3, but its string representation won't be treated as a combination in the same way.

10. Normal enum vs Flags enum

Normal enum	Flags enum
Represents one state/choice	Represents multiple options
Values can be 0,1,2,3...	Values should normally be powers of 2
Example: Pending, Approved	Example: Read, Write, Delete

Normal enum	Flags enum
Usually assign one value	Combine values with
Check equality commonly	Check with & or HasFlag()

Easy way to remember

Normal enum:

"What is the status?"

```
</> Plain text  
  
Pending  
Approved  
Rejected
```

Flags enum:

"Which capabilities/options are enabled?"

```
</> Plain text  
  
Read + Write + Delete
```

Interview answer

An enum is a value type that defines a set of named constants. A Flags enum is an enum designed to represent a combination of values using individual bits, typically assigning powers of two to each flag and combining them with the bitwise OR operator.

explain IDisposable and using()

Absolutely. IDisposable and using are very important C#/.NET interview topics because they are about resource management.

1. What is IDisposable ?

IDisposable is an interface that tells us:

"This object owns resources that should be explicitly released when you're finished with it."

It contains one method:

```
</> C#

public interface IDisposable
{
    void Dispose();
}
```

So a class can implement it:

```
</> C#

class MyResource : IDisposable
{
    public void Dispose()
    {
        // Release resources
    }
}
```

2. Why do we need Dispose() ?

Some resources are not just normal managed memory.

Examples:

- File handles
- Database connections
- Network connections
- Streams
- Sockets
- Operating-system handles
- Other unmanaged resources

For example:

```
</> C#

FileStream file = new FileStream("test.txt", FileMode.Open);
```

The `FileStream` may hold an OS-level file handle.

When you're finished, you should release it:

```
</> C#

file.Dispose();
```

This tells the object:

"I'm done with you. Release the resources you're holding."

3. Dispose() vs Garbage Collector

This is one of the most important concepts.

The **Garbage Collector (GC)** manages **managed** memory.

`Dispose()` is about **releasing resources deterministically**.

For example:

```
</> Plain text

Managed memory
    ↓
Garbage Collector
    ↓
Eventually cleaned up
```

Whereas:

```
</> Plain text

File / DB connection / socket
    ↓
Dispose()
    ↓
Release now
```

So:

GC determines when an object's memory is reclaimed. `Dispose()` lets you release disposable resources when you are finished with them.

The GC does **not** immediately call `Dispose()` for you.

4. The problem with manually calling `Dispose()`

You could do:

```
</> C#

FileStream file = new FileStream(...);

file.Read(...);

file.Dispose();
```

But what happens if an exception occurs?

```
</> C#

FileStream file = new FileStream(...);

file.Read(...); // exception!

file.Dispose(); // never reached
```

Now you may fail to release the resource.

That's why we have `using`.

5. What is `using`?

`using` automatically ensures that `Dispose()` is called when execution leaves the block.

```
</> C#

using (FileStream file = new FileStream(...))
{
    file.Read(...);
}
```

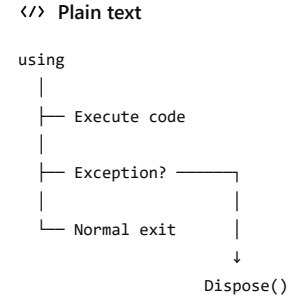
When the block finishes:

```
</> Plain text

using block
↓
Dispose()
```

And importantly, this happens even if an exception occurs.

Conceptually:



6. What does `using` actually translate to?

This:

```
</> C#

using (var resource = GetResource())
{
    DoSomething(resource);
}
```

is conceptually equivalent to:

```
</> C#

var resource = GetResource();

try
{
    DoSomething(resource);
}
finally
{
    resource?.Dispose();
}
```

That's the key thing to understand.

`using` = convenient syntax for deterministic cleanup using `try/finally`.

7. using declaration

Modern C# also allows:

```
</> C#

using var file = new FileStream(...);

file.Read(...);

// Dispose happens when the current scope ends
```

You don't need braces.

Compare:

Using block

```
</> C#

using (var file = ...)
{
    // scope
}
```

Using declaration

```
</> C#

using var file = ...;

// scope continues

// Dispose when scope ends
```

8. Does every class need `IDisposable` ?

No.

You should implement `IDisposable` when your class owns something that needs explicit cleanup.

For example:

```
</> C#

class MyService : IDisposable
{
    private FileStream _file;

    public void Dispose()
    {
        _file.Dispose();
    }
}
```

Your class is now responsible for disposing the `FileStream` it owns.

This leads to an important principle:

If your class owns a disposable resource, your class may need to implement `IDisposable` and dispose that resource.

9. using doesn't mean "dispose everything"

This is important.

You can only use the normal `using` pattern with an object that implements:

```
</> C#

IDisposable
```

or, in modern C#, the appropriate `Dispose` pattern for `using / await using`.

For example:

```
</> C#

using (var file = new FileStream(...))
{
}
```

works because `FileStream` implements `IDisposable`.

10. using and database connections

You'll commonly see:

```
</> C#

using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();

    // Execute commands
}
```

When the block ends:

```
</> Plain text

connection.Dispose()
```

is called.

In ASP.NET Core / EF Core, however, you usually **don't manually dispose injected services like DbContext** when dependency injection created them. The DI container manages their lifetime.

For example:

```
</> C#

public class MyService
{
    private readonly AppDbContext _context;

    public MyService(AppDbContext context)
    {
        _context = context;
    }
}
```

You normally don't do:

```
</> C#

_context.Dispose();
```

because the DI container manages the scoped DbContext .

11. IDisposable

There is also:

```
</> C#

IDisposable
```

It is used when cleanup itself may need to happen asynchronously.

It provides:

```
</> C#

ValueTask DisposeAsync();
```

And you use:

```
</> C#

await using var resource = ...;
```

instead of:

```
</> C#

using var resource = ...;
```

So:

```
</> Plain text

IDisposable
↓
Dispose()

IDisposable
↓
DisposeAsync()
```

12. using directive vs using statement

Don't confuse these two.

using directive

```
</> C#  
  
using System;  
using Microsoft.EntityFrameworkCore;
```

This is about namespaces.

It allows you to write:

```
</> C#  
  
List<int>
```

instead of:

```
</> C#  
  
System.Collections.Generic.List<int>
```

using statement

```
</> C#  
  
using (var file = ...)  
{  
}  
}
```

This is about resource disposal.

Same keyword, completely different purpose.

Interview summary

If they ask "What is IDisposable?":

IDisposable is an interface that defines the Dispose() method, allowing an object to explicitly release resources it owns, such as file handles, database connections, streams, or other unmanaged/native resources.

If they ask "What is using?":

using provides deterministic cleanup of disposable resources. It ensures Dispose() is called when the scope ends, even if an exception occurs. Conceptually, a using block is implemented using try/finally .

And the most important distinction:

```
</> Plain text  
  
GC      → manages managed memory  
Dispose → explicitly releases disposable resources  
using   → ensures Dispose() is called
```

i don;t understabd what dispose a resource mean y mean to remove it from memory or what?

Exactly — this is the important distinction:

Dispose() does NOT primarily mean "remove the object from memory."

It means:

"Release the external resources that this object is currently holding."

Think about a file

Suppose you do:

```
</> C#  
  
var file = new FileStream("test.txt", FileMode.Open);
```

There are actually two different things involved:



The `FileStream` object is a normal .NET object, so its **memory is managed by the Garbage Collector**.

But it also holds an **OS resource**: a handle to the opened file.

When you call:

```
</> C#  
  
file.Dispose();
```

you're essentially saying:

"I'm finished with this file stream. Release the file handle."

So:

```
</> Plain text  
  
Dispose()  
↓  
Release external resource  
↓  
File handle is closed
```

It does **NOT** necessarily mean:

```
</> Plain text  
  
Dispose()  
↓  
Delete FileStream object from memory ❌
```

The GC handles the object's memory later.

A very simple analogy

Imagine you rent a hotel room.

You have:

```
</> Plain text  
  
Hotel room → external resource
```

Your reservation record → object in memory

When you check out:

```
</> Plain text  
  
Dispose()  
↓  
Release the hotel room
```

You're **not necessarily deleting your reservation record from the hotel's computer immediately**.

Similarly:

```
</> Plain text  
  
Dispose() → release resource  
GC → eventually reclaim object's memory
```

Database example

Suppose:

```
</> C#  
  
using (var connection = new SqlConnection(connectionString))  
{  
    connection.Open();  
  
    // use database  
}
```

When the `using` block ends:

```
</> Plain text  
  
connection.Dispose()  
↓  
database connection/resource released
```

The `SqlConnection` object itself can later become garbage and be cleaned from memory by the GC.

The key distinction

Action	Who handles it?
Release file handle	Dispose()
Close/release database connection	Dispose()
Release socket/resource	Dispose()
Reclaim object's managed memory	Garbage Collector

So when you hear:

"Dispose an object"

don't mentally translate that to:

✗ "Delete the object from memory."

Instead think:

✓ "Tell the object to release the resources it owns."

And that's why `IDisposable` is called **disposable**: the object provides a standard way to clean up what it is holding when you're done with it.

what is finaliser

A **finalizer** is a special method in C# that allows an object to perform cleanup **before the Garbage Collector reclaims its memory**.

The important thing is to distinguish it from `Dispose()` .

1. `Dispose()` vs finalizer

Think of a class that owns an unmanaged resource:

</> Plain text

Your object

↳ unmanaged resource
(file handle, OS handle, native resource...)

You ideally release that resource explicitly:

</> C#

obj.Dispose();

But what if the programmer **forgets** to call `Dispose()` ?

A **finalizer** provides a last-chance cleanup mechanism.

</> Plain text

Program

└ Dispose() → cleanup explicitly ✓

└ forgot Dispose()

↓

GC eventually detects object

↓

Finalizer runs

↓

unmanaged resource cleanup

2. What does a finalizer look like?

In C#:

</> C#

class MyResource
{
 ~MyResource()
 {
 // Cleanup unmanaged resources
 }
}

The `~MyResource()` syntax is a **finalizer**.

It looks similar to a destructor in C++, but in C# it is managed by the **GC**, so calling it a "destructor" can be misleading.

3. Is the finalizer called immediately?

No.

This is very important.

You don't control exactly when the finalizer runs.

For example:

</> C#

```
var resource = new MyResource();
```

Later, the object becomes unreachable:

</> Plain text

```
resource
  ↓
null / out of scope
```

```
Heap object
  ↓
No references
```

The GC eventually identifies it as eligible for collection.

If it has a finalizer, the finalizer is scheduled to run, and the object generally requires additional GC processing before its memory is reclaimed.

So:

Finalization is nondeterministic.

That's why you shouldn't rely on finalizers for timely resource cleanup.

4. Then why do we need `Dispose()` ?

Because `Dispose()` is deterministic.

</> C#

```
using (var resource = new MyResource())
{
    // use resource
}
```

When the scope ends:

</> Plain text

```
using
  ↓
Dispose()
  ↓
resource released immediately
```

Whereas:

</> Plain text

```
Finalizer
  ↓
Wait for GC
  ↓
Finalizer eventually runs
```

5. Finalizer is mainly for unmanaged resources

For example, a class directly holding a native/OS handle might have a finalizer as a safety net.

Modern .NET code often uses **SafeHandle** rather than writing finalizers directly, because it makes unmanaged-resource cleanup safer.

So you generally shouldn't add a finalizer just because your class implements **IDisposable** .

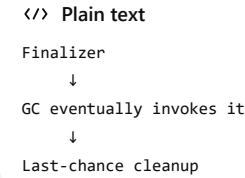
6. The interview distinction

Remember this:

</> Plain text

```
IDisposable
  ↓
Dispose()
  ↓
Explicit / deterministic cleanup
```

versus:



One-sentence interview answer

A finalizer is a GC-managed cleanup mechanism that gives an object a last chance to release unmanaged resources when `Dispose()` wasn't called, but its execution timing is nondeterministic.

And one more important point:

Finalizer ≠ Garbage Collector.

The GC detects and manages unreachable objects; the finalizer is a special cleanup mechanism that the GC infrastructure invokes for finalizable objects.

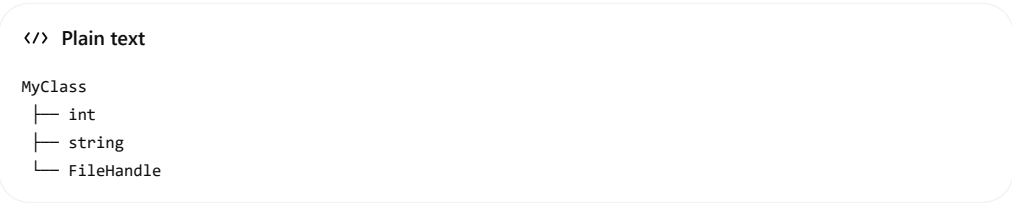
i thought finaliser is for managed resources

You're very close, but the key distinction is:

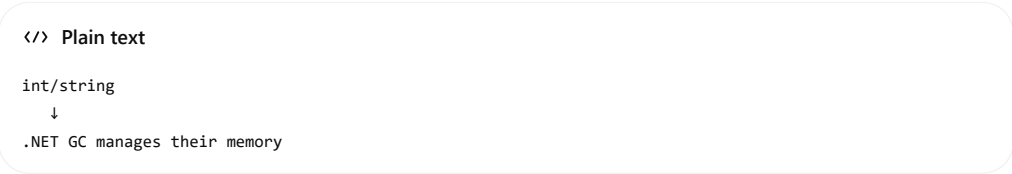
Finalizers are primarily for unmanaged resources, not managed resources.

Why?

Suppose your class has:

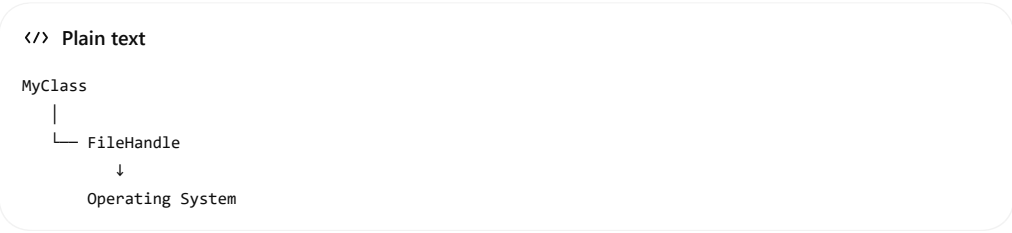


The first two are managed resources:



You normally do not need a finalizer for them.

But `FileHandle` represents an external/unmanaged resource:



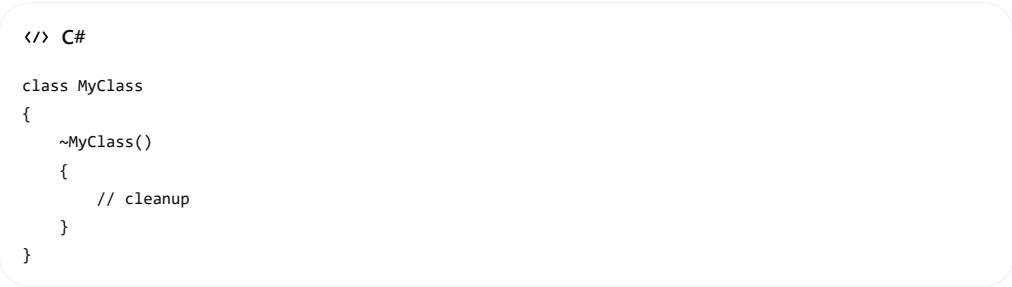
The GC doesn't know how to properly release that OS resource just by reclaiming your object's memory.

That's where cleanup mechanisms such as `Dispose()` / `SafeHandle` and, historically, finalizers come in.

Why might you have thought finalizers are for managed resources?

Because a finalizer is itself associated with a managed .NET object.

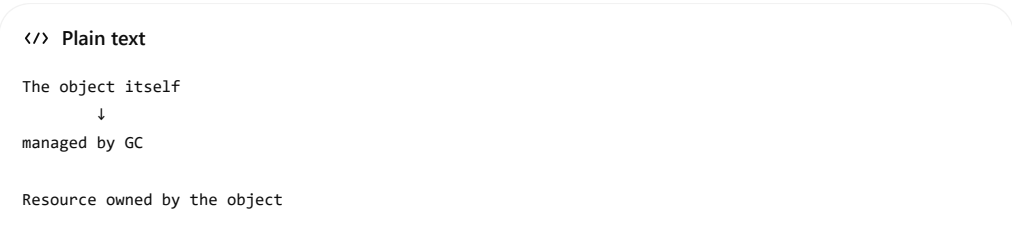
For example:



The `MyClass` object is managed by the GC.

But the resource the finalizer is intended to clean up is typically unmanaged.

So distinguish:



10/3/26, 7:48 PM

C# Recap

↓

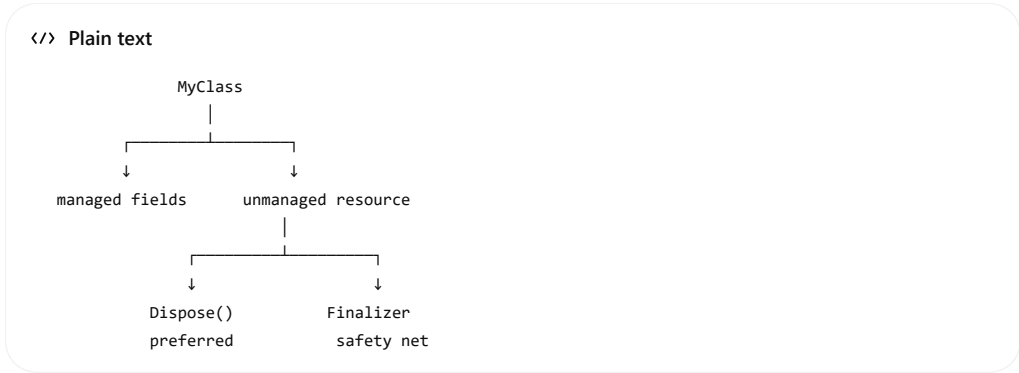
may be unmanaged

↓

needs explicit cleanup / finalization

The common pattern

For a class owning unmanaged resources:



Dispose() → preferred, deterministic cleanup.

Finalizer → backup/last chance if explicit cleanup didn't happen.

And in modern .NET, `SafeHandle` is generally preferred over writing your own finalizer directly for OS handles.

ctor is for creating instance of class and finaliser is to dispose the instance

Almost — but one important correction.

Constructor vs Finalizer

	Constructor	Finalizer
Purpose	Initialize a new object	Last-chance cleanup
When	When you create an object	Eventually, during GC processing
Controlled by	Your code	GC
Timing	Immediate/deterministic	Nondeterministic

10/3/26, 7:48 PM

C# Recap

	Constructor	Finalizer
Main use	Initialize state/resources	Clean up unmanaged resources

Constructor

When you write:

</> C#

```
var person = new Person();
```

the constructor runs:

</> C#

```
class Person
{
    public Person()
    {
        // Initialize object
    }
}
```

Think:

Constructor = prepare/initialize the object.

Finalizer

A finalizer:

</> C#

```
class MyResource
{
    ~MyResource()
    {
        // Last-chance cleanup
    }
}
```

does not dispose the instance itself.

Instead, it gives the object a chance to **release unmanaged resources** it owns before the object's memory is eventually reclaimed.

Think:

Finalizer = last-chance cleanup before the object's memory is reclaimed.

The full lifecycle

Conceptually:

</> Plain text

```
new Person()
↓
Constructor
↓
Object is usable
↓
Object becomes unreachable
↓
GC eventually processes it
↓
Finalizer (if it has one)
↓
Object's memory eventually reclaimed
```

But if you have `IDisposable`, you normally want:

</> Plain text

```
new Object()
↓
Use Object
↓
Dispose()
↓
Resources released
↓
Object eventually reclaimed by GC
```

So don't think:

✗ Constructor creates → Finalizer destroys

Think:

✓ **Constructor initializes → Dispose explicitly releases resources → Finalizer is a GC-managed safety net for unmanaged resources.**

so how is responsible for creating and disposing the class instance

Yes — this is the key distinction:

Who creates the instance?

Your code creates the instance using `new` :

</> C#

```
var person = new Person();
```

The process is roughly:

</> Plain text

```
new Person()
↓
Memory is allocated for the object
↓
Constructor runs
↓
You get a reference to the object
```

So:

new + constructor → create and initialize the object

Who destroys the instance?

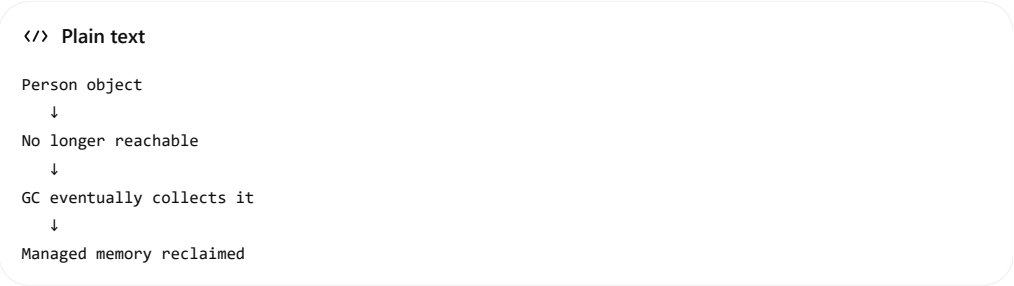
You normally **don't** explicitly destroy a class instance in C#.

The **Garbage Collector (GC)** is responsible for eventually reclaiming the object's managed memory when the object is no longer reachable.

</> Plain text

```
person
↓
Person object
```

When there are no reachable references:



Then what does `Dispose()` do?

This is where the confusion usually happens.

`Dispose()` **doesn't destroy the object.**

It releases resources that the object owns.

For example:



The object can technically still exist after `Dispose()` :

</> C#

```
resource.Dispose();

// resource object may still exist here
```

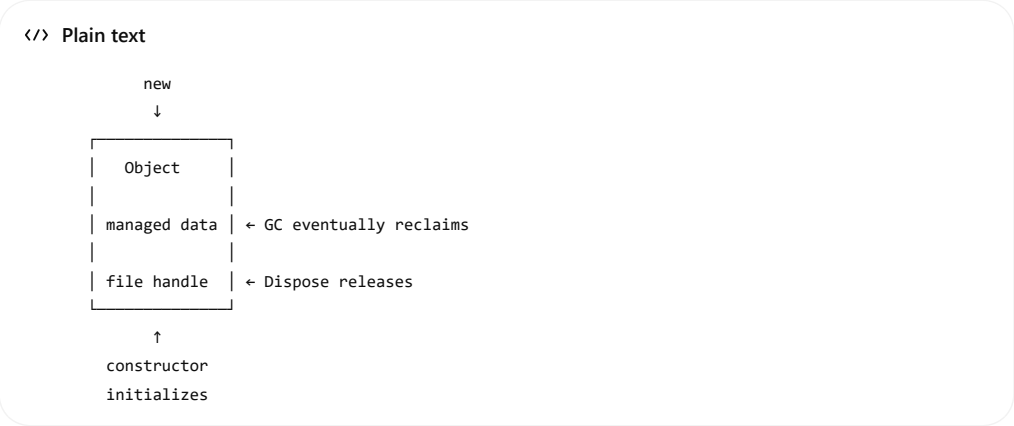
But you generally shouldn't continue using it if its contract says it's disposed.

And the finalizer?

A finalizer is a **GC-related last-chance cleanup mechanism**, mainly for unmanaged resources.

It doesn't "destroy the instance" either.

The easiest mental model



So remember these **three separate responsibilities**:

Responsibility	Mechanism
Create object	<code>new</code>
Initialize object	Constructor
Release external resources	<code>Dispose()</code>
Last-chance unmanaged cleanup	Finalizer
Reclaim object's managed memory	Garbage Collector

One important sentence

C# doesn't normally have a `delete` operation for class instances. You create them with `new` , release disposable resources with `Dispose()` , and the GC eventually reclaims the object's managed memory.

but struct comes with `new()`

Yes, but there is an important distinction: a **struct can be created with `new`**, but it doesn't have to be.

Class

For a class:

```
</> C#  
  
Person p = new Person();
```

`new` creates the object and the constructor initializes it.

Struct

For a struct:

```
</> C#  
  
Point p = new Point();
```

`new` initializes the struct to its default state (or invokes an applicable constructor).

But you can also do:

```
</> C#  
  
Point p;  
p.X = 10;  
p.Y = 20;
```

Here, you didn't use `new`.

So:

`new` is not exclusive to classes.

What about `new()` in a generic constraint?

This is different:

```
</> C#  
  
where T : new()
```

It means:

T must have an accessible parameterless constructor.

For example:

```
</> C#  
  
class Person  
{  
    public Person() { }  
}
```

Then:

```
</> C#  
  
class Factory<T> where T : new()  
{  
    // Can create T with new T()  
}
```

A struct automatically satisfies the `new()` constraint because a non-nullable struct has a public parameterless construction mechanism.

So:

```
</> C#  
  
where T : new()
```

does **not** mean "T must be a struct."

Both of these can satisfy it:

```
</> Plain text  
  
class Person  
{  
    public Person() { }  
} ✓
```

The key distinction

</> Plain text

```
new Person()
  ↓
create/initialize class instance

new Point()
  ↓
initialize struct value

where T : new()
  ↓
constraint saying T can be constructed
```

And remember from earlier: `new` does not automatically mean "allocate on the heap." A struct is a value type even when you write `new Point()` .

explain delegates

Sure. Delegates are a very important C# interview topic, especially because they are the foundation of events, Action , Func , and callbacks.

1. What is a delegate?

A delegate is a type-safe reference to a method.

In simple words:

A delegate allows you to store a method in a variable and then call that method through the variable.

For example, normally:

</> C#

```
void SayHello()
{
    Console.WriteLine("Hello");
}
```

With a delegate:

</> C#

```
delegate void MyDelegate();

MyDelegate d = SayHello;

d();
```

Think of it as:

</> Plain text

```
d
|
└──> SayHello()
```

d is now referring to the SayHello method.

2. Why do we need delegates?

The main reason is that you can pass behavior as a parameter.

Imagine:

</> C#

```
void Calculate(int a, int b, ??? operation)
```

You might want to tell Calculate :

"Add these numbers."

or:

"Multiply these numbers."

A delegate allows you to pass the operation itself.

</> Plain text

https://chatgpt.com/c/6abe704d-d204-83e9-b847-5120331f8f5d

194/448

```

Calculate
|
├─ operation → Add()
|
OR
|
└─ operation → Multiply()

```

This is called a **callback**.

3. Creating a delegate

The syntax is:

```

</> C#

delegate returnType DelegateName(parameters);

```

Example:

```

</> C#

delegate int Operation(int x, int y);

```

This means:

`Operation` can reference any method that takes two `int` s and returns an `int` .

For example:

```

</> C#

int Add(int x, int y)
{
    return x + y;
}

int Multiply(int x, int y)
{
    return x * y;
}

```

Both methods match the delegate's signature.

So:

```

</> C#

Operation op = Add;

```

or:

```

</> C#

Operation op = Multiply;

```

Then:

```

</> C#

int result = op(5, 3);

```

If `op` refers to `Add` :

```

</> Plain text

5 + 3 = 8

```

If it refers to `Multiply` :

```

</> Plain text

5 × 3 = 15

```

4. Delegate is a type

This is an important point.

When you write:

```

</> C#

delegate int Operation(int x, int y);

```

you're defining a **new type** called `Operation` .

Similar to:

```
</> C#

class Person
{
}
```

You're defining a type.

But a delegate type describes:

What kind of method can be referenced.

So:

```
</> Plain text

class
↓
describes objects

delegate
↓
describes methods that can be referenced
```

5. Method signature must match

Suppose:

```
</> C#

delegate int Operation(int x, int y);
```

This works:

```
</> C#

int Add(int x, int y)
{
    return x + y;
}
```

because:

```
</> Plain text

Parameters: int, int
Return:      int
```

But this doesn't:

```
</> C#

string Add(int x, int y)
```

because the return type doesn't match.

And this doesn't:

```
</> C#

int Add(int x)
```

because the parameters don't match.

So:

A delegate defines the signature that the referenced method must be compatible with.

6. Delegates with lambda expressions

This is where you'll see delegates constantly in modern C#.

Instead of:

```
</> C#

int Add(int x, int y)
{
    return x + y;
}

Operation op = Add;
```

you can write:

</> C#

```
Operation op = (x, y) => x + y;
```

Now the delegate directly references the lambda.

Then:

</> C#

```
int result = op(5, 3);
```

7. Action

C# provides built-in delegate types so you usually don't need to define your own.

The first important one is:

Action

`Action` represents a method that:

returns `void`.

For example:

</> C#

```
Action sayHello = () => Console.WriteLine("Hello");
```

Then:

</> C#

```
sayHello();
```

You can have parameters:

</> C#

```
Action<string> print = message =>
{
```

```
Console.WriteLine(message);
```

```
};
```

Or multiple parameters:

</> C#

```
Action<int, int> printSum = (a, b) =>
{
    Console.WriteLine(a + b);
};
```

`Action` always returns:

</> Plain text

```
void
```

8. Func

`Func` represents a delegate that **returns a value**.

For example:

</> C#

```
Func<int, int, int> add = (x, y) => x + y;
```

This means:

</> Plain text

```
Func<int, int, int>
|      |      |
|      |      | return type
|      |      |
|      |      | parameter
|      |      |
|      |      | parameter
```

So:

</> C#

gives:

```
</> Plain text

8
```

Important interview rule

For `Func<T1, T2, TResult>` :

The last generic type is the return type.

Example:

```
</> C#

Func<int, string>
```

means:

```
</> Plain text

input: int
output: string
```

9. Action vs Func

Very common interview question:

	Action	Func
Return value	void	Has a return value
Parameters	0–16	0–16 inputs + return type
Example	Action<int>	Func<int, string>

Example:

```
</> C#
```

No return.

```
</> C#

Func<int, int> square = x => x * x;
```

Returns a value.

10. Predicate<T>

There's another built-in delegate:

```
</> C#

Predicate<T>
```

It represents a method that:

Takes one `T` and returns `bool` .

For example:

```
</> C#

Predicate<int> isEven = x => x % 2 == 0;
```

Then:

```
</> C#

bool result = isEven(10);
```

Result:

```
</> Plain text

true
```

Conceptually:

</> Plain text

```
Predicate<T>
  ↓
T → bool
```

11. Delegates can reference multiple methods

A delegate can have an **invocation list**.

For example:

</> C#

```
Action action = Method1;
action += Method2;
action += Method3;
```

Now:

</> C#

```
action();
```

calls:

</> Plain text

```
Method1()
Method2()
Method3()
```

in invocation order.

This is called a **multicast delegate**.

You can remove one:

</> C#

```
action -= Method2;
```

12. Delegates and events

This is extremely important.

Events are built on delegates.

For example:

</> C#

```
public event Action? SomethingHappened;
```

The delegate describes:

What methods can respond to the event.

The event provides controlled notification.

For example:

</> Plain text

```
Button clicked
  ↓
Event raised
  ↓
Delegate invocation list
  ↓
Method1()
Method2()
Method3()
```

This is why you'll often hear:

Events use delegates as their underlying mechanism.

13. Delegates and LINQ

You've probably already used delegates without realizing it.

For example:

</> C#

```
var result = numbers.Where(x => x > 10);
```

That:

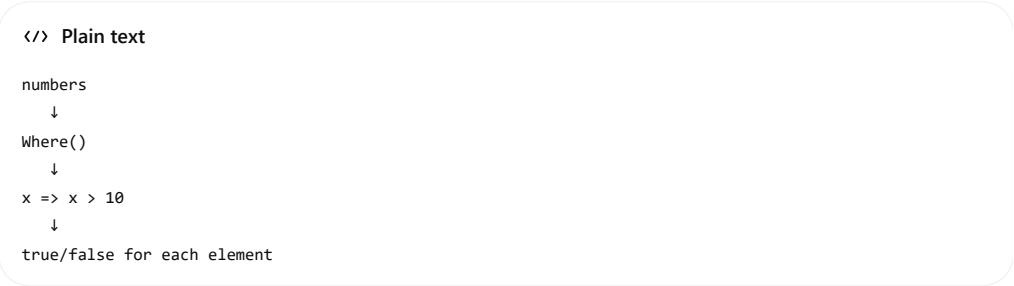
```
</> C#  
  
x => x > 10
```

is a lambda expression.

Where accepts a delegate-like parameter:

```
</> C#  
  
Func<int, bool>
```

Conceptually:



Similarly:

```
</> C#  
  
numbers.Select(x => x * 2);
```

uses a function describing how to transform each element.

14. Delegate vs interface

Another useful interview comparison.

Delegate

Best when you want to pass a **method/behavior**.

```
</> Plain text
```

```
"Do this operation."
```

Interface

Best when you want to define a **contract for an object/type**.

```
</> Plain text  
  
"This object must support these capabilities."
```

For example:

```
</> Plain text  
  
Delegate:  
Func<int, int>  
  
"I give you an int, you give me an int."
```

Interface:

```
</> Plain text  
  
IRepository  
  
"This object provides repository operations."
```

15. The mental model

The easiest way to understand delegates is:

Normally:

```
</> Plain text  
  
variable → data
```

For example:

```
</> C#  
  
int x = 10;
```

But a delegate is:

</> Plain text

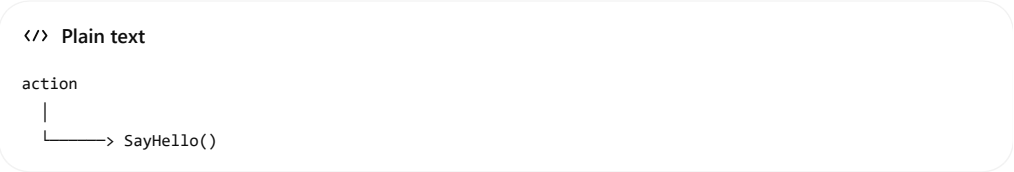
delegate variable → method

Example:

</> C#

Action action = SayHello;

Conceptually:



Then:

</> C#

action();

means:

"Call the method that this delegate currently refers to."

Interview answer

If the interviewer asks:

"What is a delegate?"

A strong answer is:

A delegate is a type-safe reference to a method. It allows methods to be passed as parameters, stored in variables, and invoked later. Delegates are commonly used for callbacks, lambdas, events, and LINQ.

And remember this hierarchy:



The next concept you should connect to delegates is `events` , because understanding why C# has `event` on top of delegates makes the whole topic much easier.

explain event

Absolutely. Since you now understand **delegates**, `event` becomes much easier.

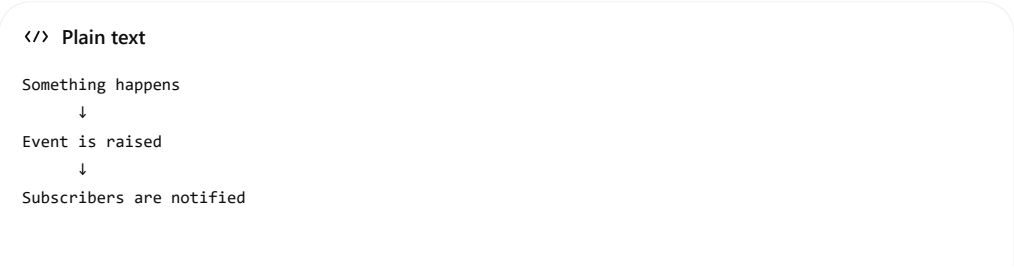
1. What is an event?

An **event** is a mechanism that allows an object to **notify other objects that something has happened**.

Examples:

- Button was clicked
- Order was created
- Payment completed
- File downloaded
- User logged in
- Temperature changed

The basic idea is:



2. Events are built on delegates

This is the most important connection:

An event uses a delegate to define what kind of methods can subscribe to it.

For example:

</> C#

```
public delegate void NotifyHandler();

public event NotifyHandler SomethingHappened;
```

Here:



3. Subscriber and publisher

Events normally involve two sides.

Publisher

The object that **raises the event**.

Subscriber

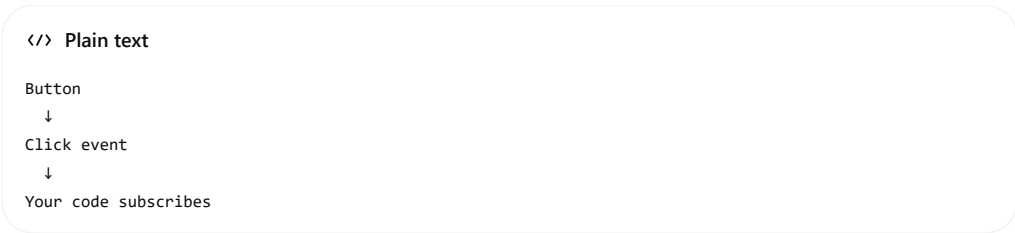
The object that **listens for the event**.

Example:

</> Plain text



For example, a `Button` is a publisher:



4. Simple example

Suppose we have:

</> C#

```
class Button
{
    public event Action? Click;

    public void Press()
    {
        Click?.Invoke();
    }
}
```

Now another part of the program subscribes:

</> C#

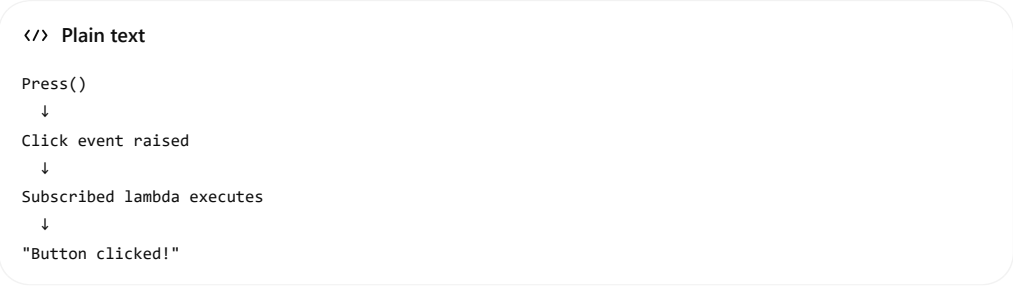
```
Button button = new Button();

button.Click += () =>
{
    Console.WriteLine("Button clicked!");
};
```

When:

```
</> C#  
  
button.Press();
```

happens:



5. What does += mean?

This:

```
</> C#  
  
button.Click += Handler;
```

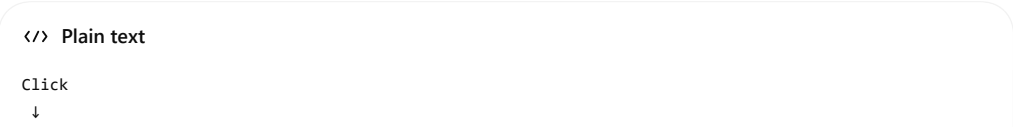
means:

Subscribe this method to the event.

You can have multiple subscribers:

```
</> C#  
  
button.Click += Handler1;  
button.Click += Handler2;  
button.Click += Handler3;
```

Then when the event is raised:



```
Handler1()  
Handler2()  
Handler3()
```

This is related to the **multicast nature of delegates** we discussed earlier.

6. What does -= mean?

To unsubscribe:

```
</> C#  
  
button.Click -= Handler1;
```

Now `Handler1` won't be called when the event is raised.

So:

```
</> Plain text  
  
+= → subscribe  
-= → unsubscribe
```

7. Who is allowed to raise the event?

This is where `event` is different from just using a delegate.

Suppose you have:

```
</> C#  
  
public Action? SomethingHappened;
```

Anyone who has access to this field could potentially do:

```
</> C#  
  
obj.SomethingHappened?.Invoke();
```

That's often undesirable.

With:

```
</> C#

public event Action? SomethingHappened;
```

outside code can **subscribe/unsubscribe**:

```
</> C#

obj.SomethingHappened += Handler;
obj.SomethingHappened -= Handler;
```

but it **cannot raise the event**.

Only the class that declares the event can normally invoke it.

This gives the publisher control over when the notification happens.

8. Why do we need event if we already have delegates?

Excellent interview question.

A delegate by itself can be:

```
</> C#

public Action SomethingHappened;
```

External code could replace the entire delegate:

```
</> C#

obj.SomethingHappened = SomeMethod;
```

or invoke it.

An event restricts outside access to essentially:

```
</> Plain text

Outside:
    += subscribe
```

```
 -= unsubscribe
```

```
Inside declaring class:
 +=
 -=
 Invoke/raise
```

So:

An event provides controlled access to a delegate, mainly for notification/subscription.

9. The standard .NET event pattern

You'll often see:

```
</> C#

public event EventHandler? SomethingHappened;
```

EventHandler is a built-in delegate.

It represents a method roughly like:

```
</> C#

void Handler(object? sender, EventArgs e)
```

So:

```
</> C#

public event EventHandler? OrderCreated;
```

A subscriber can do:

```
</> C#

OrderCreated += OnOrderCreated;
```

And the publisher raises it:

```
</> C#
```

10. Events with custom data

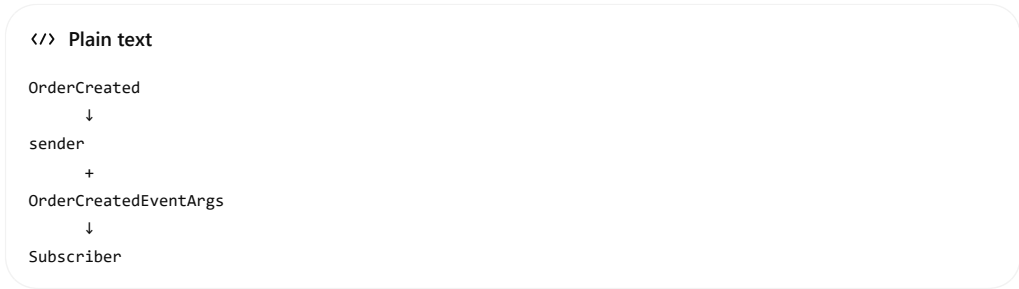
You can send information to subscribers.

For example:

</> C#

public event EventHandler<OrderCreatedEventArgs> OrderCreated;

Then the event can provide information about the order:



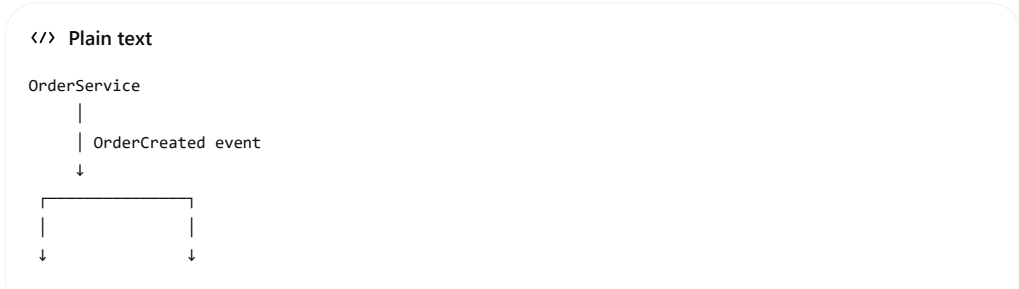
The subscriber can inspect the event arguments.

This is extremely common in real applications.

11. Real-world example

Imagine an e-commerce system.

An order is created:



The `OrderService` doesn't necessarily need to know exactly what every subscriber does.

It simply says:

"An order was created."

Subscribers decide what to do.

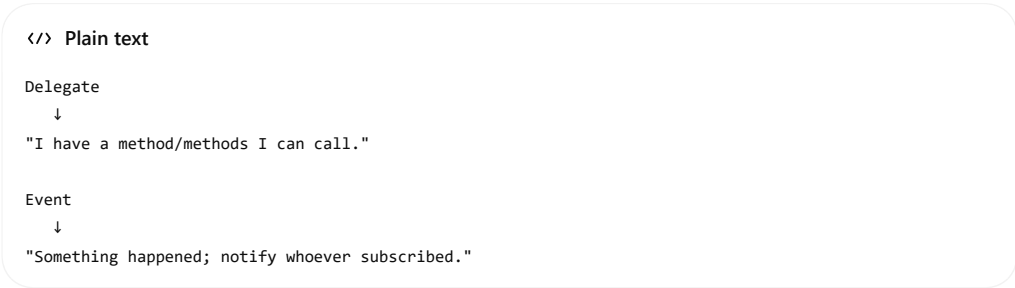
This creates a **looser coupling** between components.

12. Event vs delegate

Very common interview question:

Delegate	Event
Can reference one or multiple methods	Used for notification/subscription
Can be invoked by whoever has access	Normally only declaring class raises it
Can be assigned/replaced externally	External code can only += / -=
General-purpose method reference	Specialized notification mechanism
Foundation of events	Built on delegates

Think:



13. Event vs method call

Without events:

</> Plain text

```
OrderService
  ↓
EmailService.SendEmail()
  ↓
NotificationService.Notify()
```

OrderService needs to know about those services.

With an event:

</> Plain text

```
OrderService
  ↓
OrderCreated event
  ↓
Subscribers
├── EmailService
└── NotificationService
```

The publisher doesn't need to directly call each subscriber.

14. One subtle point

An event is **not** a separate object that magically detects something.

You still need code to **raise/trigger** the event.

For example:

</> C#

```
public void CreateOrder()
{
    // Create order...

    OrderCreated?.Invoke(this, EventArgs.Empty);
}
```

The event doesn't automatically know that an order was created.

Your code tells it:

"Now raise the event."

Interview answer

If they ask:

"What is an event in C#?"

Say:

An event is a notification mechanism built on delegates. It allows a publisher to notify subscribed objects when something happens, while restricting external code to subscribing and unsubscribing rather than raising the event.

And remember:

</> Plain text

```
Delegate
  ↓
Type-safe method reference

Event
  ↓
Controlled delegate-based notification

+=
  ↓
Subscribe

-=
  ↓
Unsubscribe

Invoke
  ↓
Raise/trigger the event
```

The key relationship is:

Delegate = "which method can be called"

Event = "notify these subscribed methods that something happened."

1 selection

i don;t understand what is the sender obj and eventargs

Think of this method:

```
</> C#

void Handler(object? sender, EventArgs e)
```

as receiving **two pieces of information** when the event happens:

```
</> Plain text

Handler
|
├─ sender → WHO raised the event?
|
└─ e      → WHAT information comes with the event?
```

1. sender

sender is the **object that raised the event**.

For example:

```
</> C#

class Button
{
    public event EventHandler? Click;

    public void Press()
    {
        Click?.Invoke(this, EventArgs.Empty);
    }
}
```

Notice:

```
</> C#

Click?.Invoke(this, EventArgs.Empty);
```

The this is passed as sender .

So if you have:

```
</> C#

Button button = new Button();

button.Click += Handler;
```

When the button raises the event:

```
</> Plain text

button
|
└─ raises Click
    ↓
  Handler(sender, e)
    ↑
    |
  button
```

Therefore:

```
</> C#

void Handler(object? sender, EventArgs e)
{
    // sender is the Button that raised the event
}
```

You can potentially get the actual type:

```
</> C#

Button button = (Button)sender!;
```

Although pattern matching is generally safer:

```
</> C#

if (sender is Button button)
{
```

```
// use button
}
```

Why is `sender` typed as `object` ?

Because any `object` can raise an event.

It could be:

</> Plain text

```
Button
OrderService
Timer
FileDownloader
...
```

So .NET uses the general `object` type.

2. What is `EventArgs` ?

`e` contains **additional information about the event**.

The basic:

</> C#

```
EventArgs
```

means:

"This event has no additional information to give you."

That's why you often see:

</> C#

```
EventArgs.Empty
```

For example:

</> C#

```
Click?.Invoke(this, EventArgs.Empty);
```

The event is basically saying:

"I am the Button, and I was clicked. I don't have any extra information."

3. But events can have extra information

Suppose an order was created.

You might want to tell subscribers:

- Order ID
- Customer ID
- Total price

You could create custom event arguments:

</> C#

```
class OrderCreatedEventArgs : EventArgs
{
    public int OrderId { get; set; }
    public decimal Total { get; set; }
}
```

Then:

</> C#

```
public event EventHandler<OrderCreatedEventArgs>? OrderCreated;
```

When raising the event:

</> C#

```
OrderCreated?.Invoke(
    this,
    new OrderCreatedEventArgs
    {
        OrderId = 10,
        Total = 500
    });
```

The handler receives:

```
</> C#

void Handler(object? sender, OrderCreatedEventArgs e)
{
    Console.WriteLine(e.OrderId);
    Console.WriteLine(e.Total);
}
```

Now:

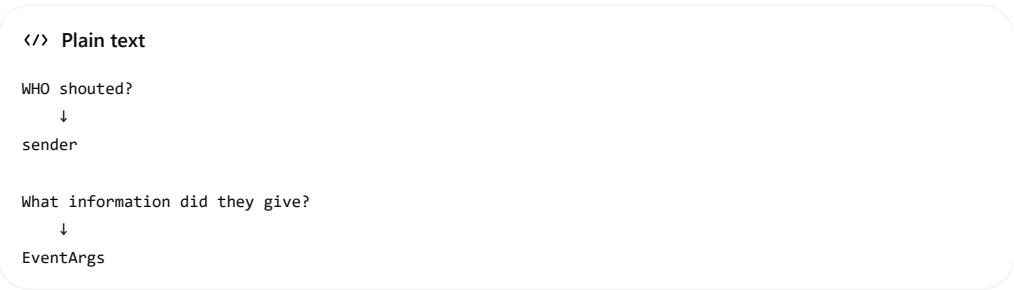


4. Very simple analogy

Imagine someone shouts:

"Fire!"

You receive two pieces of information:



For a simple event:

```
</> Plain text

sender = FireAlarm
e      = EventArgs.Empty
```

For a richer event:

```
</> Plain text

sender = FireAlarm

e = FireEventArgs
  ├── Location = "Kitchen"
  └── Severity  = "High"
```

The interview rule to remember

```
</> C#

void Handler(object? sender, EventArgs e)
```

means:

`sender` tells you which object raised the event, while `e` contains information associated with the event.

And importantly, `EventArgs` itself doesn't have to contain useful data. `EventArgs.Empty` is used when there is no extra event-specific information.

checked ,unchecked

In C#, `checked` and `unchecked` control what happens when an arithmetic operation or numeric conversion overflows.

1. checked

`checked` tells C# to **detect overflow** and throw an `OverflowException` .

```
</> C#

int x = int.MaxValue;
```

```
int y = checked(x + 1);
```

Result:

```
</> Plain text
```

```
OverflowException
```

Because `int.MaxValue + 1` cannot fit inside an `int`.

You can also use a block:

```
</> C#
```

```
checked
{
    int x = int.MaxValue;
    int y = x + 1;
}
```

2. unchecked

`unchecked` tells C# **not to throw an exception for integer overflow**.

```
</> C#
```

```
int x = int.MaxValue;

int y = unchecked(x + 1);
```

The result wraps around:

```
</> Plain text
```

```
y = -2147483648
```

That's `int.MinValue`.

Conceptually:

```
</> Plain text
```

```
int.MaxValue
  ↓ +1
int.MinValue
```

3. Very important: conversion

This also applies to explicit numeric conversions.

```
</> C#
```

```
long x = 5_000_000_000;

int y = unchecked((int)x);
```

The value doesn't fit in `int`, so the bits are converted according to unchecked integer conversion rules, producing:

```
</> Plain text
```

```
705032704
```

But:

```
</> C#
```

```
int y = checked((int)x);
```

throws:

```
</> Plain text
```

```
OverflowException
```

4. Default behavior

For normal **integral arithmetic and conversions**, C# is generally **unchecked by default** unless checked arithmetic is enabled/project-configured.

So:

```
</> C#
```

```
int x = int.MaxValue;
int y = x + 1;
```

normally wraps around instead of throwing.

You can explicitly choose the behavior:

</> C#

```
checked // detect overflow
unchecked // allow overflow
```

Interview answer

`checked` causes an `OverflowException` when an integral arithmetic operation or conversion overflows, while `unchecked` allows the overflow and produces the normal wrapped/truncated result.

Key distinction: `checked` doesn't prevent overflow—it detects it and throws.

exceptions and custom exceptions

1. What is an Exception?

An **exception** is an unexpected situation that occurs while a program is running and interrupts the normal flow of execution.

Example:

</> C#

```
int x = 10;
int y = 0;

int result = x / y;
```

This causes:

</> Plain text

```
DivideByZeroException
```

Instead of continuing normally, C# throws an exception.

2. Handling exceptions: try-catch

You can handle an exception using `try-catch` :

</> C#

```
try
{
    int result = 10 / 0;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
```

Flow

</> Plain text

```
try
↓
Exception occurs
↓
catch
↓
Handle the exception
↓
Program continues
```

3. finally

`finally` executes whether an exception occurs or not.

</> C#

```
try
{
    // code
}
catch (Exception ex)
{
    // handle exception
}
finally
{
    // cleanup code
}
```

```
// always executes
}
```

It's commonly used for cleanup.

For example, historically:

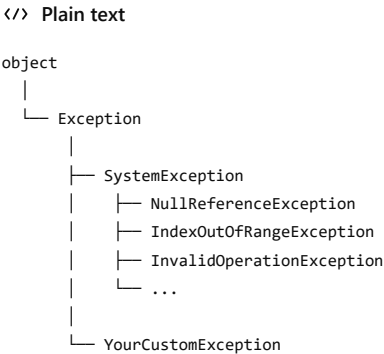
```
</> C#
try
{
    // use resource
}
finally
{
    // release resource
}
```

Although for `IDisposable` resources, `using` is usually preferred.

4. Exception hierarchy

Most exceptions derive from `System.Exception`.

Conceptually:



So you can catch a specific exception:

```
</> C#
catch (NullReferenceException ex)
```

or a general one:

```
</> C#
catch (Exception ex)
```

Important

Prefer catching the **specific exception** when you know what you're handling.

5. Multiple catch blocks

You can handle different exceptions differently:

```
</> C#
try
{
    // code
}
catch (DivideByZeroException ex)
{
    // handle division problem
}
catch (FormatException ex)
{
    // handle invalid format
}
catch (Exception ex)
{
    // fallback
}
```

The more specific catches should come **before** the general `Exception` catch.

6. throw

`throw` is used to throw an exception manually.

```
</> C#
if (age < 18)
{
```

```
throw new Exception("Age must be at least 18.");  
}
```

You're basically saying:

"Something invalid happened, so stop normal execution and report this problem."

7. Custom Exceptions

Sometimes the built-in exceptions don't describe your application's problem clearly.

You can create your own exception.

For example, in a banking application:

```
</> C#  
  
public class InsufficientBalanceException : Exception  
{  
    public InsufficientBalanceException(string message)  
        : base(message)  
    {  
    }  
}
```

Then:

```
</> C#  
  
if (balance < amount)  
{  
    throw new InsufficientBalanceException("Insufficient balance.");  
}
```

And handle it:

```
</> C#  
  
try  
{  
    Withdraw(5000);  
}  
catch (InsufficientBalanceException ex)  
{
```

```
Console.WriteLine(ex.Message);  
}
```

8. Why create custom exceptions?

Because they give your application **meaningful exception types**.

Instead of:

```
</> Plain text  
  
Exception: Something went wrong
```

you can have:

```
</> Plain text  
  
InsufficientBalanceException
```

OR:

```
</> Plain text  
  
ProductNotFoundException
```

OR:

```
</> Plain text  
  
InvalidOrderStateException
```

The caller can then specifically handle that situation.

9. Custom exception best practice

A custom exception normally:

- Inherits from `Exception`
- Has a meaningful name ending in `Exception`
- Provides constructors for a message and optionally an inner exception

Typical structure:

```
</> C#

public class ProductNotFoundException : Exception
{
    public ProductNotFoundException()
    {
    }

    public ProductNotFoundException(string message)
        : base(message)
    {
    }

    public ProductNotFoundException(string message, Exception innerException)
        : base(message, innerException)
    {
    }
}
```

The `innerException` is useful when your exception wraps another exception while preserving the original cause.

10. `throw;` vs `throw ex;`

This is an important interview question.

Inside a `catch` :

```
</> C#

catch (Exception ex)
{
    throw;
}
```

preserves the **original stack trace**.

But:

```
</> C#

catch (Exception ex)
{
```

```
    throw ex;
}
```

resets the stack-trace information from the point of the new throw.

So generally:

```
</> C#

catch (Exception ex)
{
    // log if needed
    throw;
}
```

is preferred when you want to rethrow the same exception.

Interview summary

An exception represents an error or exceptional condition that occurs during program execution. We use `try`, `catch`, and `finally` to handle exceptions. Custom exceptions are application-specific exception classes derived from `Exception`, used when a built-in exception doesn't clearly represent a domain-specific error.

And remember:

```
</> Plain text

try      → code that may fail
catch    → handle the exception
finally  → cleanup / always execute
throw    → raise an exception
Exception → base exception type
Custom Exception → application-specific exception
```

1 selection

explain more

Sure. **Rethrowing** means: you caught an exception, perhaps logged it or did some local handling, but you want the exception to continue traveling upward to another `catch`.

The important difference is between `throw;` and `throw ex;`.

1. throw; — rethrow correctly

```
</> C#

try
{
    MethodA();
}
catch (Exception ex)
{
    Console.WriteLine("Logging error...");
    throw;
}
```

throw; means:

"Take the **same exception** that I just caught and continue throwing it."

It preserves the **original stack trace**.

For example, suppose:

```
</> Plain text

MethodA()
↓
MethodB()
↓
MethodC()
↓
Exception occurs here
```

The stack trace can preserve that original path:

```
</> Plain text

MethodC()
MethodB()
MethodA()
...
```

This is extremely useful for debugging because you can see **where the exception originally happened**.

2. throw ex; — don't use this for rethrowing

```
</> C#

try
{
    MethodA();
}
catch (Exception ex)
{
    Console.WriteLine("Logging error...");
    throw ex;
}
```

Here you're effectively saying:

"Throw `ex` again from **this location**."

The stack-trace information gets reset/alterd so the original throw location can be lost from the useful stack trace.

That's why, when you simply want to rethrow the same exception, use:

```
</> C#

throw;
```

not:

```
</> C#

throw ex;
```

3. Why would we catch and rethrow?

A common reason is **logging**.

```
</> C#

try
{
    SaveOrder();
}
catch (Exception ex)
{
    Console.WriteLine("Logging error...");
    throw ex;
}
```

```
Log(ex);
throw;
}
```

The flow is:

```
</> Plain text

SaveOrder()
↓
Exception occurs
↓
catch
↓
Log the exception
↓
throw;
↓
Exception continues upward
```

Another layer can then handle it:

```
</> C#

try
{
    Service.DoSomething();
}
catch (Exception ex)
{
    // final handling
}
```

So the lower layer can **record information**, while the higher layer can decide how to respond.

4. Very important distinction

Compare:

```
</> C#

catch (Exception ex)
{
    throw;
}
```

with:

```
</> C#

catch (Exception ex)
{
    throw new Exception("Something went wrong");
}
```

The first means:

Rethrow the same exception.

The second means:

Create and throw a new exception.

If you create a new exception but want to preserve the original exception as the cause, use an **inner exception**:

```
</> C#

catch (Exception ex)
{
    throw new Exception("Failed to save order.", ex);
}
```

Now you have:

```
</> Plain text

New Exception
|
└─ InnerException
    |
    └─ Original Exception
```

This is useful when you want to add higher-level context while preserving the original error.

Interview rule to remember

throw; rethrows the current exception while preserving its original stack trace. **throw ex;** throws it from the current location and can lose the original stack-trace information.

explain exception class

Sure. Think of **Exception** as the **base class that describes an error/exception in .NET.**

1. What is **Exception** ?

Exception is a class in the `System` namespace:

```
</> C#  
  
System.Exception
```

Most exceptions you encounter inherit from it:



For example:

```
</> C#  
  
try  
{  
    int x = 10 / 0;  
}  
catch (Exception ex)  
{  
    // ex is an Exception object  
}
```

The `ex` variable refers to an **exception object** containing information about what went wrong.

2. What information does **Exception** contain?

The `Exception` class provides several important properties.

Message

Contains a human-readable description.

```
</> C#  
  
catch (Exception ex)  
{  
    Console.WriteLine(ex.Message);  
}
```

Example:

```
</> Plain text  
  
Attempted to divide by zero.
```

StackTrace

Shows the path through the code where the exception occurred.

Conceptually:



You can access it with:

```
</> C#  
  
ex.StackTrace
```

This is very useful for debugging.

InnerException

An exception can contain another exception that caused it.

```
</> C#
```

```
catch (Exception ex)
{
    Console.WriteLine(ex.InnerException);
}
```

For example:

</> Plain text

```
Exception
├── InnerException
│   └── SqlException
```

This commonly happens when a higher-level operation wraps a lower-level exception.

Source

Indicates the application/object that generated the exception.

```
</> C#
ex.Source
```

It's less commonly used in everyday application code than `Message` and `StackTrace`.

HelpLink

Can contain a URL or help reference associated with the exception.

```
</> C#
ex.HelpLink
```

It's not commonly used in normal ASP.NET Core applications.

3. Exception also has methods

ToString()

Returns a detailed textual representation of the exception, typically including:

- exception type
- message
- stack trace
- inner exception information

For example:

```
</> C#
Console.WriteLine(ex.ToString());
```

This is generally much more useful for logging than just:

```
</> C#
ex.Message
```

because `Message` only gives the message.

4. Exception has a constructor

Conceptually, you can create an exception:

```
</> C#
var ex = new Exception("Something went wrong");
```

The message is stored inside the exception object.

Then:

```
</> C#
Console.WriteLine(ex.Message);
```

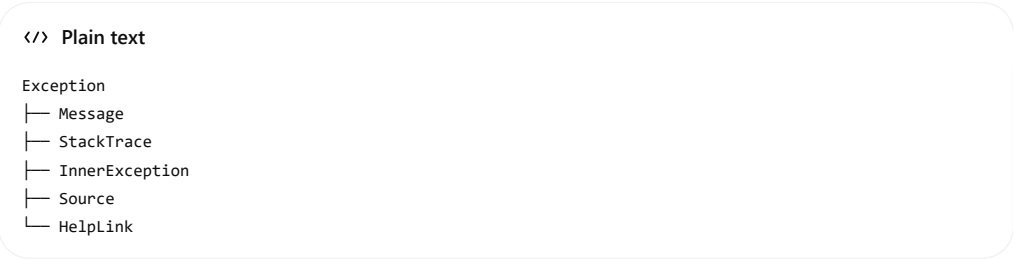
outputs:

```
</> Plain text
Something went wrong
```

You can also create one with an inner exception:

```
</> C#  
  
var ex = new Exception(  
    "Failed to process order",  
    originalException  
);
```

So the structure becomes:



5. Exception is a class, not an error itself

This distinction is important.

When you write:

```
</> C#  
  
catch (Exception ex)
```

you're saying:

"Catch any exception whose type is `Exception` or derives from it."

For example:

```
</> C#  
  
catch (Exception ex)
```

can catch:

```
</> Plain text
```

```
NullReferenceException  
ArgumentException  
ArgumentNullException  
InvalidOperationException  
IOException  
SqlException  
YourCustomException  
...
```

because they ultimately derive from `Exception`.

6. Why does every exception inherit from `Exception` ?

Because it gives all exceptions a **common structure and behavior**.

For example, regardless of whether you have:

```
</> C#  
  
NullReferenceException
```

or:

```
</> C#  
  
InvalidOperationException
```

you can do:

```
</> C#  
  
catch (Exception ex)  
{  
    Console.WriteLine(ex.Message);  
}
```

because both have the functionality inherited from `Exception`.

7. Custom exceptions

When you create:

```
</> C#  
  
public class ProductNotFoundException : Exception  
{  
}  
}
```

you're saying:

"ProductNotFoundException is an Exception, but it represents a specific situation in my application."

Therefore it automatically gets things such as:

```
</> Plain text  
  
Message  
StackTrace  
InnerException  
ToString()
```

from the base `Exception` class.

Interview answer

If the interviewer asks "What is the Exception class in C#?", you can say:

Exception is the base class for exceptions in .NET. It provides common information and behavior for errors, such as `Message`, `StackTrace`, `InnerException`, and `ToString()`. Built-in and custom exceptions generally derive from it, allowing them to be handled through a common `catch (Exception ex)` block.

The key mental model

```
</> Plain text  
  
Exception object  
├── What happened? → Message  
├── Where did it happen? → StackTrace  
├── What caused it? → InnerException  
└── Detailed representation → ToString()
```

And remember: an exception object is just an object containing information about an exceptional condition; **throw** is what actually raises it.

briefly give me a toString of real exception

A real `NullReferenceException` might look like this when you call `ToString()` :

```
</> Plain text  
  
System.NullReferenceException: Object reference not set to an instance of an object.  
   at MyApp.Services.UserService.GetUserName() in C:\MyApp\UserService.cs:line 25  
   at MyApp.Controllers.UserController.Get() in C:\MyApp\UserController.cs:line 42
```

It contains mainly:

- **Exception type:** `NullReferenceException`
- **Message:** what happened
- **Stack trace:** where it happened and how the code got there

So `ex.ToString()` gives you **much more information than** `ex.Message` .

another one for `innerexception`

For example, suppose a database error is wrapped inside a higher-level exception:

```
</> Plain text  
  
System.Exception: Failed to create order.  
---> System.InvalidOperationException: Database connection failed.  
   at MyApp.Data.OrderRepository.Save() in OrderRepository.cs:line 30  
--- End of inner exception stack trace ---  
   at MyApp.Services.OrderService.CreateOrder() in OrderService.cs:line 50
```

Here:

```
</> Plain text  
  
Outer Exception  
"Failed to create order."  
└─ InnerException  
   "Database connection failed."
```

So:

```
</> C#  
  
ex.Message
```

→ "Failed to create order."

while:

```
</> C#  
  
ex.InnerException.Message
```

→ "Database connection failed."

And `ex.ToString()` shows both the outer exception and the inner exception details.

explain params

In C#, `params` allows a method to accept a variable number of arguments of the same type.

1. Without `params`

Normally:

```
</> C#  
  
int Sum(int a, int b, int c)
```

You must provide exactly 3 arguments:

```
</> C#  
  
Sum(1, 2, 3);
```

2. With `params`

```
</> C#  
  
int Sum(params int[] numbers)  
{  
    // ...  
}
```

Now you can pass any number of integers:

```
</> C#
```

```
Sum();  
Sum(1);  
Sum(1, 2);  
Sum(1, 2, 3, 4, 5);
```

Inside the method, `numbers` is an **array**:

```
</> Plain text  
  
Sum(1, 2, 3)  
  
    ↓  
  
numbers  
┌ 1 | 2 | 3 ─┐
```

So you could do:

```
</> C#  
  
int Sum(params int[] numbers)  
{  
    int total = 0;  
  
    foreach (int n in numbers)  
        total += n;  
  
    return total;  
}
```

3. You can also pass an array

```
</> C#  
  
int[] values = { 1, 2, 3 };  
  
Sum(values);
```

So `params` gives you two convenient ways:

```
</> C#
```

```
Sum(1, 2, 3);

int[] nums = { 1, 2, 3 };
Sum(nums);
```

Important rules

params :

- Must be an **array parameter**.
- Must be the **last parameter**.
- Allows **zero or more** arguments.

For example:

```
</> C#

void Print(string name, params int[] numbers)
```

Valid:

```
</> C#

Print("Khaled");
Print("Khaled", 1, 2, 3);
```

Invalid:

```
</> C#

void Print(params int[] numbers, string name) // ❌
```

Interview answer

params allows a method to accept a variable number of arguments. The arguments are received inside the method as an array.

i don;t get how to inherit from list<> class studentlist :
List<Student>

Think of:

```
</> C#
```

```
class StudentList : List<Student>
```

as saying:

"StudentList IS a List<Student>, so it gets all the functionality of List<Student> ."

1. Normal List<Student>

You can do:

```
</> C#

List<Student> students = new List<Student>();

students.Add(student1);
students.Add(student2);

students.Remove(student1);

int count = students.Count;
```

2. Inheriting from it

Now:

```
</> C#

class StudentList : List<Student>
{
}
```

You can create:

```
</> C#

StudentList students = new StudentList();
```

And because StudentList inherits from List<Student> , it automatically has:

```
</> C#

students.Add(student1);
students.Remove(student1);
students.Count;
```

```
students.Contains(student1);
students.Clear();
```

Conceptually:

</> Plain text

```

List<Student>
  ↑
  | inherits
  |
StudentList

```

So:

</> C#

```
StudentList students = new StudentList();
```

is a `StudentList` , and because of inheritance it is also a `List<Student>` .

3. Why would you do this?

You can add your own functionality.

</> C#

```

class StudentList : List<Student>
{
    public Student GetTopStudent()
    {
        // custom logic
    }
}

```

Now:

</> C#

```

StudentList students = new StudentList();

students.Add(student1);
students.Add(student2);

students.GetTopStudent(); // your method

```

```

students.Count;           // inherited from List<Student>
students.Remove(student1); // inherited from List<Student>

```

So you're basically saying:

"Give me everything a `List<Student>` can do, **plus some student-specific functionality.**"

One important point

`List<T>` is a **generic class**.

When you write:

</> C#

```
List<Student>
```

you are creating a list specifically for `Student` objects.

Therefore:

</> C#

```
class StudentList : List<Student>
```

means:

</> Plain text

```

StudentList
  ↓
List<Student>
  ↓
IList<Student>
  ↓
ICollection<Student>
  ↓
IEnumerable<Student>

```

So `StudentList` gets the list behavior through the inheritance chain.

Interview phrase:

`class StudentList : List<Student>` means `StudentList` derives from the generic `List<Student>` class and inherits its members, while allowing us to add specialized behavior for

students.

explain data structure in C#

Sure. In C#, a **data structure** is a way of **organizing and storing data** so that we can access and manipulate it efficiently.

Think of it as:

Data structure = how you arrange your data in memory + how you work with it.

1. Main data structures you'll encounter in C#

A. Array

Fixed-size, ordered collection.

```
</> C#  
  
int[] numbers = { 10, 20, 30, 40 };
```

Access by index:

</> Plain text

Index:	0	1	2	3
	↓	↓	↓	↓
	[10]	[20]	[30]	[40]

```
</> C#  
  
numbers[2] // 30
```

Good for: fixed-size collections and fast index access.

B. List<T>

Dynamic array. Its size can grow/shrink.

```
</> C#
```

```
List<int> numbers = new();  
  
numbers.Add(10);  
numbers.Add(20);  
numbers.Add(30);
```

Conceptually:

```
</> Plain text  
  
[10] [20] [30] [ ] [ ]
```

It internally uses an array and increases its capacity when needed.

Good for: general-purpose ordered collections.

C. LinkedList<T>

Elements are connected through nodes.

```
</> Plain text  
  
[10] ⇄ [20] ⇄ [30] ⇄ [40]
```

Each node knows about neighboring nodes.

Good for: frequent insertion/removal when you already have the relevant node.

Unlike List<T> , it doesn't provide efficient random indexing.

D. Stack<T>

LIFO — Last In, First Out.

Think of a stack of plates:

```
</> Plain text  
  
[30] ← Top  
[20]  
[10]
```

```
</> C#  
  
Stack<int> stack = new();  
  
stack.Push(10);  
stack.Push(20);  
stack.Push(30);  
  
stack.Pop(); // 30
```

Good for:

- Undo operations
- Backtracking
- Function-call-like behavior
- DFS

E. Queue<T>

FIFO — First In, First Out.

Think of a waiting line:

```
</> Plain text  
  
First → [10] [20] [30] ← Last
```

```
</> C#  
  
Queue<int> queue = new();  
  
queue.Enqueue(10);  
queue.Enqueue(20);  
queue.Enqueue(30);  
  
queue.Dequeue(); // 10
```

Good for:

- Processing jobs
- Requests
- BFS
- Waiting-line scenarios

F. Dictionary<TKey,TValue>

Stores key-value pairs.

```
</> C#  
  
Dictionary<int, string> students = new();  
  
students[1] = "Ahmed";  
students[2] = "Khaled";  
students[3] = "Ali";
```

Conceptually:

```
</> Plain text  
  
Key      Value  
-----  
1   →   Ahmed  
2   →   Khaled  
3   →   Ali
```

You access the value using the key:

```
</> C#  
  
students[2] // Khaled
```

It uses hashing internally.

Good for: fast lookup by key.

G. HashSet<T>

Stores unique values.

```
</> C#  
  
HashSet<int> numbers = new();  
  
numbers.Add(10);  
numbers.Add(20);  
numbers.Add(10);
```

The second 10 isn't added.

```
</> Plain text
{ 10, 20 }
```

- Good for:
- Unique values
 - Fast membership checking

```
</> C#
numbers.Contains(20);
```

2. Important comparison

Structure	Main idea	Duplicates	Index
Array	Fixed-size sequence	✓	✓
List<T>	Dynamic array	✓	✓
LinkedList<T>	Nodes connected together	✓	✗
Stack<T>	LIFO	✓	✗
Queue<T>	FIFO	✓	✗
Dictionary<K,V>	Key → Value	Keys ✗	Key
HashSet<T>	Unique values	✗	✗

3. Data structures vs interfaces

This is important given what you've been learning.

You might see:

```
</> C#
```

and:

```
</> C#
List<Student>
```

They aren't the same thing.

IList<T> is an **interface** describing capabilities.

List<T> is the **actual collection implementation**.

```
</> Plain text
IEnumerable<T>
  ↑
ICollection<T>
  ↑
IList<T>
  ↑
List<T>
```

Similarly:

```
</> Plain text
IEnumerable<T>
  ↑
ICollection<T>
  ↑
ISet<T>
  ↑
HashSet<T>
```

So:

```
</> C#
IList<Student> students = new List<Student>();
```

means:

The actual object is a `List<Student>` , but I'm accessing it through the `IList<Student>` interface.

4. Data structures are about trade-offs

There isn't one "best" data structure.

For example:

Need fast index access?

```
</> C#  
  
List<int>
```

Need unique values?

```
</> C#  
  
HashSet<int>
```

Need lookup by ID?

```
</> C#  
  
Dictionary<int, Student>
```

Need LIFO?

```
</> C#  
  
Stack<Student>
```

Need FIFO?

```
</> C#  
  
Queue<Student>
```

Need fixed-size collection?

```
</> C#  
  
Student[]
```

Interview definition

A data structure is a way of organizing and storing data so that operations such as accessing, inserting, deleting, and searching can be performed efficiently. In C#, common data structures include arrays, lists, linked lists, stacks, queues, dictionaries, and hash sets.

The next important thing after understanding these is **Big-O complexity**—for example, why `List<T>` `[index]` is $O(1)$ while searching a list is generally $O(n)$, and why `Dictionary` lookup is usually $O(1)$ on average.

i don,t see an obvious benefit of linkedlist over list

You're right that `List<T>` is usually more useful in everyday C# code. The advantage of `LinkedList<T>` appears in a specific situation: **frequent insertions/removals in the middle when you already have the node.**

The key difference

`List<T>` → contiguous array

```
</> Plain text  
  
[10][20][30][40][50]
```

If you insert 25 between 20 and 30 :

```
</> Plain text  
  
[10][20][25][30][40][50]  
      ↑  
    shift →
```

30 , 40 , and 50 may need to be shifted.

So insertion is generally $O(n)$.

`LinkedList<T>` → connected nodes

```
</> Plain text

[10] ⇄ [20] ⇄ [30] ⇄ [40]
```

Suppose you already have the node containing 20 .

Insert 25 :

```
</> Plain text

Before:

[10] ⇄ [20] ⇄ [30]

After:

[10] ⇄ [20] ⇄ [25] ⇄ [30]
```

You just change the links.

Insertion is **O(1)** once you have the `LinkedListNode<T>` .

But here's the catch

Finding the 20 in a `LinkedList<T>` is itself generally **O(n)**:

```
</> C#

LinkedList.Find(20);
```

So if your workflow is:

```
</> Plain text

Find 20 → Insert after 20
```

the total is still generally **O(n)**.

The big advantage is when you **already have the node**.

```
</> C#

LinkedListNode<int> node = ...;
```

```
LinkedList.AddAfter(node, 25);
```

Now insertion is **O(1)**.

Another difference: index access

With `List<T>` :

```
</> C#

list[500];
```

is **O(1)**.

With `LinkedList<T>` :

```
</> C#

// No direct linkedList[500]
```

You have to traverse nodes, so accessing an arbitrary position is **O(n)**.

So when would you actually choose `LinkedList`?

Imagine a system where you continuously maintain a sequence and frequently move/remove/insert **known nodes**.

For example:

```
</> Plain text

Music playlist

Song A ⇄ Song B ⇄ Song C ⇄ Song D
```

If you already have a reference to Song B's node, inserting a song after it is very cheap.

But if you're simply doing:

```
</> Plain text
```

10/3/26, 7:48 PM

C# Recap

Add items

Read items by index

Loop through items

Search items

then `List<T>` is usually the natural choice.

Interview takeaway

Operation	List<T>	LinkedList<T>
list[index]	O(1)	O(n)
Search	O(n)	O(n)
Insert at end*	O(1) amortized	O(1)
Insert with known node	O(n)	O(1)
Remove with known node	O(n)	O(1)

* `List<T>` may occasionally resize its internal array.

The main idea: `LinkedList<T>` isn't "faster than `List`." It is optimized for a **different access pattern**. In typical application development, you'll encounter `List<T>` much more often.

1 selection

how

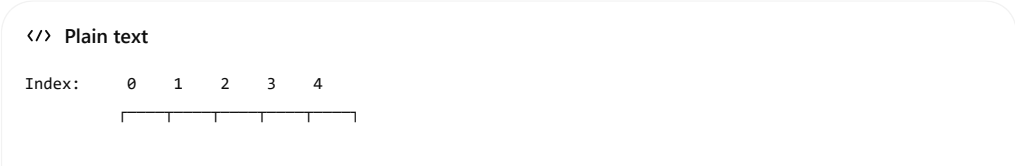
Because a `List<T>` is internally backed by an **array**.

Suppose:

</> C#

List<int> numbers = new() { 10, 20, 30, 40, 50 };

Conceptually, the internal array looks like:



10/3/26, 7:48 PM

C# Recap

Array: | 10 | 20 | 30 | 40 | 50 |

When you do:

</> C#

numbers[3]

`List<T>` doesn't start from index 0 and search:

</> Plain text

10 → 20 → 30 → 40

Instead, the index is used to directly access the corresponding position in the underlying array.

Conceptually:



So whether you request:

</> C#

numbers[3]

numbers[500]

numbers[900000]

the number of steps needed to calculate/access that position is essentially constant.

That's why:

Array/List index access = O(1)

Compare with LinkedList

A linked list looks like:

</> Plain text

[10] → [20] → [30] → [40] → [50]

If you want the 4th element, it has to follow:

</> Plain text

10 → 20 → 30 → 40

So accessing an arbitrary position requires traversal:

LinkedList positional access = O(n)

Simple mental model

</> Plain text

List<T>
[index] → element
direct access

LinkedList<T>
[start] → node → node → node → element
traversal

That's the fundamental reason List<T> has O(1) index access.

what is the bucket thing in dictionary

If you mean the "bucket" in Dictionary<TKey,TValue> , it's basically a place where the dictionary organizes entries based on their hash code.

Simple example

</> C#

Dictionary<int, string> students = new();

students.Add(10, "Ahmed");
students.Add(20, "Khaled");
students.Add(30, "Ali");

Conceptually, the dictionary has an array of buckets:

</> Plain text

Bucket 0 → ...
Bucket 1 → ...
Bucket 2 → 10 → "Ahmed"
Bucket 3 → ...
Bucket 4 → 20 → "Khaled"
Bucket 5 → 30 → "Ali"

When you ask:

</> C#

students[20]

the dictionary roughly does:

</> Plain text

Key 20
↓
GetHashCode()
↓
Determine bucket
↓
Go to that bucket
↓
Find key 20
↓
"Khaled"

Why buckets?

Instead of searching every key:

</> Plain text

10 → 20 → 30 → 40 → 50 → ...

the dictionary uses the key's hash code to quickly determine where to look.

That's why dictionary lookup is O(1) on average.

What if two keys go to the same bucket?

That's called a collision.

For example:

</> Plain text

```
Bucket 3
  ↓
[Key 10]
[Key 25]
[Key 40]
```

The dictionary has mechanisms to handle collisions and then checks the actual keys for equality.

So remember:

Bucket = a slot/group inside the dictionary's internal structure where entries are organized based on their hash codes.

explain var

var in C# means:

Let the compiler determine the variable's type from the value assigned to it.

It is called **implicit typing**.

Example

</> C#

```
var age = 26;
```

The compiler sees 26 is an int , so this is effectively:

</> C#

```
int age = 26;
```

Another example:

</> C#

```
var name = "Khaled";
```

The compiler determines:

</> C#

```
string name = "Khaled";
```

And:

</> C#

```
var students = new List<Student>();
```

becomes effectively:

</> C#

```
List<Student> students = new List<Student>();
```

Important: var is NOT dynamic

This is a common interview question.

</> C#

```
var x = 10;
```

x is still **strongly typed** as int .

You cannot later do:

</> C#

```
x = "hello"; // ❌
```

The type was determined at **compile time**.

With **dynamic** :

</> C#

```
dynamic x = 10;
x = "hello"; // ✅
```

The type behavior is resolved at runtime.

var must be initialized

This doesn't work:

```
</> C#
var x; // ❌
```

Because the compiler has no value from which to determine the type.

You need:

```
</> C#
var x = 10; // ✅
```

var vs explicit type

These are essentially equivalent:

```
</> C#
var age = 26;
```

```
</> C#
int age = 26;
```

The main difference is who writes the type:

```
</> Plain text
int age = 26;
↑
You specify it
```

```
</> Plain text
var age = 26;
↑
Compiler determines it
```

Interview answer

var enables implicit typing. The compiler determines the variable's type at compile time based on the initialization expression. It is still strongly typed and is not the same as **dynamic** .

readonly required init const

These four are easy to mix up because they all relate to **how properties/fields can be assigned**.

1. const

A **const** value is a **compile-time constant**.

```
</> C#
const double Pi = 3.14159;
```

Once declared, you cannot change it:

```
</> C#
Pi = 4; // ❌
```

Important:

- Must be initialized immediately.
- Cannot be changed.
- Usually used for fixed values.
- Implicitly **static** .

```
</> C#
const int MaxUsers = 100;
```

Think:

const = never changes and known at compile time.

2. readonly

readonly means a field can be assigned **only during initialization or in a constructor**.

```
</> C#
```

```
class Student
{
    private readonly int _id;

    public Student(int id)
    {
        _id = id;
    }
}
```

After construction:

```
</> C#

student._id = 10; // ❌
```

But different objects can have different values:

```
</> Plain text

Student 1 → _id = 10
Student 2 → _id = 20
```

So:

readonly = assigned once per object, usually during construction.

const vs readonly

```
</> Plain text

const
↓
same fixed value
compile-time

readonly
↓
can be different per object
runtime/constructor
```

3. init

init is mainly used with **properties**.

It means:

The property can be assigned during object initialization, but not modified afterward.

```
</> C#

class Student
{
    public string Name { get; init; }
}
```

You can:

```
</> C#

var student = new Student
{
    Name = "Khaled"
};
```

But afterward:

```
</> C#

student.Name = "Ahmed"; // ❌
```

So:

```
</> Plain text

object creation
↓
Name = "Khaled" ✅
↓
object exists
↓
Name = "Ahmed" ❌
```

set vs init

```
</> C#

public string Name { get; set; }
```

Can change anytime:

```
</> C#

student.Name = "Ahmed"; // ✔️
student.Name = "Ali";   // ✔️
```

With:

```
</> C#

public string Name { get; init; }
```

Only initialization:

```
</> C#

new Student { Name = "Khaled" }; // ✔️
student.Name = "Ahmed";          // ❌
```

4. required

required means:

The caller must provide a value for this property when creating the object.

```
</> C#

class Student
{
    public required string Name { get; set; }
}
```

Now:

```
</> C#

var student = new Student
{
    Name = "Khaled"
};
```

✔️ Valid.

But:

```
</> C#

var student = new Student();
```

❌ Compiler error because Name is required.

5. required + init

You'll often see them together:

```
</> C#

class Student
{
    public required string Name { get; init; }
}
```

This means **two different things**:

required

You **must provide** the property.

init

Once provided, you **cannot change it afterward**.

```
</> C#

var student = new Student
{
    Name = "Khaled"
};
```

Then:

```
</> C#

student.Name = "Ahmed"; // ❌
```

Quick comparison

Keyword	Used mainly with	Meaning
const	fields/local constants	Fixed compile-time value
readonly	fields	Assign only during initialization/constructor
init	properties	Assign only during object initialization
required	properties/fields	Caller must initialize it

Easy way to remember

</> Plain text

```
const    → Cannot change
readonly → Assign during construction, then cannot change
init     → Assign during object initialization, then cannot change
required → You MUST provide a value
```

And one useful combination:

</> C#

```
public required string Name { get; init; }
```

means:

"You must give me a Name when creating this object, and after creation you can't change it."

```
Subject sub = new Subject(1, "linq", 22) { id = 5, name = "EF",
duration = 12 };
```

This is an example of **object initialization combined with a constructor call**.

</> C#

```
Subject sub = new Subject(1, "linq", 22)
{
    id = 5,
    name = "EF",
    duration = 12
};
```

Let's break it down.

1. Constructor runs first

</> C#

```
new Subject(1, "linq", 22)
```

C# creates the object and calls the constructor:

</> C#

```
Subject(int id, string name, int duration)
```

So initially:

</> Plain text

```
id      = 1
name    = "linq"
duration = 22
```

2. Object initializer runs afterward

Then:

</> C#

```
{
    id = 5,
    name = "EF",
    duration = 12
}
```

assigns the properties/fields again:

</> Plain text

```
id      = 5
name    = "EF"
duration = 12
```

So the **final object** contains:

</> Plain text

```
id      → 5
name    → "EF"
duration → 12
```

Equivalent idea

It's roughly like:

</> C#

```
Subject sub = new Subject(1, "ling", 22);
```

```
sub.id = 5;
sub.name = "EF";
sub.duration = 12;
```

So the important point is:

Constructor arguments initialize the object first, then the object initializer assigns the specified properties/fields afterward.

Why would you do this?

Usually, you wouldn't pass values to the constructor and then immediately overwrite the same values. You might instead do either:

</> C#

```
var sub = new Subject(1, "ling", 22);
```

or

</> C#

```
var sub = new Subject
{
    id = 5,
    name = "EF",
    duration = 12
};
```

The second approach is particularly useful when properties have `init` setters:

</> C#

```
public int id { get; init; }
```

because the initializer can set them **during object creation**, but they cannot be changed afterward.

explain record

In C#, a **record** is a type designed mainly for **holding data**, with **value-based equality** built in.

The easiest way to understand it is:

Class → usually identity-based equality

Record → value-based equality

1. Normal class

</> C#

```
class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Now:

</> C#

```
var s1 = new Student { Id = 1, Name = "Khaled" };
var s2 = new Student { Id = 1, Name = "Khaled" };

Console.WriteLine(s1 == s2);
```

Normally:

</> Plain text

```
false
```

Because `s1` and `s2` are **two different objects**.

2. Record

```
</> C#  
  
record Student(int Id, string Name);
```

Now:

```
</> C#  
  
var s1 = new Student(1, "Khaled");  
var s2 = new Student(1, "Khaled");  
  
Console.WriteLine(s1 == s2);
```

Result:

```
</> Plain text  
  
true
```

Because records compare their **data/value**.

```
</> Plain text  
  
s1 → Id = 1, Name = Khaled  
s2 → Id = 1, Name = Khaled  
  
Same values → Equal
```

That's the main feature.

3. Records are reference types by default

This is important.

```
</> C#  
  
record Student(int Id, string Name);
```

is a **reference type**, just like a class.

So don't think:

```
</> Plain text  
  
record = struct
```

No.

You can also have:

```
</> C#  
  
record struct Student(int Id, string Name);
```

which is a **value type**.

So:

```
</> Plain text  
  
record           → reference type  
record struct → value type
```

4. Records are great for DTOs

For example, in ASP.NET Core:

```
</> C#  
  
public record StudentDto(int Id, string Name);
```

You can create:

```
</> C#  
  
var student = new StudentDto(1, "Khaled");
```

This is very convenient for objects that mainly **carry data**.

That's why you'll often see records used for:

- DTOs
- API request/response models
- CQRS commands/queries

- Immutable data models

For example:

```
</> C#  
  
public record CreateStudentCommand(  
    string Name,  
    int Age  
)  
;
```

5. with expression

Records have another very useful feature: with .

```
</> C#  
  
var student1 = new Student(1, "Khaled");  
  
var student2 = student1 with  
{  
    Name = "Ahmed"  
};
```

You get a new record:

```
</> Plain text  
  
student1 → Id=1, Name=Khaled  
  
student2 → Id=1, Name=Ahmed
```

The original isn't modified.

This works especially well with immutable data.

6. Record and init

Records commonly use immutable properties.

```
</> C#  
  
public record Student  
{
```

```
public int Id { get; init; }  
public string Name { get; init; }  
}
```

You can initialize:

```
</> C#  
  
var student = new Student  
{  
    Id = 1,  
    Name = "Khaled"  
};
```

But afterward:

```
</> C#  
  
student.Name = "Ahmed"; // ✖
```

You can instead use:

```
</> C#  
  
var updatedStudent = student with  
{  
    Name = "Ahmed"  
};
```

7. Record vs class

Feature	class	record
Reference type	✓	✓
Value-based equality by default	✖	✓
with expression	✖	✓
Good for DTOs	✓	★
Good for entities	★	Usually not preferred

Feature	class	record
Immutable style	Possible	Natural

Why usually not records for EF Core entities?

Entities often have **identity and mutable state**.

For example:

</> Plain text

Student ID 5

represents a particular student even if their name changes.

Records are designed around **data/value equality**, which can be undesirable for entity identity.

Interview answer

A record is a C# type designed primarily for data-centric models. Unlike a normal class, records provide value-based equality by default and support features such as non-destructive mutation with `with`. A normal `record` is a reference type, while `record struct` is a value type.

The biggest thing to remember:

</> Plain text

class → "Are these the same object?"
record → "Do these objects contain the same data?"

record struct

`record struct` combines two ideas:

- `record` → value-based equality
- `struct` → value type

So:

</> C#

public record struct Student(int Id, string Name);

is a **value type** with **record-style value equality**.

Example

</> C#

var s1 = new Student(1, "Khaled");
var s2 = new Student(1, "Khaled");

Console.WriteLine(s1 == s2);

Result:

</> Plain text

true

Because their values are equal.

Compare with a normal struct:

</> C#

public struct Student
{
 public int Id;
 public string Name;
}

A normal struct also has value semantics, but `record struct` gives you additional record features and convenient generated members.

record vs record struct

</> Plain text

record Student
↓
reference type
value-based equality

record struct Student
↓

```
value type
value-based equality
```

For example:

```
</> C#

public record Student(int Id, string Name);
```

```
</> C#

public record struct Student(int Id, string Name);
```

The first is a **class-like reference type**; the second is a **struct-like value type**.

with also works

```
</> C#

var s1 = new Student(1, "Khaled");

var s2 = s1 with
{
    Name = "Ahmed"
};
```

`s1` remains unchanged, while `s2` is a separate value.

When useful?

`record struct` is useful for **small data values** where you want:

- value semantics
- lightweight value-type behavior
- concise syntax
- record features such as `with`

For example, a coordinate:

```
</> C#

public record struct Point(int X, int Y);
```

Then:

```
</> C#

var p1 = new Point(10, 20);
var p2 = new Point(10, 20);

p1 == p2; // true
```

Interview answer:

A `record struct` is a value type that provides record features such as value-based equality and concise data-oriented syntax.

explain partial class and its uses also explain primary ctor

Sure. These are two useful modern C# features.

1. Partial class

A **partial class** allows you to split **one class into multiple files**.

For example, instead of putting everything in one file:

```
</> C#

public partial class Student
{
    public int Id { get; set; }
    public string Name { get; set; }

    public void Study()
    {
    }

    public void TakeExam()
    {
    }
}
```

You can split it:

Student.cs

```
</> C#

public partial class Student
{
    public int Id { get; set; }
}
```

```
public string Name { get; set; }
}
```

StudentMethods.cs

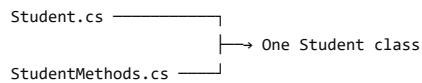
```
</> C#

public partial class Student
{
    public void Study()
    {
    }

    public void TakeExam()
    {
    }
}
```

The compiler combines them into **one Student class**.

</> Plain text



```
Student.cs -----|
                  |-----> One Student class
StudentMethods.cs -----|
```

You still create it normally:

```
</> C#

var student = new Student();
```

There are **not two Student objects**.

Why use partial classes?

1. Generated code

This is one of the biggest uses.

Some tools/frameworks generate part of a class automatically.

For example, you might have:

</> Plain text

```
GeneratedStudent.cs
YourStudent.cs
```

The generated file can contain generated code while you keep your custom code separately.

Then when the tool regenerates its file, your code isn't overwritten.

2. Large classes

If a class becomes very large, you can organize different responsibilities into separate files.

However, don't use partial classes simply to hide a poorly designed giant class. Often the better solution is to split it into multiple classes.

3. Source generators / framework tooling

Modern .NET uses generated source in various scenarios, and partial types allow generated code and developer-written code to work together.

Important rule

Every part must use `partial` :

</> C#

```
public partial class Student
```

and:

</> C#

```
public partial class Student
```

You cannot do:

</> C#

```
public class Student
```

in one file and `partial class Student` in another.

2. Primary constructor

A **primary constructor** allows you to declare constructor parameters directly in the class/struct declaration.

Instead of:

```
</> C#

public class Student
{
    public int Id { get; }
    public string Name { get; }

    public Student(int id, string name)
    {
        Id = id;
        Name = name;
    }
}
```

You can write:

```
</> C#

public class Student(int id, string name)
{
    public int Id { get; } = id;
    public string Name { get; } = name;
}
```

Much shorter.

What exactly happened?

This:

```
</> C#

public class Student(int id, string name)
```

declares the **primary constructor parameters**:

</> Plain text

```
id
name
```

They are available throughout the class body.

Then:

</> C#

```
public int Id { get; } = id;
```

stores the parameter in the property.

Primary constructor ≠ automatically properties

This is important.

If you write:

</> C#

```
public class Student(int id, string name)
{
}
```

you do **not** automatically get:

</> C#

```
student.Id
student.Name
```

The parameters are not automatically properties.

If you want properties:

</> C#

```
public class Student(int id, string name)
{
```

```
public int Id { get; } = id;
public string Name { get; } = name;
}
```

3. Primary constructor with dependency injection

This is especially useful in **ASP.NET Core**, so you'll likely see it often.

Traditional:

```
</> C#

public class StudentService
{
    private readonly IStudentRepository _repository;

    public StudentService(IStudentRepository repository)
    {
        _repository = repository;
    }
}
```

With a primary constructor:

```
</> C#

public class StudentService(IStudentRepository repository)
{
    public void GetStudents()
    {
        repository.GetAll();
    }
}
```

The constructor parameter `repository` is available inside the class.

This makes DI classes much shorter.

4. Primary constructor vs normal constructor

Traditional

```
</> C#
```

```
public class Student
{
    private readonly int _id;

    public Student(int id)
    {
        _id = id;
    }
}
```

Primary constructor

```
</> C#

public class Student(int id)
{
    private readonly int _id = id;
}
```

Same basic idea, less boilerplate.

5. Primary constructors work with structs too

You can write:

```
</> C#

public struct Point(int x, int y)
{
    public int X { get; } = x;
    public int Y { get; } = y;
}
```

And:

```
</> C#

var point = new Point(10, 20);
```

6. Primary constructor + record

You may have already seen this:

```
</> C#  
  
public record Student(int Id, string Name);
```

This looks very similar, but **records have additional generated behavior**.

For a normal class:

```
</> C#  
  
public class Student(int id, string name)  
{  
}  
}
```

the parameters are simply primary constructor parameters.

For:

```
</> C#  
  
public record Student(int Id, string Name);
```

the record syntax also generates corresponding properties and record behavior.

Interview summary

Partial class

A partial class allows one class to be split across multiple files. The compiler combines all parts into one class. It's commonly used with generated code and source generators.

Primary constructor

A primary constructor allows constructor parameters to be declared directly in the class or struct declaration, reducing constructor boilerplate. The parameters are available inside the type body but aren't automatically properties for normal classes.

The two patterns you'll most commonly see in your .NET code are:

```
</> C#  
  
public partial class MyClass  
{
```

```
}
```

and:

```
</> C#  
  
public class MyService(IMyRepository repository)  
{  
    // use repository here  
}
```

The second one is particularly common in **modern ASP.NET Core dependency injection**.

so any parameter in primary ctor is available through the entire class?

Yes — for a normal class, primary constructor parameters are in scope throughout the class body.

For example:

```
</> C#  
  
public class Student(int id, string name)  
{  
    public void Print()  
    {  
        Console.WriteLine(id);  
        Console.WriteLine(name);  
    }  
  
    public string GetName()  
    {  
        return name;  
    }  
}
```

id and name are available inside:

- Methods
- Properties
- Field initializers
- Property initializers
- Constructors
- Other members of the class

But important distinction

The parameters are **not automatically stored as fields/properties**.

```
</> C#

public class Student(int id, string name)
{
}
```

You can use `id` inside the class, but you don't automatically get:

```
</> C#

student.id // ❌
student.name // ❌
```

If you want them to become properties:

```
</> C#

public class Student(int id, string name)
{
    public int Id { get; } = id;
    public string Name { get; } = name;
}
```

Now:

```
</> C#

var student = new Student(1, "Khaled");

Console.WriteLine(student.Id); // ✅
Console.WriteLine(student.Name); // ✅
```

One more important detail

The parameter is available **throughout the type body**, but if the class needs to retain that value as part of its state, the compiler may capture/store it when you reference the primary constructor parameter from an instance member.

So the interview-level rule is:

Primary constructor parameters are in scope throughout the class body, but they are not automatically public properties.

Today 2:10 PM

give me all the possible scenarios of inheritance and polymorphism like a class inherits 2 interfaces with same method signature and name and scenarios like that

Yes. For a C# interview, there are several **inheritance + polymorphism edge cases** you should know.

The key is to separate:

1. Class → class inheritance
2. Class → multiple interfaces
3. Interface → interface inheritance
4. Same member name/signature conflicts
5. Overriding vs hiding
6. Abstract members
7. Interface default implementations

Here is the full practical map.

1. Class inherits from one class

C# supports **single class inheritance**.

```
</> C#

class Animal
{
    public virtual void Speak() { }
}

class Dog : Animal
{
    public override void Speak() { }
}
```

```
</> C#

Animal a = new Dog();
a.Speak();
```

What happens?

`a` is an `Animal` reference, but the actual object is `Dog`.

Because `Speak()` is `virtual` + `override`, **runtime polymorphism** calls:

</> Plain text

```
Dog.Speak()
```

2. Class cannot inherit from two classes

✗ Not allowed:

</> C#

```
class Dog : Animal, Mammal
{
}
```

A class can have only **one base class**.

But it can implement multiple interfaces.

3. Class implements multiple interfaces

</> C#

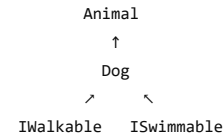
```
interface IWalkable
{
    void Move();
}

interface ISwimmable
{
    void Swim();
}

class Dog : Animal, IWalkable, ISwimmable
{
    public void Move() { }
    public void Swim() { }
}
```

This is completely valid.

</> Plain text



4. Two interfaces have the SAME method

This is one of the most important interview scenarios.

</> C#

```
interface IPrinter
{
    void Print();
}

interface IScanner
{
    void Print();
}
```

A class can implement both with **one method**:

</> C#

```
class Machine : IPrinter, IScanner
{
    public void Print()
    {
        Console.WriteLine("Print");
    }
}
```

The single `Print()` satisfies both interfaces.

</> C#

```
IPrinter p = new Machine();
IScanner s = new Machine();

p.Print(); // Print
s.Print(); // Print
```

Why?

Both interfaces require:

```
</> Plain text
```

```
void Print()
```

and `Machine.Print()` matches both contracts.

5. Two interfaces have SAME method name but DIFFERENT parameters

```
</> C#
```

```
interface IPrinter
{
    void Print(int pages);
}
```

```
interface IScanner
{
    void Print(string document);
}
```

One class can implement both through **method overloading**:

```
</> C#
```

```
class Machine : IPrinter, IScanner
{
    public void Print(int pages) { }

    public void Print(string document) { }
}
```

This is not a conflict because the signatures are different.

```
</> Plain text
```

```
Print(int)
Print(string)
```

6. Two interfaces have SAME method signature but need DIFFERENT behavior

This is where **explicit interface implementation** becomes important.

```
</> C#
```

```
interface IEnglish
{
    void Speak();
}
```

```
interface IArabic
{
    void Speak();
}
```

Suppose you want different implementations:

```
</> C#
```

```
class Person : IEnglish, IArabic
{
    void IEnglish.Speak()
    {
        Console.WriteLine("Hello");
    }

    void IArabic.Speak()
    {
        Console.WriteLine("مرحبا");
    }
}
```

Now:

```
</> C#
```

```
IEnglish english = new Person();
IArabic arabic = new Person();

english.Speak(); // Hello
arabic.Speak(); // مرحبا
```

This is called:

Explicit interface implementation

7. Explicit interface implementation cannot be called through the class directly

With:

```
</> C#

class Person : IEnglish, IArabic
{
    void IEnglish.Speak() { }
    void IArabic.Speak() { }
}
```

This doesn't work:

```
</> C#

Person p = new Person();

p.Speak(); // ❌
```

Because there is no public `Person.Speak()` .

You must use the interface reference:

```
</> C#

IEnglish p = new Person();
p.Speak();
```

or:

```
</> C#

((IEnglish)person).Speak();
```

8. One interface + class has same method

Consider:

```
</> C#
```

```
interface IAnimal
{
    void Speak();
}

class Dog : IAnimal
{
    public void Speak()
    {
        Console.WriteLine("Woof");
    }
}
```

You can access it through either:

```
</> C#

Dog dog = new Dog();
dog.Speak();
```

or:

```
</> C#

IAnimal animal = new Dog();
animal.Speak();
```

Same implementation.

9. Base class + interface have the same method

Very important.

```
</> C#

class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal");
    }
}

interface IAnimal
{
    void Speak();
}
```

```

    }

    class Dog : Animal, IAnimal
    {
        public override void Speak()
        {
            Console.WriteLine("Dog");
        }
    }
}

```

The `override` method can also satisfy the interface.

```

</> C#

Animal a = new Dog();
a.Speak();      // Dog

IAnimal i = new Dog();
i.Speak();      // Dog

Dog d = new Dog();
d.Speak();      // Dog

```

One implementation serves both the base class contract and interface contract.

10. Base class has NON-virtual method + interface has same method

```

</> C#

class Animal
{
    public void Speak()
    {
        Console.WriteLine("Animal");
    }
}

interface IAnimal
{
    void Speak();
}

class Dog : Animal, IAnimal
{
    public void Speak()

```

```

    {
        Console.WriteLine("Dog");
    }
}

```

Now things become interesting.

```

</> C#

Dog dog = new Dog();
dog.Speak();

```

Output:

```

</> Plain text

Dog

```

But:

```

</> C#

Animal animal = new Dog();
animal.Speak();

```

Output:

```

</> Plain text

Animal

```

Because `Animal.Speak()` is **not virtual**.

The derived `Dog.Speak()` is hiding the base method, not overriding it.

11. virtual + override

This is normal runtime polymorphism.

```

</> C#

class Animal
{
    public virtual void Speak()

```

```

    {
        Console.WriteLine("Animal");
    }
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog");
    }
}

```

```

</> C#

Animal animal = new Dog();

animal.Speak();

```

Output:

```

</> Plain text

Dog

```

Because runtime type = Dog .

12. new method hiding

```

</> C#

class Animal
{
    public void Speak()
    {
        Console.WriteLine("Animal");
    }
}

class Dog : Animal
{
    public new void Speak()
    {
        Console.WriteLine("Dog");
    }
}

```

```

    }
}

```

Now:

```

</> C#

Dog dog = new Dog();
dog.Speak();    // Dog

```

But:

```

</> C#

Animal animal = new Dog();
animal.Speak();    // Animal

```

Because new means:

"Hide the base member."

It does **not** mean override.

13. Multiple levels of overriding

```

</> C#

class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal");
    }
}

class Mammal : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Mammal");
    }
}

class Dog : Mammal
{

```

```
public override void Speak()
```



Then:

```
</> C#
```

```
Animal a = new Dog();
a.Speak();
```

Output:

```
</> Plain text
```

```
Dog
```

Runtime dispatch goes to the most-derived override.

14. Interface inherits another interface

Interfaces can have multiple inheritance.

```
</> C#
```

```
interface IAnimal
{
    void Speak();
}
```

```
interface IMammal
{
    void Feed();
}
```

```
interface IDog : IAnimal, IMammal
{
    void Bark();
}
```

A class implementing `IDog` must satisfy all contracts:

```
</> C#
```

```
class Dog : IDog
{
```

```
public void Speak() { }
public void Feed() { }
public void Bark() { }
}
```

Think:

```
</> Plain text
```

```

IAnimal      IMammal
  ↑           ↑
  \           /
   IDog
   ↑
  Dog
```

15. Interface inherits two interfaces with SAME method

```
</> C#
```

```
interface IA
{
    void DoSomething();
}
```

```
interface IB
{
    void DoSomething();
}
```

```
interface IC : IA, IB
{
}
```

This is valid.

`IA` and `IB` require the same method, so `IC` effectively has one required contract:

```
</> C#
```

```
void DoSomething();
```

Then:

```
</> C#

class MyClass : IC
{
    public void DoSomething()
    {
    }
}
```

One implementation satisfies both.

16. Interface inheritance + different implementations

Suppose:

```
</> C#

interface IA
{
    void DoSomething();
}

interface IB
{
    void DoSomething();
}

class MyClass : IA, IB
{
    void IA.DoSomething()
    {
        Console.WriteLine("A");
    }

    void IB.DoSomething()
    {
        Console.WriteLine("B");
    }
}
```

Then:

```
</> C#

IA a = new MyClass();
IB b = new MyClass();

a.DoSomething(); // A
b.DoSomething(); // B
```

Again, explicit implementation solves the ambiguity.

17. Interface default implementation

Modern C# interfaces can provide implementations.

```
</> C#

interface IAnimal
{
    void Speak()
    {
        Console.WriteLine("Animal");
    }
}
```

A class can simply implement it:

```
</> C#

class Dog : IAnimal
{
}
```

Then:

```
</> C#

IAnimal animal = new Dog();

animal.Speak();
```

The interface's default implementation is used.

18. Class overrides interface default implementation

```
</> C#

interface IAnimal
{
    void Speak()
    {
```

```

        Console.WriteLine("Animal");
    }
}

class Dog : IAnimal
{
    public void Speak()
    {
        Console.WriteLine("Dog");
    }
}

```

Now:

```

</> C#

IAnimal animal = new Dog();

animal.Speak();

```

Output:

```

</> Plain text

Dog

```

The class implementation takes precedence.

19. Two interfaces have conflicting default implementations

This is another important edge case.

```

</> C#

interface IA
{
    void Speak()
    {
        Console.WriteLine("A");
    }
}

interface IB
{
    void Speak()
    {

```

```

        Console.WriteLine("B");
    }
}

```

Then:

```

</> C#

class C : IA, IB
{
}

```

The class has a conflict if you try to use `Speak()` through the interface because both interfaces provide different default implementations.

A common solution is for `C` to provide its own implementation:

```

</> C#

class C : IA, IB
{
    public void Speak()
    {
        Console.WriteLine("C");
    }
}

```

Or use explicit interface implementations when you need separate behavior.

20. Abstract class + interface

A class can inherit an abstract class and implement interfaces:

```

</> C#

abstract class Animal
{
    public abstract void Speak();
}

interface IWalkable
{
    void Walk();
}

```

```
class Dog : Animal, IWalkable
{
    public override void Speak()
    {
    }

    public void Walk()
    {
    }
}
```

Dog must implement:

</> Plain text

```
Animal.Speak()
IWalkable.Walk()
```

21. Abstract class implements interface but doesn't implement it

This is valid.

```
</> C#

interface IAnimal
{
    void Speak();
}

abstract class Animal : IAnimal
{
}
```

Animal doesn't have to implement Speak() because it is abstract.

A concrete derived class must eventually implement it:

```
</> C#

class Dog : Animal
{
    public void Speak()
    {
    }
}
```

22. Abstract class partially implements interface

```
</> C#

interface IAnimal
{
    void Speak();
    void Move();
}

abstract class Animal : IAnimal
{
    public void Move()
    {
    }

    public abstract void Speak();
}
```

Then:

```
</> C#

class Dog : Animal
{
    public override void Speak()
    {
    }
}
```

The abstract class handles one part; derived class handles the remaining part.

23. Class inherits base class that implements interface

```
</> C#

interface IAnimal
{
    void Speak();
}

class Animal : IAnimal
{
    public void Speak()
    {
    }
}
```

```

}

class Dog : Animal
{
}

```

Dog automatically implements IAnimal .

You don't have to write:

```

</> C#

class Dog : Animal, IAnimal

```

because it inherits the implementation from Animal .

24. Derived class implements another interface

```

</> C#

interface IAnimal
{
    void Speak();
}

interface IWalkable
{
    void Walk();
}

class Animal : IAnimal
{
    public void Speak() { }
}

class Dog : Animal, IWalkable
{
    public void Walk() { }
}

```

Dog supports both:

```

</> Plain text

```

```

IAnimal
↑
Animal
↑
Dog
↑
IWalkable

```

25. Same interface implemented by base and derived class

```

</> C#

interface IAnimal
{
    void Speak();
}

class Animal : IAnimal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal");
    }
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog");
    }
}

```

Dog doesn't need to explicitly say IAnimal because Animal already implemented it.

26. Explicit interface implementation + public method with same name

Very useful edge case.

```

</> C#

interface IA
{
    void Speak();
}

```

```

}

class Dog : IA
{
    public void Speak()
    {
        Console.WriteLine("Public");
    }

    void IA.Speak()
    {
        Console.WriteLine("Interface");
    }
}

```

Now:

```

</> C#

Dog dog = new Dog();
dog.Speak();

```

Output:

```

</> Plain text

Public

```

But:

```

</> C#

IA animal = new Dog();
animal.Speak();

```

Output:

```

</> Plain text

Interface

```

So the class can have:

```

</> Plain text

```

```

Dog.Speak()      → Public implementation
IA.Speak()       → Explicit interface implementation

```

27. Base class + two interfaces + same method

This is probably the ultimate interview scenario.

```

</> C#

class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal");
    }
}

interface IA
{
    void Speak();
}

interface IB
{
    void Speak();
}

```

Option A — one override satisfies everything

```

</> C#

class Dog : Animal, IA, IB
{
    public override void Speak()
    {
        Console.WriteLine("Dog");
    }
}

```

Then:

```

</> C#

Animal a = new Dog();
IA x = new Dog();
IB y = new Dog();

```

```
a.Speak(); // Dog
x.Speak(); // Dog
y.Speak(); // Dog
```

One method satisfies all three.

28. Base class + two interfaces with different behavior

If you want different interface behavior:

```
</> C#

class Dog : Animal, IA, IB
{
    public override void Speak()
    {
        Console.WriteLine("Animal/Dog behavior");
    }

    void IA.Speak()
    {
        Console.WriteLine("IA behavior");
    }

    void IB.Speak()
    {
        Console.WriteLine("IB behavior");
    }
}
```

Now:

```
</> C#

Dog d = new Dog();
d.Speak();      // Animal/Dog behavior

Animal a = d;
a.Speak();      // Animal/Dog behavior

IA x = d;
x.Speak();      // IA behavior

IB y = d;
y.Speak();      // IB behavior
```

This demonstrates different polymorphic views of the same object.

29. The most important mental model

Whenever you see:

```
</> C#

SomeType variable = new ActualType();
```

ask two questions:

1. What is the variable/reference type?

This determines **what members you can access**.

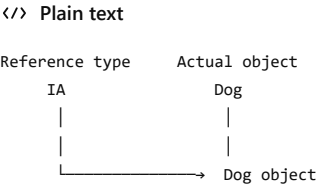
2. What is the actual object type?

This determines **runtime polymorphic behavior** for virtual/interface dispatch.

For example:

```
</> C#

IA x = new Dog();
```



x can only directly access members exposed by IA , but the actual implementation comes from Dog where applicable.

Interview cheat sheet

Scenario	Allowed?	Key concept
Class inherits class	✔	Single inheritance

Scenario	Allowed?	Key concept
Class inherits 2 classes	✗	C# doesn't support multiple class inheritance
Class implements multiple interfaces	✓	Multiple interface implementation
Two interfaces same method	✓	One implementation can satisfy both
Two interfaces same method, different behavior	✓	Explicit implementation
Same name, different parameters	✓	Method overloading
Base virtual + derived override	✓	Runtime polymorphism
Base non-virtual + derived same method	✓	Method hiding
new method	✓	Hides base member
Interface inherits multiple interfaces	✓	Multiple interface inheritance
Two interfaces inherit same interface	✓	Interface diamond is allowed
Abstract class implements interface	✓	Abstract class can defer implementation
Concrete class inherits interface implementation	✓	Implementation is inherited
Interface default implementation	✓	Modern C#
Two interfaces with conflicting defaults	Needs resolution	Class/explicit implementation
Base class + multiple interfaces same method	✓	One override can satisfy all
Base class + interfaces different behavior	✓	Explicit interface implementation

The 5 cases I'd memorize for interviews

</> Plain text

1. Class → Class
↓

virtual/override

2. Class → Multiple Interfaces
↓
one implementation

3. Multiple Interfaces
same signature
↓
explicit implementation
↓
IA x → behavior A
IB y → behavior B

The **biggest interview trap** is confusing **overriding** with **hiding** and **explicit interface implementation**. If you understand those three, most inheritance/polymorphism questions become straightforward.

Test me

Absolutely — let’s test your **C# inheritance + polymorphism**, especially the tricky interface cases.

Give me another 10-question C# inheritance and polymorphism quiz, harder than this one, focusing on interface conflicts, explicit implementation, virtual/override/new, abstract classes, and runtime dispatch.

Give me another 10-question C# inheritance and polymorphism quiz at expert interview difficulty, with fresh scenarios involving interface default implementations, interface diamonds, abstract overrides, method hiding across multiple inheritance levels, explicit implementations, and runtime dispatch.

Here’s a fresh **expert-level** set. I’ve avoided recycling the previous scenarios and pushed the questions toward subtle dispatch rules and interface resolution.

ok all good

Nice — **7/9 on the Expert quiz** so far, with the two misses being the genuinely tricky parts:

- Q6:** abstract override is valid. An abstract class can re-abstract an inherited virtual method.

- **Q8:** A class's explicit interface implementation takes precedence over the interface's default implementation for that interface call.

Your earlier Hard Mode score was **9/9**, so you're handling `new` vs `override` , interface conflicts, and runtime dispatch very well.

The next useful step would be an **expert code-tracing round** where you predict the exact output of 10 multi-level inheritance/interface programs.

give me in details a detailed comparison between interface and abstract

Absolutely. Since you're preparing for **C#/.NET interviews**, the important thing is not just memorizing “interface vs abstract class,” but understanding **when each is appropriate, what the CLR/C# actually allows, and how polymorphism behaves.**

Interface vs Abstract Class in C#

1. The fundamental difference

Think of them this way:

Interface = contract / capability

An interface answers:

“What can this object do?”

</> Plain text

IPrintable
↓
Can be printed

ISavable
↓
Can be saved

IAuthenticatable
↓
Can be authenticated

A class can implement many interfaces.

Abstract class = common base / shared identity

An abstract class answers:

“What is this object, and what common behavior/state does it share with related objects?”

</> Plain text

```
graph TD
    Animal --> Dog
    Animal --> Cat
```

`Dog` and `Cat` are both `Animal` , so inheritance makes sense.

2. Basic syntax

Interface

</> C#

```
public interface IAnimal
{
    void Speak();
}
```

A class implements it:

</> C#

```
public class Dog : IAnimal
{
    public void Speak()
    {
        Console.WriteLine("Woof");
    }
}
```

Abstract class

</> C#

```
public abstract class Animal
{
```

```
public abstract void Speak();

public void Eat()
{
    Console.WriteLine("Eating");
}
}
```

Derived class:

```
</> C#

public class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Woof");
    }
}
```

The important difference is that the abstract class can provide **shared implementation and state**, while still forcing derived classes to implement abstract members.

3. The biggest difference: multiple inheritance

This is one of the most important interview questions.

A class can inherit from **only one class**:

```
</> C#

class Dog : Animal
{
}
```

You cannot do:

```
</> C#

class Dog : Animal, Vehicle
{
}
```

✗ A class cannot have two base classes.

But a class can implement **multiple interfaces**:

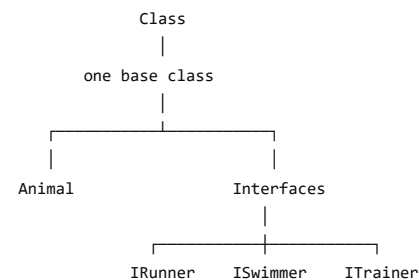
```
</> C#

class Dog : Animal, IRunner, ISwimmer, ITrainer
{
}
```

✓ Completely valid.

So:

</> Plain text



Interview answer

A class can inherit from only one base class, but it can implement multiple interfaces.

4. State: abstract classes can have instance fields

This is a major difference.

An abstract class can contain state:

```
</> C#

public abstract class Employee
{
    protected decimal salary;
    protected string name;

    public void DisplaySalary()
    {
        Console.WriteLine(salary);
    }
}
```

```
}  
}
```

Derived classes share that inherited state.

Interfaces traditionally don't provide instance state owned by the interface.

You should think:

</> Plain text

Abstract class

Fields

Properties

Constructors

Methods

Abstract methods

Virtual methods

Events

Shared state

versus:

</> Plain text

Interface

Contract

Methods

Properties

Events

Indexers

Modern default implementations

Static members

Modern C# interfaces are more powerful than the old “method declarations only” model, but they still aren't a replacement for an object-oriented base class with inherited instance state.

5. Constructors

Abstract classes can have constructors.

```
</> C#
```

```
public abstract class Animal  
{  
    protected string name;  
  
    protected Animal(string name)  
    {  
        this.name = name;  
    }  
}
```

Then:

```
</> C#  
  
public class Dog : Animal  
{  
    public Dog(string name) : base(name)  
    {  
    }  
}
```

The abstract class constructor executes when a derived object is created.

Interfaces don't have normal instance constructors.

You cannot do:

```
</> C#  
  
IAnimal animal = new IAnimal(...);
```

because an interface isn't an object that can be instantiated.

6. Can you instantiate them?

Neither can be instantiated directly.

Abstract class

```
</> C#  
  
Animal animal = new Animal();
```



Because `Animal` is abstract.

Interface

```
</> C#  
  
IAAnimal animal = new IAAnimal();
```



Because interfaces cannot be instantiated.

But both can be used as **reference types**:

```
</> C#  
  
Animal animal = new Dog();
```

and:

```
</> C#  
  
IAAnimal animal = new Dog();
```

This is where polymorphism comes in.

7. Abstract methods vs interface methods

Abstract class

```
</> C#  
  
public abstract class Animal  
{  
    public abstract void Speak();  
}
```

The derived class must provide the implementation:

```
</> C#
```

```
public class Dog : Animal  
{  
    public override void Speak()  
    {  
        Console.WriteLine("Woof");  
    }  
}
```

Notice:

```
</> Plain text  
  
abstract  
↓  
override
```

Interface

```
</> C#  
  
public interface IAAnimal  
{  
    void Speak();  
}
```

The implementing class provides the implementation:

```
</> C#  
  
public class Dog : IAAnimal  
{  
    public void Speak()  
    {  
        Console.WriteLine("Woof");  
    }  
}
```

Notice:

```
</> Plain text  
  
interface contract  
↓
```

implementation

8. Abstract classes can mix abstract and concrete behavior

This is one of their strongest advantages.

</> C#

```
public abstract class Animal
{
    public abstract void Speak();

    public void Eat()
    {
        Console.WriteLine("Eating");
    }

    public virtual void Sleep()
    {
        Console.WriteLine("Sleeping");
    }
}
```

You can have:

</> Plain text

```
Animal
|
|— abstract Speak()
|   ↓ must implement
|
|— concrete Eat()
|   ↓ already implemented
|
|— virtual Sleep()
|   ↓ can override
```

A derived class can therefore:

- implement abstract members
- inherit concrete members
- override virtual members
- add new members

This makes abstract classes excellent for **shared implementation**.

9. Interfaces can also have default implementations

This is where modern C# becomes more interesting.

An interface can provide a default implementation:

</> C#

```
public interface ILogger
{
    void Log()
    {
        Console.WriteLine("Default logging");
    }
}
```

A class can simply implement the interface without providing `Log()` :

</> C#

```
public class FileLogger : ILogger
{
}
```

The interface default implementation can be used.

So the old interview statement:

“Interfaces cannot contain implementation.”

is **not correct for modern C#**.

A better statement is:

Interfaces can provide default implementations, but abstract classes are designed to represent a shared base with inherited state and behavior.

10. Default interface implementation vs abstract implementation

Compare:

Abstract class

```
</> C#  
  
abstract class Animal  
{  
    public virtual void Eat()  
    {  
        Console.WriteLine("Animal eating");  
    }  
}
```

The implementation becomes part of the **class inheritance hierarchy**.

Interface

```
</> C#  
  
interface IAnimal  
{  
    void Eat()  
    {  
        Console.WriteLine("Default eating");  
    }  
}
```

The implementation belongs to the **interface contract**.

This distinction becomes particularly important when multiple interfaces are involved.

11. Interface supports multiple contracts

Imagine:

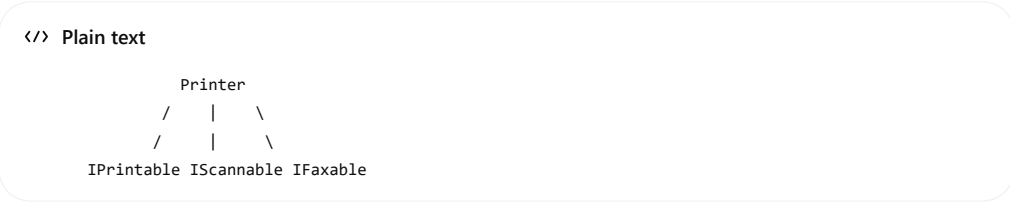
```
</> C#  
  
interface IPrintable  
{  
    void Print();  
}  
  
interface IScannable  
{  
    void Scan();  
}
```

```
interface IFaxable  
{  
    void Fax();  
}
```

A printer can implement all three:

```
</> C#  
  
class Printer : IPrintable, IScannable, IFaxable  
{  
    public void Print() { }  
    public void Scan() { }  
    public void Fax() { }  
}
```

Conceptually:



This is one of the main reasons interfaces are so useful.

12. Abstract class + multiple interfaces

You can combine both:

```
</> C#  
  
abstract class Animal  
{  
    public abstract void Speak();  
}  
  
interface IRunner  
{  
    void Run();  
}  
  
interface ISwimmer  
{  
    void Swim();  
}
```

```

}

class Dog : Animal, IRunner, ISwimmer
{
    public override void Speak() { }
    public void Run() { }
    public void Swim() { }
}

```

This is extremely common in real applications.

The class gets:

</> Plain text

Animal
↓
shared identity/base behavior

IRunner
↓
capability

ISwimmer
↓
capability

13. Abstract class can implement an interface

Very important interview scenario.

</> C#

```

interface IAnimal
{
    void Speak();
}

```

Then:

</> C#

```

abstract class Animal : IAnimal
{
    public abstract void Speak();
}

```

This is valid.

Animal says:

"I promise that every concrete Animal will have Speak, but I don't implement it yet."

Then:

</> C#

```

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Woof");
    }
}

```

Now Dog fulfills the interface contract.

14. Abstract class can partially implement an interface

For example:

</> C#

```

interface IEmployee
{
    void Work();
    void TakeBreak();
}

```

Abstract class:

</> C#

```

abstract class Employee : IEmployee
{
    public void TakeBreak()
    {
        Console.WriteLine("Taking break");
    }

    public abstract void Work();
}

```

So:

</> Plain text

```

IEmployee
├─ Work()
└─ TakeBreak()

Employee
├─ Work()      → abstract/deferred
└─ TakeBreak() → implemented

Developer
├─ Work()      → implemented

```

This is a very useful pattern.

15. Explicit interface implementation

Interfaces become especially powerful when two interfaces have the same method.

</> C#

```

interface IPrinter
{
    void Start();
}

interface IEngine
{
    void Start();
}

```

You can provide different implementations:

</> C#

```

class Machine : IPrinter, IEngine
{
    void IPrinter.Start()
    {
        Console.WriteLine("Printer starting");
    }

    void IEngine.Start()
    {

```

```

        Console.WriteLine("Engine starting");
    }
}

```

Now:

</> C#

```

IPrinter p = new Machine();
p.Start();

```

prints:

</> Plain text

Printer starting

while:

</> C#

```

IEngine e = new Machine();
e.Start();

```

prints:

</> Plain text

Engine starting

This is called:

Explicit interface implementation.

16. Why abstract classes don't have this exact mechanism

An object has only one base-class inheritance chain.

You can't have:

</> Plain text

because C# doesn't support multiple class inheritance.

But interfaces allow:

</> Plain text

Machine

└─ IPrinter

└─ IEngine

and explicit implementations can distinguish their contracts.

This is one of the strongest practical reasons for interfaces.

17. Access modifiers

Abstract class members can use normal class access modifiers:

</> C#

public
protected
private
internal
protected internal
private protected

For example:

</> C#

abstract class Animal
{
 protected string name;

 protected abstract void Speak();
}

Interfaces have different accessibility rules.

For interviews, remember:

Abstract classes participate directly in class inheritance and access control; interfaces primarily define externally consumable contracts.

18. virtual , override , and abstract classes

Abstract classes work naturally with virtual dispatch.

</> C#

abstract class Animal
{
 public virtual void Speak()
 {
 Console.WriteLine("Animal");
 }
}

class Dog : Animal
{
 public override void Speak()
 {
 Console.WriteLine("Dog");
 }
}

Then:

</> C#

Animal animal = new Dog();
animal.Speak();

Output:

</> Plain text

Dog

Because:

</> Plain text

Reference type: Animal
Runtime type: Dog

virtual dispatch
↓
Dog.Speak()

19. Abstract override — advanced interview case

You can even re-abstract an inherited virtual member:

</> C#

abstract class A
{
 public virtual void M()
 {
 Console.WriteLine("A");
 }
}

abstract class B : A
{
 public abstract override void M();
}

class C : B
{
 public override void M()
 {
 Console.WriteLine("C");
 }
}

This is valid.

Think:

</> Plain text

A
└─ virtual M()
 ↓
B
└─ abstract override M()

↓

C
└─ concrete override M()

B is essentially saying:

“Although A provided an implementation, subclasses of B are required to provide their own implementation.”

This is an advanced but important distinction.

20. Interface vs abstract class: identity vs capability

This is perhaps the best conceptual rule.

Abstract class

Use when you have an IS-A relationship:

</> Plain text

Dog IS AN Animal
Car IS A Vehicle
Manager IS AN Employee

Interface

Use when you have a CAN-DO capability/contract:

</> Plain text

Dog CAN Run
Dog CAN Swim
Employee CAN Authenticate
Document CAN BePrinted
Payment CAN BeRefunded

So:

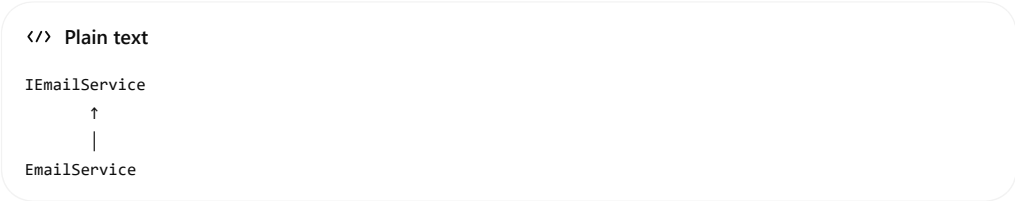
</> Plain text

Dog
|
└─ Animal ← IS-A
└─ IRunner ← CAN-DO

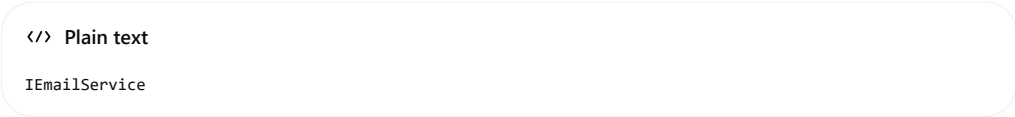
21. Dependency Injection strongly favors interfaces

You'll see this constantly in .NET.

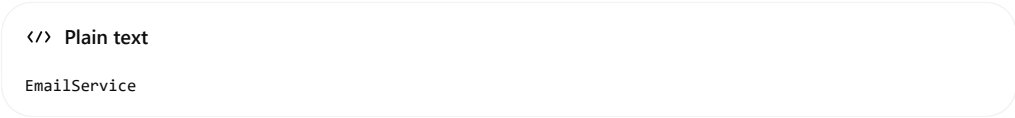
For example:



Your controller depends on:



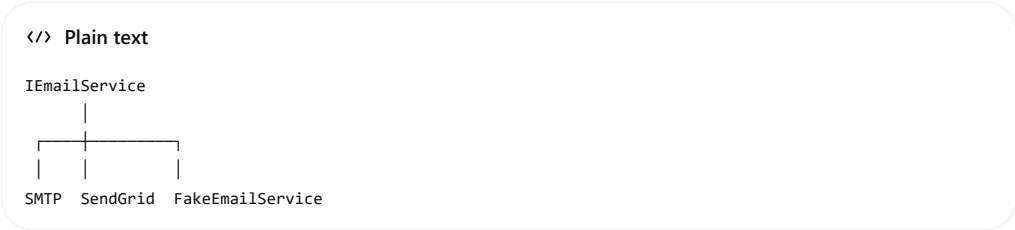
rather than:



Why?

Because the controller depends on the **contract**, not the concrete implementation.

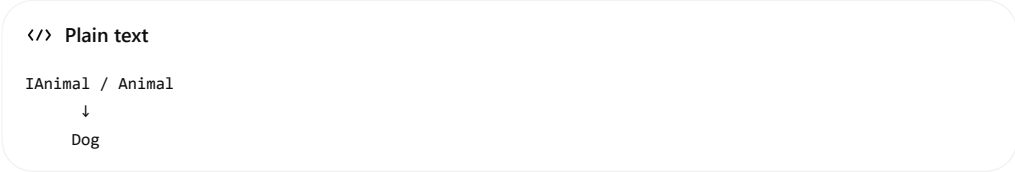
You can then have:



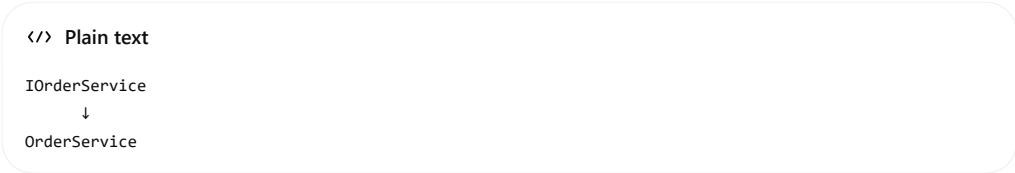
This makes testing and replacing implementations easier.

22. Abstract class and Dependency Injection

You can technically register an abstract base type:



But the more common service architecture is:

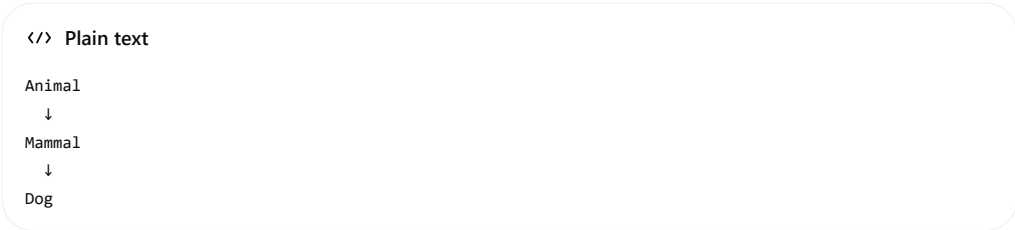


because the interface describes the service contract while the concrete class provides implementation.

23. Interface inheritance vs class inheritance

Both can have inheritance relationships.

Abstract/class inheritance



Single class inheritance chain.

Interface inheritance

```
</> Plain text
IAAnimal
/  \
IRunnable ISwimmable
\  /
IDog
```

An interface can inherit from multiple interfaces.

This is called an **interface diamond**.

For example:

```
</> C#
interface IA
{
    void M();
}

interface IB : IA
{
}

interface IC : IA
{
}

interface ID : IB, IC
{
}
```

ID ultimately inherits IA through both paths.

C# allows this.

24. Interface diamond vs class diamond

C# does **not** allow:

```
</> Plain text
A
/  \
B    C
```

```
\ /
D
```

when B and C are classes and D tries to inherit both.

That's the classic multiple-class-inheritance problem.

But it allows:

```
</> Plain text
IA
/  \
IB  IC
\  /
ID
```

because interfaces are contracts and C# has rules for resolving inherited interface members.

This is an important interview distinction.

25. Abstract class has inherited implementation

Suppose:

```
</> C#
abstract class Animal
{
    public void Eat()
    {
        Console.WriteLine("Eating");
    }
}
```

Then:

```
</> C#
class Dog : Animal
{
}
```

Dog automatically has:

</> Plain text

```
Dog
└─ Eat()
```

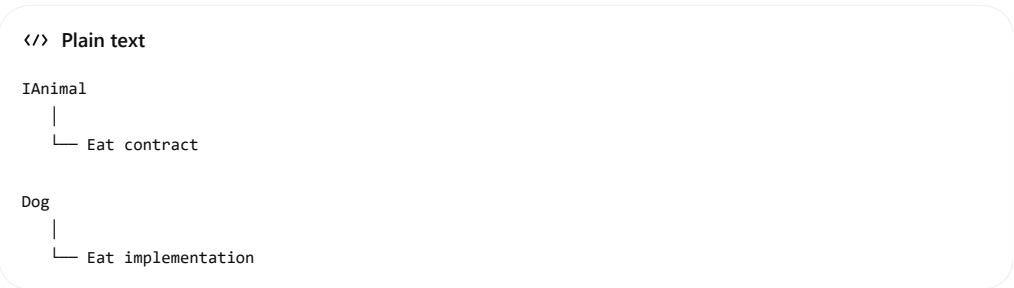
The implementation is inherited.

With an interface:

</> C#

```
interface IAnimal
{
    void Eat();
}
```

the normal model is:



With modern C#, a default interface implementation can provide fallback behavior, but that is conceptually different from inheriting instance state and base-class implementation.

26. Abstract class can enforce constructor initialization

This is another important advantage.

</> C#

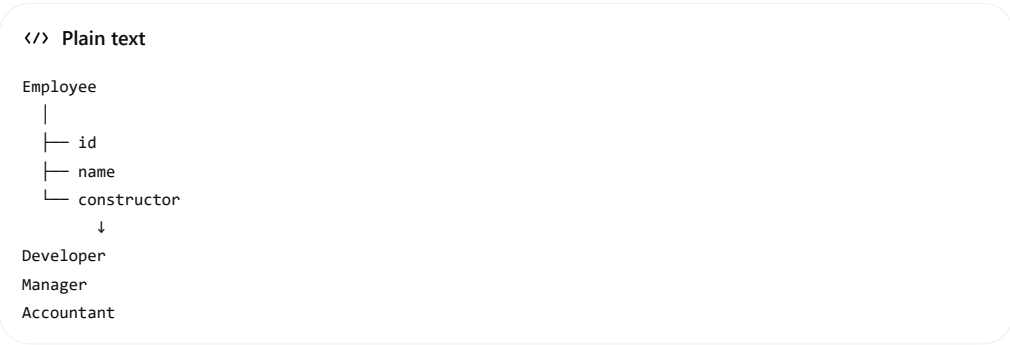
```
abstract class Employee
{
    protected int id;
    protected string name;

    protected Employee(int id, string name)
    {
        this.id = id;
    }
}
```

```
        this.name = name;
    }
}
```

Every derived class must deal with that base initialization.

For example:



This is not what interfaces are designed to do.

27. Which one gives you shared state?

Feature	Interface	Abstract class
Instance fields	✗	✓
Constructors	✗ normal instance constructors	✓
Shared instance state	✗	✓
Abstract methods	Contract members	✓
Concrete methods	✓ default implementations in modern C#	✓
Virtual methods	Not in the class-inheritance sense	✓
Override	No override for normal interface implementation	✓
Multiple inheritance	✓ multiple interfaces	✗ one base class

Feature	Interface	Abstract class
Interface inheritance	✓	N/A
Explicit implementation	✓	✗
Access to protected state	✗ as a normal interface mechanism	✓
Can instantiate directly	✗	✗
Used polymorphically	✓	✓

28. The most important practical difference

Imagine you're designing a hospital system.

You could have:

</> Plain text

```
abstract class Person
```

because:

</> Plain text

```
Doctor IS A Person
Patient IS A Person
Nurse IS A Person
```

Person might contain:

</> Plain text

```
Id
Name
DateOfBirth
Address
```

and shared behavior:

</> Plain text

GetFullName()

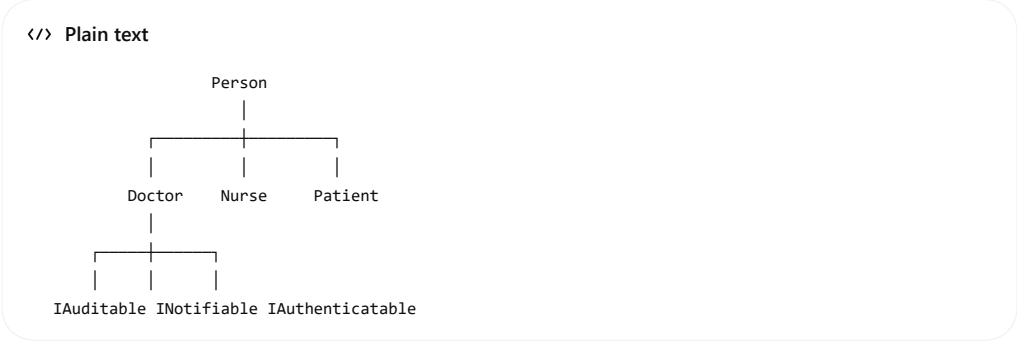
Then interfaces:

</> Plain text

```
IAuditable
INotifiable
IAuthenticatable
```

because different types can have those capabilities.

So:



This is a realistic way to combine both.

29. When should you choose an abstract class?

Use an abstract class when:

1. There is a strong base-class relationship

</> Plain text

```
Dog IS Animal
```

2. You need shared state

</> C#

```
protected int id;
protected string name;
```

3. You need constructors

```
</> C#

protected Animal(string name)
```

4. You need shared implementation

```
</> C#

public void Eat()
```

5. You need protected members

```
</> C#

protected void Validate()
```

6. You need a controlled inheritance hierarchy

```
</> Plain text
```

```
Animal
  ↓
Mammal
  ↓
Dog
```

30. When should you choose an interface?

Use an interface when:

1. You want a contract

```
</> Plain text

IPaymentService
```

2. Multiple unrelated classes need the same capability

```
</> Plain text
```

```
Dog : IRunner
Car : IRunner
Robot : IRunner
```

They don't need to share a common base class.

3. You need multiple contracts

```
</> C#
```

```
class OrderService :
    IOrderService,
    IValidator,
    ICacheable
```

4. You want loose coupling

```
</> Plain text
```

```
Controller
  ↓
IOrderService
  ↓
OrderService
```

5. You want easy replacement/mocking for testing

```
</> Plain text
```

```
IService
  |
  +--- RealEmail
  +--- FakeEmail
```

6. You need explicit implementations

```
</> Plain text
```

```
IPrinter.Print()
IEngine.Print()
```

with different behavior.

31. A common interview trap

Question:

"Should I always use an interface instead of an abstract class?"

No.

And:

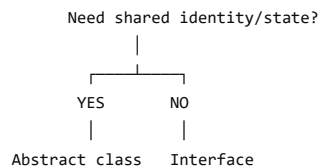
"Should I always use an abstract class because it can contain implementation?"

Also no.

They solve different design problems.

Think:

</> Plain text



But in real design, you can use **both together**.

32. Interface + abstract class together

This is a very common architecture:

</> C#

```

interface IRepository
{
    void Add();
    void Delete();
}

abstract class BaseRepository : IRepository
  
```

```

{
    public void Delete()
    {
        // shared implementation
    }

    public abstract void Add();
}

class CustomerRepository : BaseRepository
{
    public override void Add()
    {
  
```

Now:

</> Plain text

```

IRepository
  ↓
BaseRepository
  ↓
CustomerRepository
  
```

Each layer has a different responsibility.

33. The mental model I recommend for interviews

Don't memorize a huge table.

Remember these **three questions**:

Question 1 — "Is it an IS-A relationship?"

If yes, an abstract/base class may make sense.

</> Plain text

```
Dog IS-A Animal
```

Question 2 — "Is it a capability/contract?"

If yes, interface is often appropriate.

</> Plain text

Question 3 — "Do I need shared state/implementation?"

If yes, an abstract class is useful.

</> Plain text

Animal
├─ name
├─ id
├─ Eat()
└─ abstract Speak()

34. Interview-ready answer

If an interviewer asks:

"What is the difference between an interface and an abstract class in C#?"

A strong answer would be:

An interface primarily defines a contract or capability that a class agrees to provide, while an abstract class represents a common base type that can provide shared state and implementation. A class can inherit from only one base class but can implement multiple interfaces. Abstract classes can have constructors, fields, protected members, concrete methods, virtual methods, and abstract methods. Modern C# interfaces can also have default implementations, but they are still primarily used for contracts and multiple capabilities. I would generally use an abstract class when related types need shared state or behavior, and an interface when I need a contract, loose coupling, or multiple capabilities.

That's a very solid Junior → Mid-level interview answer.

The shortest memory trick

</> Plain text

ABSTRACT CLASS
"What ARE you?"

+
"What's common between you?"

INTERFACE
"What CAN you do?"

And the most important C# rule:

</> Plain text

1 base class
+
many interfaces

That single rule explains a huge portion of the difference.

interface access modifiers c#

Yes. **C# interface access modifiers** are a little tricky because there are two different things to consider:

- 1. The access modifier of the **interface itself**
- 2. The access modifier of **members inside the interface**

1. Access modifier of the interface itself

An interface can generally be declared with:

</> C#

public interface IAnimal
{
}

or:

</> C#

internal interface IAnimal
{
}

public

Accessible from other assemblies/projects.

</> C#

```
public interface ICustomerService
{
}
```

This is common for contracts that are part of a public API.

internal

Accessible only within the same assembly.

```
</> C#

internal interface ICustomerService
{
}
```

This is the default accessibility for a **top-level interface** when you don't specify an access modifier.

So:

```
</> C#

interface ICustomerService
{
}
```

means essentially:

```
</> C#

internal interface ICustomerService
{
}
```

2. Access modifiers for interface members

This is where interviews get interesting.

Consider:

```
</> C#

public interface IAnimal
{
}
```

```
void Speak();
}
```

You don't write:

```
</> C#

public interface IAnimal
{
    public void Speak();
}
```

The reason is that a normal interface member is **implicitly public**.

So:

```
</> C#

interface IAnimal
{
    void Speak();
}
```

means conceptually:

```
</> C#

interface IAnimal
{
    public void Speak();
}
```

The important idea:

A normal interface member is publicly accessible through the interface.

3. Can interface members be **private** ?

Modern C# — yes, in specific cases.

Interfaces can have private members, but these are **implementation helpers**, not members that implementing classes are required to implement.

For example:

```
</> C#

interface ILogger
{
    void Log()
    {
        WriteLog();
    }

    private void WriteLog()
    {
        Console.WriteLine("Logging...");
    }
}
```

Here:

```
</> Plain text

ILogger
├── Log()      → public interface member
└── WriteLog() → private helper
```

WriteLog() cannot be called through an interface reference:

```
</> C#

ILogger logger = ...;

logger.WriteLog(); // ❌
```

It's only available internally to the interface's own implementation members.

4. Can interface members be protected ?

You should distinguish this carefully from classes.

A normal interface contract does **not** work like an abstract class with protected instance members.

However, modern C# supports more advanced interface member forms, including private implementation helpers. For interview purposes:

Don't think of interfaces as a place for protected state/behavior. If you need protected members for derived classes, an abstract class is the natural choice.

For example:

```
</> C#

abstract class Animal
{
    protected void Validate()
    {
    }
}
```

A derived class can use it:

```
</> C#

class Dog : Animal
{
    void Test()
    {
        Validate(); // ✅
    }
}
```

This is fundamentally different from the normal interface contract model.

5. Can interface members be static ?

Yes, modern C# supports static interface members.

For example:

```
</> C#

interface ICalculator
{
    static abstract int Add(int x, int y);
}
```

A class implementing the interface must provide the required static member:

```
</> C#

class Calculator : ICalculator
{
    public static int Add(int x, int y)
    {
        return x + y;
    }
}
```

This is mainly used in advanced generic/static-polymorphism scenarios.

For junior interviews, know that:

Modern C# interfaces can contain static members, including static abstract members.

6. Can interface members be `virtual` ?

This is a common trap.

You normally **don't write**:

```
</> C#

interface IAnimal
{
    virtual void Speak();
}
```

Instead, an interface member without an implementation is already a contract that an implementing type must fulfill.

Modern C# also supports **default interface implementations**:

```
</> C#

interface IAnimal
{
    void Speak()
    {
        Console.WriteLine("Animal");
    }
}
```

So don't confuse:

```
</> Plain text

abstract class
↓
virtual / abstract / override
```

with:

```
</> Plain text

interface
↓
contract / default implementation
```

7. Explicit interface implementation

This is another important access-related concept.

Suppose:

```
</> C#

interface IPrinter
{
    void Start();
}
```

You can implement it explicitly:

```
</> C#

class Printer : IPrinter
{
    void IPrinter.Start()
    {
        Console.WriteLine("Starting");
    }
}
```

Notice there is **no `public`** .

That's because the implementation is specifically tied to the interface.

Therefore:

```
</> C#

Printer p = new Printer();

p.Start(); // ❌
```

But:

```
</> C#

IPrinter p = new Printer();

p.Start(); // ✅
```

Think:

```
</> Plain text

Printer reference
|
└─ Start() ❌

IPrinter reference
|
└─ Start() ✅
```

8. Why isn't explicit implementation public?

Because it intentionally hides the implementation from the class's normal public API.

For example:

```
</> C#

interface IPrinter
{
    void Start();
}

interface IEngine
{
    void Start();
}
```

Then:

```
</> C#

class Machine : IPrinter, IEngine
{
    void IPrinter.Start()
    {
        Console.WriteLine("Printer");
    }

    void IEngine.Start()
    {
        Console.WriteLine("Engine");
    }
}
```

Now:

```
</> C#

Machine m = new Machine();
```

There is no:

```
</> C#

m.Start();
```

But:

```
</> C#

IPrinter p = m;
p.Start();
```

and:

```
</> C#

IEngine e = m;
e.Start();
```

can access the appropriate implementations.

9. Quick access-modifier table

Interface thing	Allowed?	Important point
public interface ITest	✔	Public interface
internal interface ITest	✔	Assembly-only
No modifier on top-level interface	✔	Defaults to internal
Normal void M()	✔	Implicitly public contract
public void M()	✔ modern C#	Explicitly public
private void Helper()	✔ modern C#	Interface-internal helper
protected normal interface member	✘	Not a normal interface contract mechanism
static member	✔	Modern C#
static abstract member	✔	Advanced generic/static polymorphism
Default implementation	✔	Modern C#
Explicit implementation	✔	Implemented through interface view

10. The important difference from abstract classes

Compare:

Abstract class

```
</> C#  
  
abstract class Animal  
{  
    private int id;  
  
    protected string name;
```

```
public void Eat()  
{  
}  
  
protected abstract void Speak();  
}
```

You have lots of control over accessibility:

</> Plain text

```
private  
protected  
public  
internal  
...
```

because it's a class.

Interface

</> C#

```
interface IAnimal  
{  
    void Speak();  
}
```

The normal member is part of the **public contract**.

You don't use interfaces to model:

</> Plain text

```
private state  
protected state  
shared instance fields
```

That's one of the reasons you choose an abstract class instead.

Interview question you should remember

Q: Are interface methods public by default?

Yes. A normal interface member is implicitly public as part of the interface contract. Modern C# also allows certain private implementation members and default implementations, but those are different from the public contract members.

Q: Can an interface have protected fields?

No. Interfaces don't provide normal instance fields or protected instance state like an abstract class. If I need shared state or protected implementation for derived classes, I'd use an abstract class.

Q: Why doesn't an explicit interface implementation use public ?

Because it's intentionally exposed through the interface contract rather than through the implementing class's normal public API.

1 selection

but Protected interface members are allowed in C# as part of the Default Interface Methods feature.

Yes — you're right, and my previous statement was too broad/outdated.

In modern C#, interfaces can have protected members, as part of the advanced interface features introduced around default interface methods.

The important distinction is:

1. Protected interface member

A protected interface member is not part of the public contract available to every interface consumer. It can be used in interface inheritance scenarios.

For example, conceptually:

```
</> C#  
  
interface IA  
{  
    protected void Helper()  
    {  
        Console.WriteLine("Helper");  
    }  
}  
  
interface IB : IA  
{
```

```
void Test()  
{  
    Helper(); // allowed  
}  
}
```

Here:

- IA.Helper() is protected.
- IA itself doesn't expose Helper() to ordinary callers.
- A derived interface such as IB can access it.
- This is different from a normal interface method like void M(); , which is part of the public interface contract.

2. What about protected fields?

This is the important correction:

Protected interface members are allowed, but interface instance fields are still not allowed.

So:

```
</> C#  
  
interface IA  
{  
    protected void Helper(); // possible in the appropriate interface-member context  
}
```

but:

```
</> C#  
  
interface IA  
{  
    protected int x; // ❌ interface instance field  
}
```

An interface doesn't have per-object instance storage like a class.

Interview distinction

Member	Interface
Public contract method	✓

Member	Interface
Default implementation	✓
Private method	✓
Protected method/member	✓ modern C#
Static members	✓
Static abstract members	✓
Instance fields	✗
Instance constructors	✗
Protected instance field	✗

So the precise answer to “Can an interface have protected fields?” is:

No, interfaces cannot have instance fields. However, modern C# does allow protected interface members, particularly for interface inheritance/default implementations.

That distinction is important.

explain what is virtual and dispatch behind the scenes

Absolutely. This is one of the most important concepts behind C# polymorphism.

1. What does virtual mean?

When you write:

```
</> C#  
  
class Animal  
{  
    public virtual void Speak()  
    {  
        Console.WriteLine("Animal");  
    }  
}
```

virtual means:

"This method can be replaced (overridden) by a derived class, and calls to it can be dynamically dispatched at runtime."

Then:

```
</> C#  
  
class Dog : Animal  
{  
    public override void Speak()  
    {  
        Console.WriteLine("Dog");  
    }  
}
```

Now:

```
</> C#  
  
Animal x = new Dog();  
x.Speak();
```

prints:

```
</> Plain text  
  
Dog
```

Even though the reference type is Animal .

2. Why does it print Dog?

There are two different types involved:

```
</> Plain text  
  
Animal x = new Dog();  
^^^^^^      ^^^^^  
reference    actual object  
type         runtime type
```

Think of it as:

</> Plain text

```
x
|
▼
```

Dog object

Speak → Dog()

The compiler sees:

</> C#

```
x.Speak();
```

and knows:

Animal has a virtual Speak() .

But it doesn't simply say:

"Call Animal.Speak."

Instead, because the method is **virtual**, runtime dispatch is involved.

3. What is dispatch?

Dispatch means:

Determining which implementation of a method should actually execute.

There are two major ideas you should know:

Static dispatch

The method is determined based on the compile-time type.

Example:

</> C#

```
class Animal
{
    public void Speak()
    {
```

```
        Console.WriteLine("Animal");
    }
}

class Dog : Animal
{
    public new void Speak()
    {
        Console.WriteLine("Dog");
    }
}
```

Then:

</> C#

```
Animal x = new Dog();
x.Speak();
```

Output:

</> Plain text

```
Animal
```

Because Speak() isn't virtual.

The compiler effectively resolves:

</> Plain text

```
Animal reference
    ↓
Animal.Speak
```

Dynamic dispatch

With virtual :

</> C#

```
class Animal
{
    public virtual void Speak()
    {
```

```
        Console.WriteLine("Animal");
    }
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog");
    }
}
```

Then:

```
</> C#

Animal x = new Dog();
x.Speak();
```

Runtime considers the actual object:

</> Plain text

reference type: Animal
runtime type: Dog

x

|

▼

Dog object

|

▼

virtual dispatch

|

▼

Dog.Speak()

So it prints:

```
</> Plain text

Dog
```

4. What happens behind the scenes?

This is where things become interesting.

Very simplified, you can imagine a class with virtual methods having a **virtual method table (vtable)**.

For example:

```
</> C#

class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal");
    }
}
```

Conceptually:

</> Plain text

Animal vtable

| Speak → Animal |

Then:

```
</> C#

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog");
    }
}
```

Conceptually Dog's virtual method table becomes:

</> Plain text

Dog vtable

So when:

</> C#

```
Animal x = new Dog();
x.Speak();
```

the runtime sees:



Therefore:

</> Plain text

```
Dog
```

Important

This is a **conceptual model** of runtime dispatch. Don't think of the C# language specification as saying "there must literally be a vtable like this." The CLR/JIT is free to implement dispatch differently and optimize it.

5. Why does new behave differently?

Consider:

</> C#

Now:

</> C#

```
A x = new B();
x.M();
```

prints:

</> Plain text

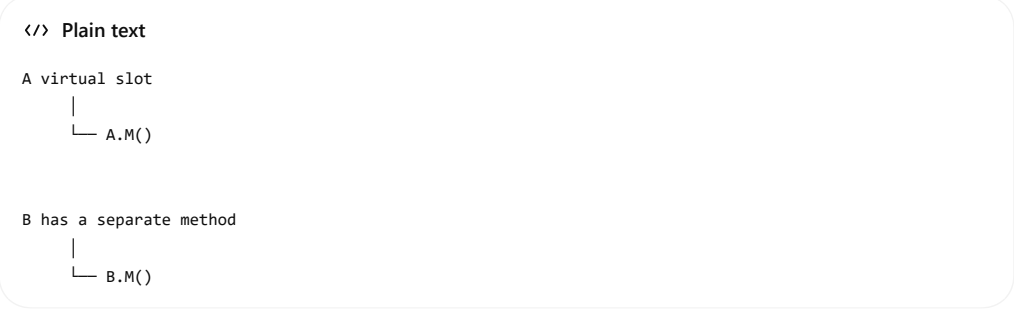
```
A
```

Why?

Because B.M() did **not override** the virtual method.

It created a **different member that hides the inherited one**.

Conceptually:



Therefore:

</> Plain text

```
A x = new B();

x.M()
|
└─ A's virtual slot
    |
    └─ A.M()
```

6. override continues the virtual chain

This is extremely important.

</> C#

```
class A
{
    public virtual void M()
    {
        Console.WriteLine("A");
    }
}

class B : A
{
    public override void M()
    {
        Console.WriteLine("B");
    }
}

class C : B
{
    public override void M()
    {
```

Now:

</> C#

```
A x = new C();
x.M();
```

Output:

</> Plain text

C

Conceptually:

</> Plain text

```
A.M
|
| virtual slot
▼
B.M
|
| override same slot
▼
C.M
|
▼
C.M executes
```

The important idea is:

override continues the same virtual dispatch chain.

7. new virtual starts a new chain

This is the tricky interview case:

</> C#

```
class A
{
    public virtual void M()
    {
        Console.WriteLine("A");
    }
}

class B : A
{
    public new virtual void M()
    {
        Console.WriteLine("B");
    }
}
```

```
}  
}  
  
class C : B  
{  
    public override void M()  
{  
    }  
}
```

There are now effectively **two virtual chains**:



Therefore:

```
</> C#  
  
A x = new C();  
x.M();
```

prints:

```
</> Plain text  
  
A
```

But:

```
</> C#  
  
B x = new C();  
x.M();
```

prints:

```
</> Plain text  
  
C
```

Because the reference determines **which virtual slot/chain you're calling into**.

8. The mental model you should memorize

When you see:

```
</> C#  
  
Base x = new Derived();  
x.M();
```

ask:

Step 1

Does `Base` have `M` ?

Step 2

Is `M` virtual?

- **No** → normal/static dispatch → base implementation.
- **Yes** → runtime dispatch.

Step 3

Does `Derived` override that virtual method?

- **Yes** → derived override executes.
- **No** → inherited implementation executes.

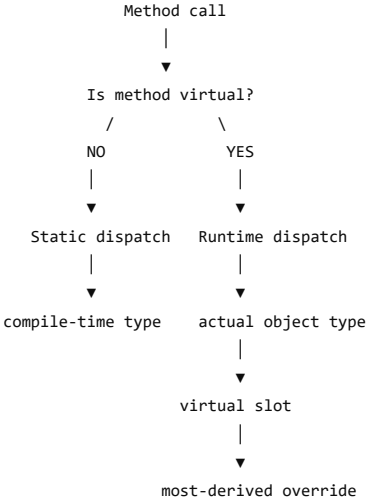
Step 4

Did the derived class use `new` instead of `override` ?

If yes, you have a **new/hiding member**, not an override of the original virtual slot.

9. The big picture

</> Plain text



Interview answer

If the interviewer asks:

"What does virtual do in C#?"

A strong answer is:

" virtual allows a derived class to override a method. When the method is called through a base-class reference, the CLR can use runtime/dynamic dispatch to execute the most-derived override corresponding to the actual object."

And if they ask:

"What happens behind the scenes?"

Say:

"Conceptually, virtual methods are associated with virtual dispatch slots, often modeled using a vtable. The runtime uses the object's runtime type to resolve the appropriate override. The exact implementation is an internal CLR/JIT detail and may be optimized."

That is a much better interview answer than simply saying "virtual allows overriding."

Today 5:00 PM

explain String Interpolation

- Raw String Literals

- Top-level Statements
- Global Usings
- File-scoped Namespaces

These are all **modern C# features** that you'll commonly see in newer .NET projects. Let's take them one by one with interview-focused examples.

1. String Interpolation

String interpolation lets you insert expressions directly inside a string using `$`.

Without interpolation

```
</> C#

string name = "Khaled";
int age = 26;

string message = "My name is " + name + " and I am " + age + " years old.";
```

With interpolation

```
</> C#

string name = "Khaled";
int age = 26;

string message = $"My name is {name} and I am {age} years old.";
```

Much cleaner.

Expressions are allowed

```
</> C#

int x = 10;
int y = 20;

Console.WriteLine($"Sum = {x + y}");
```

Output:

</> Plain text

You can also call methods:

```
</> C#

Console.WriteLine($"Name: {name.ToUpper()}");
```

Formatting

```
</> C#

double price = 1234.5678;

Console.WriteLine($"Price: {price:F2}");
```

Output:

```
</> Plain text

Price: 1234.57
```

Common formats:

```
</> Plain text

:F2      2 decimal places
:C       currency
:P       percentage
:N2      number with 2 decimals
```

Interview answer

String interpolation allows expressions to be embedded directly inside strings using `$"..."` and `{}` .

2. Raw String Literals

Raw string literals were introduced in **C# 11**.

They make it much easier to write strings containing:

Instead of escaping everything.

Normal string

Imagine JSON:

```
</> C#

string json = "{ \"name\": \"Khaled\", \"age\": 26 }";
```

Lots of escaping.

With a raw string:

```
</> C#

string json = """
{
    "name": "Khaled",
    "age": 26
}
""";
```

Much cleaner.

The delimiter is **three double quotes**:

```
</> Plain text

"""
content
"""
```

Multiline text

```
</> C#

string message = """
Hello Khaled,
```

```
Welcome to the .NET course.
```

```
Good luck!  
"";
```

The newlines are preserved.

Why are they called "raw"?

Because characters such as:

```
</> Plain text
```

```
"  
\  
{  
}
```

generally don't need the normal escaping you'd need in an ordinary string.

For example:

```
</> C#
```

```
string path = ""  
C:\Users\Khaled\Documents  
"";
```

No need for:

```
</> C#
```

```
"C:\\Users\\Khaled\\Documents"
```

Raw interpolation

You can combine raw strings with interpolation:

```
</> C#
```

```
string name = "Khaled";
```

```
string json = $$$"  
{
```

```
"name": "{name}"  
}  
"";
```

The number of \$ signs controls how many { characters are required to start interpolation.

This becomes useful when the string itself contains lots of {} such as JSON.

Interview answer

Raw string literals, introduced in C# 11, allow multiline strings and content containing quotes or special characters without extensive escaping.

3. Top-Level Statements

Normally a C# program used to look like:

```
</> C#
```

```
class Program  
{  
    static void Main(string[] args)  
    {  
        Console.WriteLine("Hello");  
    }  
}
```

With **top-level statements**, you can simply write:

```
</> C#
```

```
Console.WriteLine("Hello");
```

That's it.

The compiler generates the necessary Program type and entry point for you.

Example

Instead of:

```
</> C#
```

```
using System;

class Program
{
    static void Main(string[] args)
    {
        int x = 10;
        Console.WriteLine(x);
    }
}
```

You can write:

```
</> C#

using System;

int x = 10;
Console.WriteLine(x);
```

Where have you seen this?

Probably in your ASP.NET Core projects.

For example:

```
</> C#

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();

app.Run();
```

That's a **top-level statement** style.

There is no:

```
</> C#

class Program
{
```

```
static void Main()
{
    ...
}
```

because the compiler generates the entry point for you.

Important limitation

A project can have **only one file containing top-level statements**.

Typically:

```
</> Plain text

Program.cs
```

is that file.

Interview answer

Top-level statements allow us to write executable code directly in a C# file without explicitly declaring `Program` and `Main`. The compiler generates the entry point behind the scenes.

4. Global Usings

Normally, if you need a namespace, you write:

```
</> C#

using System;
using System.Collections.Generic;
using System.Linq;
```

in every file that needs them.

With **global using**, you can declare a namespace once:

```
</> C#

global using System;
global using System.Collections.Generic;
```

```
global using System.Linq;
```

Then all `.cs` files in that project can use those namespaces without repeating the `using` .

Example

Create:

</> Plain text

```
GlobalUsings.cs
```

and put:

</> C#

```
global using System;
global using System.Collections.Generic;
global using System.Linq;
```

Then another file can simply do:

</> C#

```
List<int> numbers = new();

var result = numbers.Where(x => x > 5);
```

without:

</> C#

```
using System.Collections.Generic;
using System.Linq;
```

ASP.NET Core

You may have seen:

</> C#

```
global using Microsoft.AspNetCore.Mvc;
global using Microsoft.EntityFrameworkCore;
```

Then controllers and other files don't need to repeat those imports.

Implicit global usings

Modern .NET projects can also enable:

</> XML

```
<ImplicitUsings>enable</ImplicitUsings>
```

The SDK automatically provides common global usings appropriate for the project type.

That's why a new ASP.NET Core project may have very few explicit `using` statements.

Interview answer

Global usings allow a namespace to be imported once for the entire project instead of repeating the `using` directive in every file.

5. File-scoped Namespaces

Traditional namespace syntax:

</> C#

```
namespace MyApp.Services
{
    public class CustomerService
    {
    }
}
```

Notice the extra `{ }` .

With a file-scoped namespace:

</> C#

```
namespace MyApp.Services;
```

```
public class CustomerService
{
}
```

The semicolon means:

Everything in this file belongs to this namespace.

Why is this useful?

It removes one level of indentation.

Instead of:

```
</> C#

namespace MyApp.Services
{
    public class CustomerService
    {
        public void Add()
        {
        }
    }
}
```

You get:

```
</> C#

namespace MyApp.Services;

public class CustomerService
{
    public void Add()
    {
    }
}
```

Cleaner, especially in projects with many classes.

6. How these appear together

A modern .NET project might look like:

GlobalUsings.cs

```
</> C#

global using Microsoft.EntityFrameworkCore;
global using System.Linq;
```

CustomerService.cs

```
</> C#

namespace MyApp.Services;

public class CustomerService
{
    public string GetMessage(string name)
    {
        return $"Hello, {name}";
    }
}
```

Program.cs

```
</> C#

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();

app.Run();
```

So you're seeing several modern C# features together:

```
</> Plain text

Global Usings
↓
Less repeated using statements

File-scoped Namespace
↓
Less namespace boilerplate
```

10/3/26, 7:48 PM

C# Recap

Top-level Statements

↓

Less Program/Main boilerplate

String Interpolation

↓

Cleaner string construction

Raw String Literals

↓

Cleaner multiline/JSON/etc. strings

Interview cheat sheet

Feature	Introduced	Main purpose
String interpolation	C# 6	Embed expressions in strings
File-scoped namespaces	C# 10	Reduce namespace boilerplate
Global usings	C# 10	Share using directives across project
Top-level statements	C# 9	Remove explicit Program/Main boilerplate
Raw string literals	C# 11	Write complex/multiline strings with minimal escaping

One important distinction: these features mostly improve **syntax/readability**; they don't fundamentally change what your application is capable of doing.

Target-typed new Discards nameof Ranges & Indices (^ , ..)

- Tuples
- Lazy<T>
- Generic Math / Static abstract interface members
- Extension members

These are a mix of **modern C# syntax/features** and one major advanced topic. Let's go through them in an interview-friendly way.

1. Target-typed new

Normally:

10/3/26, 7:48 PM

C# Recap

</> C#

List<int> numbers = new List<int>();

The type is repeated.

With **target-typed new** :

</> C#

List<int> numbers = new();

The compiler knows that the target type is List<int> , so it can infer what new() means.

Another example

</> C#

Customer customer = new();

is equivalent to:

</> C#

Customer customer = new Customer();

With constructor arguments

</> C#

Customer customer = new("Khaled", 26);

The compiler knows the constructor must belong to Customer .

Important limitation

There must be a target type.

This doesn't work:

</> C#

var customer = new(); // ❌

Why?

Because `var` itself needs the compiler to determine the type, while `new()` needs an already-known target type.

Interview answer

Target-typed `new` allows the compiler to infer the type of an object creation expression from the variable or parameter's target type.

2. Discards `_`

A **discard** means:

"I don't care about this value."

The `_` symbol is used when you intentionally don't need a value.

Example: Deconstruction

```
</> C#
var (name, age) = GetPerson();
```

Suppose you only need the name:

```
</> C#
var (name, _) = GetPerson();
```

The second returned value is intentionally ignored.

out parameters

Suppose:

```
</> C#
int.TryParse("123", out int number);
```

You need the result.

But maybe you only care whether parsing succeeded:

```
</> C#
bool success = int.TryParse("123", out _);
```

You're saying:

Give me the success/failure result, but I don't need the parsed number.

Lambda parameters

Modern C# also supports discards in lambdas when parameter names aren't needed:

```
</> C#
(_, _) => Console.WriteLine("Hello")
```

Important

`_` isn't always a discard.

For example:

```
</> C#
var _ = 10;
```

Here `_` is actually a variable.

But in contexts such as:

```
</> C#
out _
```

it is a discard.

Interview answer

A discard is `_` , used when a value is intentionally ignored.

3. nameof

`nameof` gives you the **name of a variable, property, method, class, etc. as a string**, while being checked by the compiler.

Example:

```
</> C#  
  
string name = "Khaled";  
  
Console.WriteLine(nameof(name));
```

Output:

```
</> Plain text  
  
name
```

Notice:

```
</> C#  
  
nameof(name)
```

returns:

```
</> Plain text  
  
"name"
```

not:

```
</> Plain text  
  
"Khaled"
```

Why is it useful?

Especially for exceptions and validation.

Instead of:

```
</> C#  
  
throw new ArgumentNullException("name");
```

you can write:

```
</> C#  
  
throw new ArgumentNullException(nameof(name));
```

If you later rename:

```
</> C#  
  
name
```

to:

```
</> C#  
  
customerName
```

your IDE/compiler tooling can update the `nameof` reference.

Properties

```
</> C#  
  
public class Customer  
{  
    public string Name { get; set; }  
  
    public void Print()  
    {  
        Console.WriteLine(nameof(Name));  
    }  
}
```

Output:

</> Plain text

Name

Methods and types

</> C#

nameof(Customer)
nameof(Print)

produce:

</> Plain text

Customer
Print

Interview answer

nameof returns the compile-time name of a symbol as a string. It's commonly used for validation, exceptions, logging, and refactoring-safe parameter/property names.

4. Ranges and Indices

These were introduced in C# 8.

They make working with arrays, strings, and other indexable types much easier.

Index ^

^ means:

Count from the end.

Suppose:

</> C#

int[] numbers = { 10, 20, 30, 40, 50 };

Normal indexing:

</> Plain text

index: 0 1 2 3 4
value: 10 20 30 40 50

From the end:

</> Plain text

^5 ^4 ^3 ^2 ^1

Therefore:

</> C#

numbers[^1]

means:

</> Plain text

last element

Result:

</> Plain text

50

And:

</> C#

numbers[^2]

means:

Result:

</> Plain text

40

Important

^1 = last element.

There is no ^0 valid element index.

Range ..

.. means:

Select a range of elements.

</> C#

int[] numbers = { 10, 20, 30, 40, 50 };

</> C#

var result = numbers[1..4];

gives:

</> Plain text

20 30 40

The end is **exclusive**.

It's conceptually similar to:

</> Plain text

Examples

</> C#

numbers[..3]

First 3 elements:

</> Plain text

10 20 30

</> C#

numbers[2..]

From index 2 to the end:

</> Plain text

30 40 50

</> C#

numbers[^3..]

Last 3 elements:

</> Plain text

30 40 50

</> C#

numbers[..^1]

Everything except the last element:

</> Plain text

10 20 30 40

Interview answer

^ represents an index counted from the end, while .. represents a range with a start and an exclusive end.

5. Tuples

A tuple lets you group multiple values together without creating a separate class.

</> C#

```
var person = ("Khaled", 26);
```

You can access them:

</> C#

```
Console.WriteLine(person.Item1);
Console.WriteLine(person.Item2);
```

Output:

</> Plain text

Khaled
26

Named tuples

Much better:

</> C#

```
var person = (Name: "Khaled", Age: 26);

Console.WriteLine(person.Name);
Console.WriteLine(person.Age);
```

Returning multiple values

Very useful:

</> C#

```
(string Name, int Age) GetPerson()
{
    return ("Khaled", 26);
}
```

Then:

</> C#

```
var person = GetPerson();

Console.WriteLine(person.Name);
```

Tuple deconstruction

You can directly split the tuple:

</> C#

```
var (name, age) = GetPerson();
```

Now:

</> Plain text

name → "Khaled"
age → 26

This connects directly to **discards**:

</> C#

```
var (name, _) = GetPerson();
```

Interview answer

A tuple groups multiple values into one value, often useful for returning multiple values from a method without creating a dedicated type.

6. Lazy<T>

Lazy<T> is used for **lazy initialization**.

Normally:

```
</> C#
```

```
MyExpensiveObject obj = new MyExpensiveObject();
```

The object is created immediately.

With:

```
</> C#
```

```
Lazy<MyExpensiveObject> obj =  
    new Lazy<MyExpensiveObject>();
```

the object isn't created immediately.

It is created when you first access:

```
</> C#
```

```
obj.Value
```

Example

```
</> C#
```

```
Lazy<CustomerService> service =  
    new Lazy<CustomerService>(() =>  
    {  
        return new CustomerService();  
    });
```

At this point:

```
</> Plain text
```

```
CustomerService object  
    ✖ not created yet
```

Then:

```
</> C#
```

```
var x = service.Value;
```

Now:

```
</> Plain text
```

```
service.Value  
    ↓  
CustomerService created
```

Subsequent calls:

```
</> C#
```

```
service.Value
```

return the same initialized value.

Why use it?

Useful when:

- initialization is expensive
- you may not need the object
- you want initialization to happen only when required

Important

Lazy<T> is **not** simply "create an object later."

It also provides controlled lazy initialization and has thread-safety behavior appropriate for its default mode.

Interview answer

Lazy<T> provides lazy initialization, meaning the value is created only when value is first accessed.

7. Generic Math / Static Abstract Interface Members

This is one of the more advanced modern C# features.

The problem:

Suppose you want a generic method that adds numbers.

You might want:

</> Plain text

```
int
long
double
decimal
```

But traditionally, generic C# couldn't simply say:

</> C#

```
T Add<T>(T a, T b)
{
    return a + b;
}
```

because the compiler didn't know that arbitrary T supports + .

Static abstract interface members

Modern C# allows interfaces to define static abstract members.

Conceptually:

</> C#

```
interface ICalculator<T>
{
```

</> C#

```
static abstract T Add(T a, T b);
}
```

A type implementing the interface must provide the static member.

This allows generic code to work with operations that are normally static.

Generic Math

.NET provides interfaces such as:

</> Plain text

```
INumber<T>
IAdditionOperators<TSelf, TOther, TResult>
IMultiplyOperators<...>
```

So you can write generic numeric algorithms.

Conceptually:

</> C#

```
T Add<T>(T x, T y)
    where T : INumber<T>
{
    return x + y;
}
```

Now the same generic method can work with numeric types that satisfy the constraint:

</> Plain text

```
int
long
float
double
decimal
...
```

Why is this powerful?

Before static abstract interface members, generic code had difficulty expressing:

"T must support the + operator."

Now the interface constraint can express that capability.

Interview answer

Static abstract interface members allow interfaces to define static contracts that implementing types must provide. They enable generic algorithms to use operators and other static members, which is the foundation of .NET Generic Math.

8. Extension Members

This is the newest/most advanced item in your list.

Traditional C# has **extension methods**.

For example:

```
</> C#

public static class StringExtensions
{
    public static bool IsLong(this string value)
    {
        return value.Length > 10;
    }
}
```

Then:

```
</> C#

string name = "Khaled Ibrahim";

bool result = name.IsLong();
```

Even though `IsLong()` isn't actually declared inside `System.String`.

The compiler treats it roughly like:

```
</> C#

StringExtensions.IsLong(name);
```

Extension members

Modern C# goes beyond extension methods with the newer **extension members** feature.

The idea is that an extension can provide additional members for an existing type, not just methods.

Conceptually, extension members can include things such as:

</> Plain text

```
extension methods
extension properties
extension indexers
```

This allows APIs to feel more naturally integrated with an existing type without modifying that type's source code.

Important for interviews

If you're preparing for a typical **Junior .NET interview**, know **extension methods** very well first.

You should understand:

```
</> C#

public static ReturnType MethodName(
    this Type parameter)
```

The `this` on the first parameter is what makes it an extension method.

Example:

```
</> C#

public static class IntExtensions
{
    public static bool IsEven(this int number)
    {
        return number % 2 == 0;
    }
}
```

Then:

```
</> C#
```

10/3/26, 7:48 PMC# Recap

```
int x = 10;

x.IsEven();
```

Final cheat sheet

Feature	Main idea
Target-typed new	Infer type from the target
Discards _	Ignore a value intentionally
nameof	Get a symbol's name as a string
^	Index from the end
..	Select a range
Tuples	Group multiple values without a dedicated type
Lazy<T>	Initialize a value only when needed
Static abstract interface members	Interfaces can define static contracts
Generic Math	Generic algorithms can work with numeric operators
Extension members	Add members to existing types without modifying them

The ones I'd prioritize for your .NET interview

High priority:

- nameof
- Discards
- Tuples
- Ranges & indices
- Target-typed new
- Extension methods

Know conceptually:

- Lazy<T>
- Static abstract interface members / Generic Math

10/3/26, 7:48 PMC# Recap

Modern/advanced:

9. Extension members

Pattern Matching (type, property, relational, list, switch expressions) Local Functions

- Static local functions

These are very important modern C# features, especially **pattern matching**, because you'll see them frequently in APIs, validation, and business logic.

1. Pattern Matching

Pattern matching lets you ask:

"Does this value have a particular shape/type/value, and if so, give me access to it?"

The main forms you mentioned are:

- Type patterns
- Property patterns
- Relational patterns
- List patterns
- Switch expressions

2. Type Pattern

Checks the **runtime type** of an object.

```
</> C#

object value = "Hello";

if (value is string)
{
    Console.WriteLine("It's a string");
}
```

But the really useful part is that you can **capture the value**:

```
</> C#

if (value is string text)
{
```

```
Console.WriteLine(text.Length);
}
```

Here:

```
</> Plain text

value
↓
Is it a string?
↓
Yes
↓
text refers to that string
```

You can also combine conditions:

```
</> C#

if (value is string text && text.Length > 5)
{
    Console.WriteLine(text);
}
```

is vs casting

Traditional:

```
</> C#

if (value is string)
{
    string text = (string)value;
}
```

Pattern matching:

```
</> C#

if (value is string text)
{
    // use text
}
```

Cleaner and avoids a separate cast.

3. Property Pattern

Property patterns check an object's **properties**.

Suppose:

```
</> C#

class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

You can write:

```
</> C#

if (person is { Age: >= 18 })
{
    Console.WriteLine("Adult");
}
```

You're essentially asking:

```
</> Plain text

Is person:
    an object
    whose Age >= 18?
```

Multiple properties

```
</> C#

if (person is { Name: "Khaled", Age: 26 })
{
    Console.WriteLine("Match");
}
```

Nested properties

Suppose:

```
</> C#

class Person
{
    public Address Address { get; set; }
}

class Address
{
    public string City { get; set; }
}
```

You can do:

```
</> C#

if (person is { Address.City: "Menoufiya" })
{
    Console.WriteLine("From Menoufiya");
}
```

Very useful for checking object structure without writing multiple conditions.

4. Relational Patterns

These compare values using:

```
</> Plain text

>
>=
<
<=
```

Example:

```
</> C#

if (age is >= 18)
{
    Console.WriteLine("Adult");
}
```

Instead of:

```
</> C#

if (age >= 18)
```

The advantage becomes more obvious when combined with patterns.

```
</> C#

if (age is >= 18 and < 60)
{
    Console.WriteLine("Working age");
}
```

You can also use or :

```
</> C#

if (age is < 18 or >= 60)
{
    Console.WriteLine("Outside working age");
}
```

So:

```
</> Plain text

and → both patterns must match
or → at least one must match
not → pattern must NOT match
```

Example:

```
</> C#

if (age is not 18)
{
    ...
}
```

5. Switch Expression

Traditional switch :

```
</> C#

string GetRoleName(string role)
{
    switch (role)
    {
        case "Admin":
            return "Administrator";

        case "User":
            return "Normal User";

        default:
            return "Unknown";
    }
}
```

A switch expression makes this much shorter:

```
</> C#

string GetRoleName(string role)
{
    return role switch
    {
        "Admin" => "Administrator",
        "User" => "Normal User",
        _ => "Unknown"
    };
}
```

Notice:

```
</> Plain text

_ => "Unknown"
```

_ means:

Anything else.

6. Switch + Type Patterns

This is where pattern matching becomes powerful.

```
</> C#

string Describe(object value)
{
    return value switch
    {
        int x => $"Integer: {x}",
        string s => $"String: {s}",
        bool b => $"Boolean: {b}",
        _ => "Unknown"
    };
}
```

For:

```
</> C#

Describe(10)
```

→

```
</> Plain text

Integer: 10
```

For:

```
</> C#

Describe("Hello")
```

→

```
</> Plain text

String: Hello
```

7. Switch + Property Patterns

You can combine them:

```
</> C#

string GetStatus(Person person)
{
    return person switch
    {
        { Age: < 18 } => "Minor",
        { Age: >= 18 } => "Adult",
        _ => "Unknown"
    };
}
```

Or:

```
</> C#

return person switch
{
    { Age: >= 18, Name: "Khaled" } => "Khaled is an adult",
    { Age: >= 18 } => "Adult",
    _ => "Minor"
};
```

8. List Patterns

List patterns let you match the **structure/content of collections**.

```
</> C#

int[] numbers = { 1, 2, 3 };
```

You can check:

```
</> C#

if (numbers is [1, 2, 3])
{
    Console.WriteLine("Exact match");
}
```

This means:

```
</> Plain text
```

```
[1, 2, 3]
  ↑ ↑ ↑
must match exactly
```

Match first element

```
</> C#

if (numbers is [1, ..])
{
    Console.WriteLine("Starts with 1");
}
```

.. means:

zero or more remaining elements.

Match last element

```
</> C#

if (numbers is [.., 3])
{
    Console.WriteLine("Ends with 3");
}
```

Match first and last

```
</> C#

if (numbers is [1, .., 3])
{
    Console.WriteLine("Starts with 1 and ends with 3");
}
```

This is different from the **range operator** we discussed earlier.

Here:

```
</> Plain text
```

```
[1, .., 3]
```

is a **list pattern**.

Whereas:

```
</> C#
```

```
numbers[1..3]
```

is a **range expression**.

9. Local Functions

A local function is a method declared **inside another method**.

Example:

```
</> C#
```

```
void ProcessOrder()
{
    CalculateTax();

    void CalculateTax()
    {
        Console.WriteLine("Calculating tax...");
    }
}
```

`CalculateTax()` exists only inside `ProcessOrder()` .

It isn't a member of the class.

Why use local functions?

Useful when a small piece of logic:

- is needed only by one method
- shouldn't be exposed as a class method
- makes the main method easier to read

Example:

```
</> C#
```

```
int CalculateTotal(int price, int quantity)
{
    int CalculateSubtotal()
    {
        return price * quantity;
    }

    return CalculateSubtotal() + 10;
}
```

The local function can access variables from the containing method:

```
</> Plain text
```

```
CalculateTotal
|
├─ price
├─ quantity
└─ CalculateSubtotal()
    |
    └─ price
        └─ quantity
```

This is called a **closure/capturing local state** when applicable.

10. Static Local Functions

A local function can be declared `static` :

```
</> C#
```

```
int CalculateTotal(int price, int quantity)
{
    return Multiply(price, quantity);

    static int Multiply(int x, int y)
    {
        return x * y;
    }
}
```

The important difference:

Normal local function

```
</> C#  
  
int Multiply(int x, int y)  
{  
    return x * y;  
}
```

can access variables from the containing method:

```
</> C#  
  
int CalculateTotal(int price, int quantity)  
{  
    int Multiply(int x)  
    {  
        return x * price; // can access price  
    }  
  
    return Multiply(quantity);  
}
```

Static local function

```
</> C#  
  
static int Multiply(int x, int y)  
{  
    return x * y;  
}
```

cannot capture variables from the containing method.

So this is invalid:

```
</> C#  
  
int CalculateTotal(int price)  
{  
    static int Multiply(int x)  
    {  
        return x * price; // ✖  
    }  
}
```

```
    return Multiply(10);  
}
```

Because `price` belongs to the enclosing method.

Why use `static` ?

It communicates:

"This local function is completely independent of the surrounding method's local variables/state."

It can also avoid unnecessary closure/capture behavior.

Pattern Matching Cheat Sheet

Pattern	Example	Meaning
Type	<code>x is string s</code>	Is <code>x</code> a string?
Property	<code>x is { Age: 18 }</code>	Does Age equal 18?
Relational	<code>x is >= 18</code>	Is <code>x ≥ 18</code> ?
and	<code>x is >= 18 and < 60</code>	Both conditions
or	<code>x is < 18 or > 60</code>	Either condition
not	<code>x is not null</code>	Doesn't match
List	<code>x is [1, 2, 3]</code>	Exact list structure
List + ..	<code>x is [1, ..]</code>	Starts with 1
Switch expression	<code>x switch { ... }</code>	Match and return result

Interview answer

Pattern matching allows C# to test a value's type, properties, relationships, or collection structure and optionally extract values at the same time. It is commonly used with `is` and

switch expressions.

And for local functions:

A local function is a method declared inside another method. A static local function cannot capture variables from its enclosing method.

These two topics are especially worth practicing through code-tracing questions, because interviewers can combine type + property + relational patterns in one switch .

- SortedList, SortedDictionary, SortedSet
- Concurrent Collections
- Immutable Collections

These are important collection types in .NET, especially when an interviewer asks "Why would you use this collection instead of Dictionary/List?"

1. SortedList<TKey,TValue>

SortedList stores key-value pairs sorted by key.

</> C#

```
SortedList<int, string> students = new()
{
    { 3, "Ahmed" },
    { 1, "Khaled" },
    { 2, "Omar" }
};
```

When enumerated:

</> Plain text

```
1 → Khaled
2 → Omar
3 → Ahmed
```

Important characteristics

Internally, SortedList is based on arrays.

</> Plain text

Keys: [1] [2] [3] [4] ...

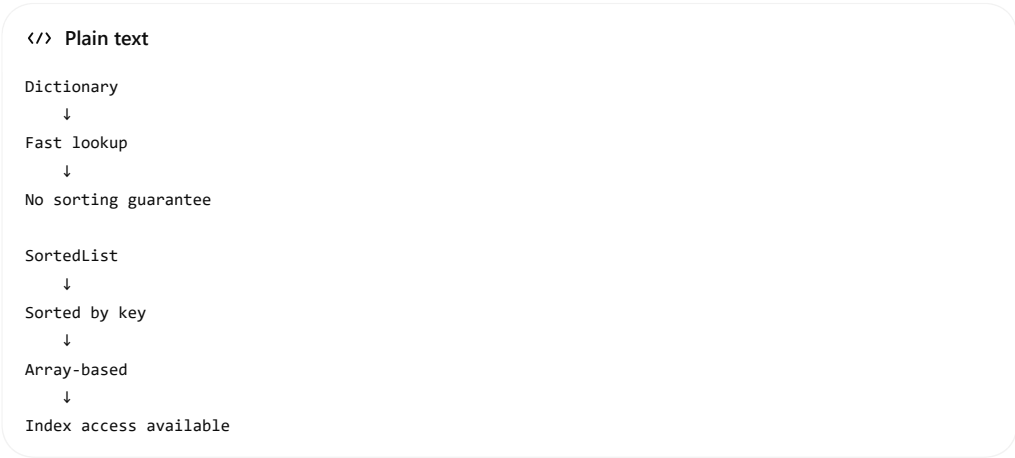
Values: [A] [B] [C] [D] ...

Therefore it supports:

- Key lookup
- Index-based access
- Sorted iteration
- Relatively memory-efficient storage

But inserting/removing in the middle can require shifting elements.

SortedList vs Dictionary



When useful?

When you need:

Sorted key-value data + occasional index-based access.

2. SortedDictionary<TKey,TValue>

It also stores key-value pairs sorted by key:

</> C#

```
SortedList<int, string> students = new()
{
    { 3, "Ahmed" },
    { 1, "Khaled" },
}
```

10/3/26, 7:48 PMC# Recap

```
{ 2, "Omar" }  
};
```

Enumeration:

</> Plain text

```
1 → Khaled  
2 → Omar  
3 → Ahmed
```

But unlike SortedList , it is implemented using a **balanced tree** structure.

Conceptually:

</> Plain text

```
      2  
     / \  
    1   3
```

Therefore insertion/removal doesn't require shifting an array.

Complexity

Typical operations:

</> Plain text

```
Search      O(log n)  
Insert      O(log n)  
Remove      O(log n)
```

Compare with Dictionary :

</> Plain text

```
Dictionary lookup  O(1) average  
SortedDictionary  O(log n)
```

So don't use SortedDictionary simply because it's "better."

You're trading some lookup performance for **maintaining sorted order**.

10/3/26, 7:48 PMC# Recap

3. SortedList vs SortedDictionary

This is a very common interview question.

	SortedList	SortedDictionary
Sorted by key	✓	✓
Underlying structure	Arrays	Balanced tree
Lookup	O(log n)	O(log n)
Insert	O(n) generally	O(log n)
Remove	O(n) generally	O(log n)
Index access	✓	✗
Memory	Generally lower	Generally higher
Best when	Less modification / index access	Frequent insert/remove

Easy memory trick

SortedList = sorted arrays

SortedDictionary = sorted tree

4. SortedSet<T>

SortedSet<T> is like HashSet<T> , but it keeps elements **sorted**.

</> C#

```
SortedSet<int> numbers = new()  
{  
    50,  
    10,  
    30,  
    20  
};
```

Enumeration:

```
</> Plain text

10
20
30
50
```

And duplicates are automatically ignored:

```
</> C#

numbers.Add(30);
```

The set still contains:

```
</> Plain text

10, 20, 30, 50
```

HashSet vs SortedSet

```
</> Plain text

HashSet
↓
Unique
↓
Fast average lookup O(1)
↓
No sorted ordering

SortedSet
↓
Unique
↓
Sorted
↓
O(log n)
```

Use SortedSet when you need:

Uniqueness + sorted order.

5. Concurrent Collections

Now we're talking about thread-safe collections designed for concurrent access.

Namespace:

```
</> C#

System.Collections.Concurrent
```

Examples:

```
</> Plain text

ConcurrentDictionary<TKey,TValue>
ConcurrentQueue<T>
ConcurrentStack<T>
ConcurrentBag<T>
BlockingCollection<T>
```

Why do we need them?

Imagine two threads accessing a normal Dictionary :

```
</> Plain text

Thread 1 ┌───┐
          │   │
Thread 2 ┌───┐─> Dictionary
```

If both modify it simultaneously, you can get race conditions and corrupted/inconsistent behavior.

You could manually use:

```
</> C#

lock (...)
{
    dictionary.Add(...);
}
```

But .NET provides concurrent collections designed specifically for these scenarios.

6. ConcurrentDictionary

Probably the most important one for interviews.

</> C#

```
ConcurrentDictionary<int, string> users = new();

users.TryAdd(1, "Khaled");

users.TryGetValue(1, out string? name);

users.TryRemove(1, out _);
```

It supports concurrent operations safely.

Useful methods include:

</> Plain text

```
TryAdd
TryGetValue
TryRemove
TryUpdate
GetOrAdd
AddOrUpdate
```

For example:

</> C#

```
users.GetOrAdd(1, id => LoadUser(id));
```

This is useful when multiple threads may access/update the dictionary.

Important distinction

Don't say:

"ConcurrentDictionary makes every operation magically atomic."

Instead:

It provides thread-safe concurrent operations and specialized atomic methods such as `TryAdd` , `GetOrAdd` , and `AddOrUpdate` .

7. ConcurrentQueue<T>

Thread-safe FIFO queue.

</> Plain text

```
First In
↓
A → B → C
↓
First Out
```

Example:

</> C#

```
ConcurrentQueue<string> queue = new();

queue.Enqueue("A");
queue.Enqueue("B");

queue.TryDequeue(out string? item);
```

Useful for producer/consumer scenarios.

8. ConcurrentStack<T>

Thread-safe LIFO collection.

</> Plain text

```
C ← top
B
A
```

</> C#

```
ConcurrentStack<int> stack = new();

stack.Push(10);
stack.Push(20);

stack.TryPop(out int value);
```

9. ConcurrentBag<T>

An unordered thread-safe collection optimized for scenarios where multiple threads frequently add/remove items.

```
</> C#

ConcurrentBag<int> bag = new();

bag.Add(10);
bag.Add(20);

bag.TryTake(out int value);
```

Important:

Don't use ConcurrentBag when you need ordering.

10. Immutable Collections

Namespace:

```
</> C#

System.Collections.Immutable
```

The key idea is:

Once an immutable collection is created, you don't modify that collection.

For example:

```
</> C#

ImmutableList<int> numbers =
    ImmutableList.Create(1, 2, 3);
```

You can't do:

```
</> C#

numbers.Add(4);
```

and expect the existing collection to change.

Instead:

```
</> C#

var newNumbers = numbers.Add(4);
```

Now:

```
</> Plain text

numbers
  ↓
[1, 2, 3]

newNumbers
  ↓
[1, 2, 3, 4]
```

The original remains unchanged.

Why is this useful?

Immutability is useful when you want data that can safely be shared without worrying about someone modifying the original collection.

For example:

```
</> Plain text

Thread A ┌
          │
          ▼
        immutable
        collection
          ▲
          │
Thread B └
```

Neither thread can modify the original collection.

This can simplify concurrent programming.

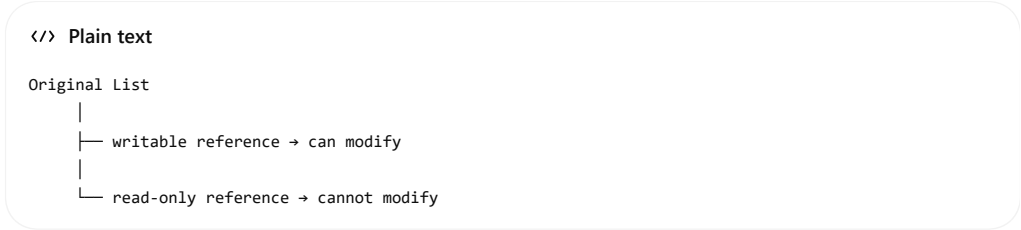
11. Important: Immutable ≠ ReadOnly

This distinction is frequently tested.

ReadOnly

A read-only wrapper can prevent **you** from modifying through that reference, but the underlying collection may still change elsewhere.

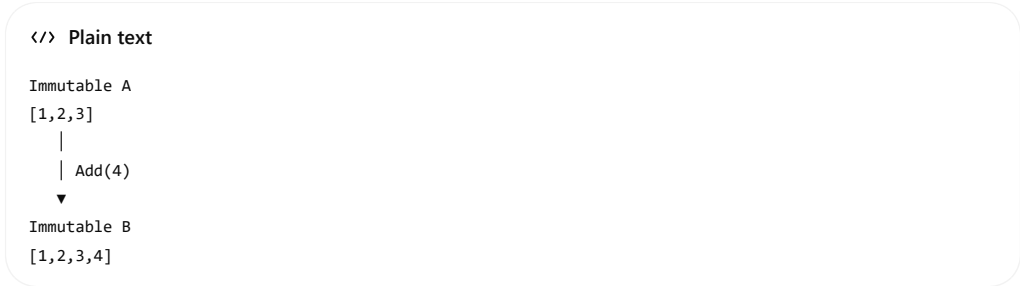
Conceptually:



Immutable

The collection itself cannot be modified.

Instead, operations produce a new immutable collection/version.



So:

Read-only = can't modify through this view.

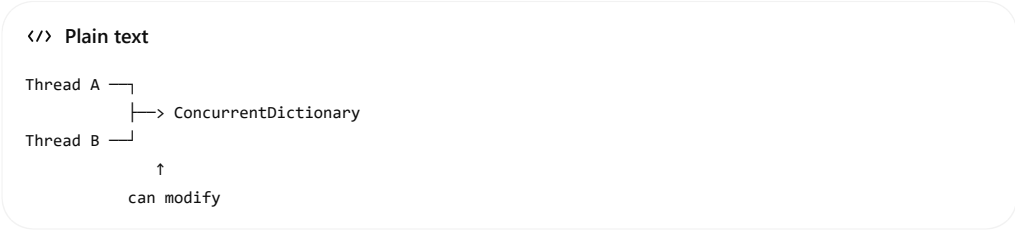
Immutable = the collection itself doesn't change.

12. Immutable vs Concurrent

These solve different problems.

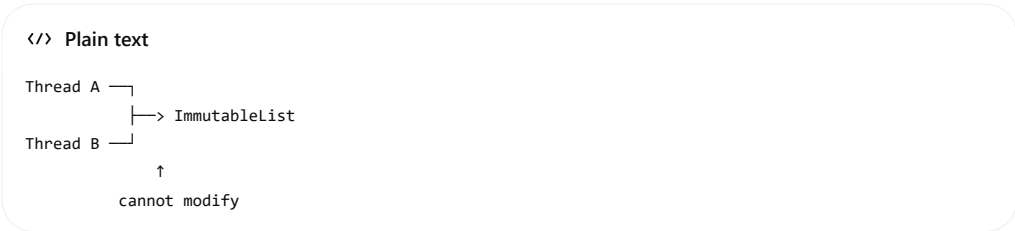
Concurrent collections

Allow multiple threads to modify the collection safely.



Immutable collections

Allow multiple threads to safely **share unchanged versions**.



If you need updates, you create a new version.

13. Big Interview Comparison

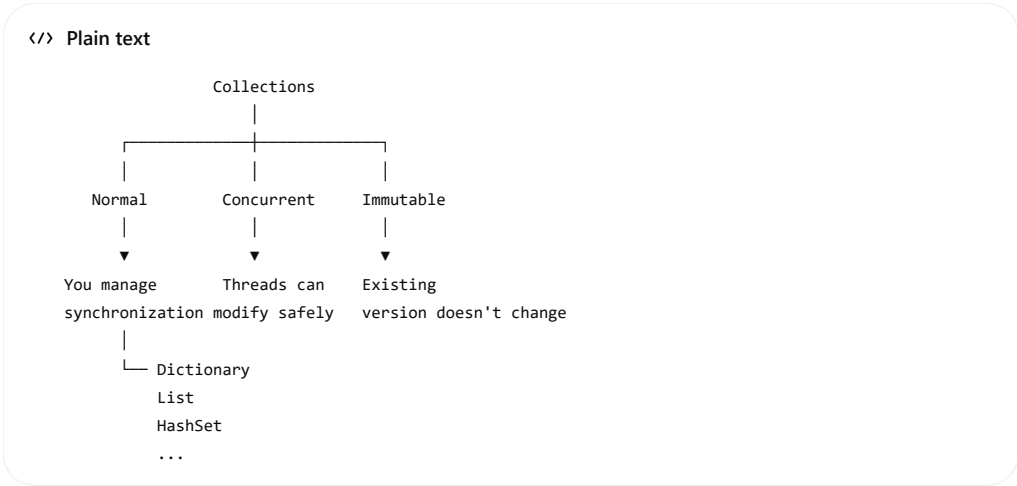
Collection	Sorted	Unique	Thread-safe	Main structure/use
Dictionary	✗	Keys unique	✗	Fast key lookup
HashSet	✗	✓	✗	Fast unique values
SortedList	✓	Keys unique	✗	Sorted key/value + index
SortedDictionary	✓	Keys unique	✗	Sorted key/value + frequent changes
SortedSet	✓	✓	✗	Sorted unique values
ConcurrentDictiona ry	✗	Keys unique	✓	Concurrent key/value
ConcurrentQueue	✗	✗	✓	Concurrent FIFO

10/3/26, 7:48 PM

C# Recap

Collection	Sorted	Unique	Thread-safe	Main structure/use
ConcurrentStack	✗	✗	✓	Concurrent LIFO
ConcurrentBag	✗	✗	✓	Concurrent unordered items
ImmutableList	✗	✗	Naturally shareable	Never changes
ImmutableHashSet	✗	✓	Naturally shareable	Immutable unique values
ImmutableDictionary	✗	Keys unique	Naturally shareable	Immutable key/value

The mental model



Interview answers to memorize

SortedList vs SortedDictionary?

Both maintain key order. `SortedList` is array-based and supports index access, while `SortedDictionary` is tree-based and is generally better for frequent insertions and removals.

HashSet vs SortedSet?

10/3/26, 7:48 PM

C# Recap

Both store unique values. `HashSet` is optimized for fast lookup, while `SortedSet` maintains sorted order with typically $O(\log n)$ operations.

Why ConcurrentDictionary?

To safely perform concurrent dictionary operations from multiple threads without manually synchronizing every access.

Immutable collection?

An immutable collection cannot be modified after creation. Operations that appear to modify it return a new collection instead, allowing safe sharing of existing versions.

Today 7:45 PM

- Tuples & Deconstruction
- Anonymous Types

Sure. These are closely related C# features and commonly asked in interviews.

1. Tuples

A **tuple** lets you group multiple values together without creating a separate class/struct.

</> C#

```
var person = ("Khaled", 26);
```

The tuple contains two values:

</> Plain text

```
person
├── Item1 → "Khaled"
└── Item2 → 26
```

You can access them:

</> C#

```
Console.WriteLine(person.Item1); // Khaled
Console.WriteLine(person.Item2); // 26
```

Named tuples

You can give the elements meaningful names:

```
</> C#  
  
var person = (Name: "Khaled", Age: 26);  
  
Console.WriteLine(person.Name);  
Console.WriteLine(person.Age);
```

This is much clearer than `Item1` and `Item2` .

Returning multiple values

One of the most useful applications is returning multiple values from a method:

```
</> C#  
  
(string Name, int Age) GetPerson()  
{  
    return ("Khaled", 26);  
}
```

Then:

```
</> C#  
  
var person = GetPerson();  
  
Console.WriteLine(person.Name);  
Console.WriteLine(person.Age);
```

This is useful when you need to return a few related values without creating a dedicated DTO/class.

2. Deconstruction

Deconstruction means taking the individual values from a tuple and assigning them to separate variables.

Instead of:

```
</> C#
```

```
var person = GetPerson();  
  
var name = person.Name;  
var age = person.Age;
```

You can do:

```
</> C#  
  
var (name, age) = GetPerson();
```

Now:

```
</> Plain text  
  
(name, age)  
  ↓  
("Khaled", 26)
```

So:

```
</> C#  
  
Console.WriteLine(name); // Khaled  
Console.WriteLine(age);  // 26
```

Named tuple names don't control deconstruction

For example:

```
</> C#  
  
var person = (Name: "Khaled", Age: 26);  
  
var (x, y) = person;
```

`x` receives `"Khaled"` and `y` receives `26` .

Deconstruction is based on **position**, not the tuple element names.

Discards with deconstruction

If you don't need one of the values:

```
</> C#

var (name, _) = GetPerson();
```

The `_` means:

"I don't care about this value."

This connects directly to the **discards** feature we discussed earlier.

3. Deconstructing custom types

Deconstruction isn't limited to tuples.

A class can define a `Deconstruct` method:

```
</> C#

class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public void Deconstruct(out string name, out int age)
    {
        name = Name;
        age = Age;
    }
}
```

Then:

```
</> C#

var person = new Person
{
    Name = "Khaled",
    Age = 26
};

var (name, age) = person;
```

The compiler uses the `Deconstruct` method.

Interview point

Deconstruction allows an object to expose multiple values that can be assigned to separate variables.

4. Tuple vs Class/Record

This is an important interview distinction.

Tuple

Good for a **small temporary grouping of values**:

```
</> C#

(string Name, int Age)
```

Class/Record

Better when the data represents an important domain concept:

```
</> C#

class Customer
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Think:

```
</> Plain text

Tuple
→ temporary/simple grouping

Class / Record
→ meaningful reusable data model
```

For example, returning `(bool Success, string Message)` from a small method can be reasonable, while a complex API response is usually better represented by a DTO/record.

5. Anonymous Types

Example:

```
</> C#  
  
var person = new  
{  
    Name = "Khaled",  
    Age = 26  
};
```

You didn't create:

```
</> C#  
  
class Person
```

but C# creates an anonymous type behind the scenes.

You can access:

```
</> C#  
  
person.Name  
person.Age
```

The type is strongly typed even though **you didn't give the type a name**.

Why are anonymous types commonly used?

They're extremely common with LINQ.

For example:

```
</> C#  
  
var result = customers.Select(c => new  
{  
    c.Id,  
    c.Name  
});
```

You're selecting only the properties you need.



This is especially useful for projections.

6. Anonymous Type vs Tuple

This is a common interview question.

	Tuple	Anonymous Type
Syntax	(Name, Age)	new { Name, Age }
Named type	No	No
Access	.Name / .Item1	.Name
Deconstruction	✔ Yes	✗ Not by default
Common use	Return multiple values	LINQ projections
Can be returned easily as public API type?	Yes	Not conveniently
Properties	Fields/properties depending on tuple form	Read-only properties

Example:

Tuple

```
</> C#  
  
var person = (Name: "Khaled", Age: 26);  
  
var (name, age) = person;
```

Anonymous type

```
</> C#  
  
var person = new  
{  
    Name = "Khaled",  
    Age = 26  
};  
  
Console.WriteLine(person.Name);
```

7. Important: var doesn't mean anonymous type

This is a common misunderstanding.

```
</> C#  
  
var person = new  
{  
    Name = "Khaled",  
    Age = 26  
};
```

var means:

Let the compiler infer the **actual compile-time type**.

It does **not** mean:

Make this dynamically typed.

The anonymous type is still strongly typed.

Interview answers

What is a tuple?

A tuple is a lightweight way to group multiple values, potentially of different types, into a single value. It is useful for returning or passing a small number of related values.

What is deconstruction?

Deconstruction extracts the individual values of a tuple or a type with a `Deconstruct` method into separate variables.

What is an anonymous type?

An anonymous type is a compiler-generated type without an explicit type name, commonly used for temporary data and LINQ projections.

Tuple vs anonymous type?

Tuples are convenient for grouping and returning multiple values and support deconstruction. Anonymous types are commonly used for temporary projections, especially in LINQ, and expose strongly typed read-only properties.

One important distinction

```
</> C#  
  
var x = (Name: "Khaled", Age: 26);           // Tuple  
var y = new { Name = "Khaled", Age = 26 }; // Anonymous type
```

Both look similar conceptually, but they are **different C# types/features**.

`(int, string)` is tuple a datatype

Yes — but there is an important distinction.

A **tuple is a type**, but "tuple" isn't one single concrete datatype like `int` or `string` .

For example:

```
</> C#  
  
(int, string)
```

is a **tuple type** consisting of:

```
</> Plain text
```

```
int + string
```

And:

```
</> C#  
  
(string Name, int Age)
```

is another tuple type.

So:

```
</> C#  
  
var person = ("Khaled", 26);
```

person has a tuple type equivalent to:

```
</> C#  
  
(string, int)
```

Under the hood

C# tuples are implemented using the generic `System.ValueTuple` types.

For example:

```
</> C#  
  
(string, int)
```

is essentially:

```
</> C#  
  
ValueTuple<string, int>
```

So tuples are **value types**.

That's an important interview point:

```
</> Plain text
```

```
Tuple  
↓  
ValueTuple<T1, T2, ...>  
↓  
struct  
↓  
value type
```

For example:

```
</> C#  
  
(string Name, int Age) person = ("Khaled", 26);
```

is a `ValueTuple<string, int>` value type.

Don't confuse it with `Tuple<T>`

There are actually two related concepts:

```
</> C#  
  
Tuple<string, int>           // old reference-type Tuple  
(string, int)               // modern ValueTuple
```

Modern C# normally uses:

```
</> C#  
  
(string, int)
```

which is based on `ValueTuple` and is a **struct/value type**.